Threat Modeling

Black Duck Software Security Training Series

Student Handbook

© 2024 Black Duck

CONFIDENTIAL INFORMATION

The information contained in this presentation is the confidential and proprietary information of Black Duck Software. You are not permitted to disseminate or use any of the information provided to you in this presentation outside of Black Duck Software
without prior written authorization.

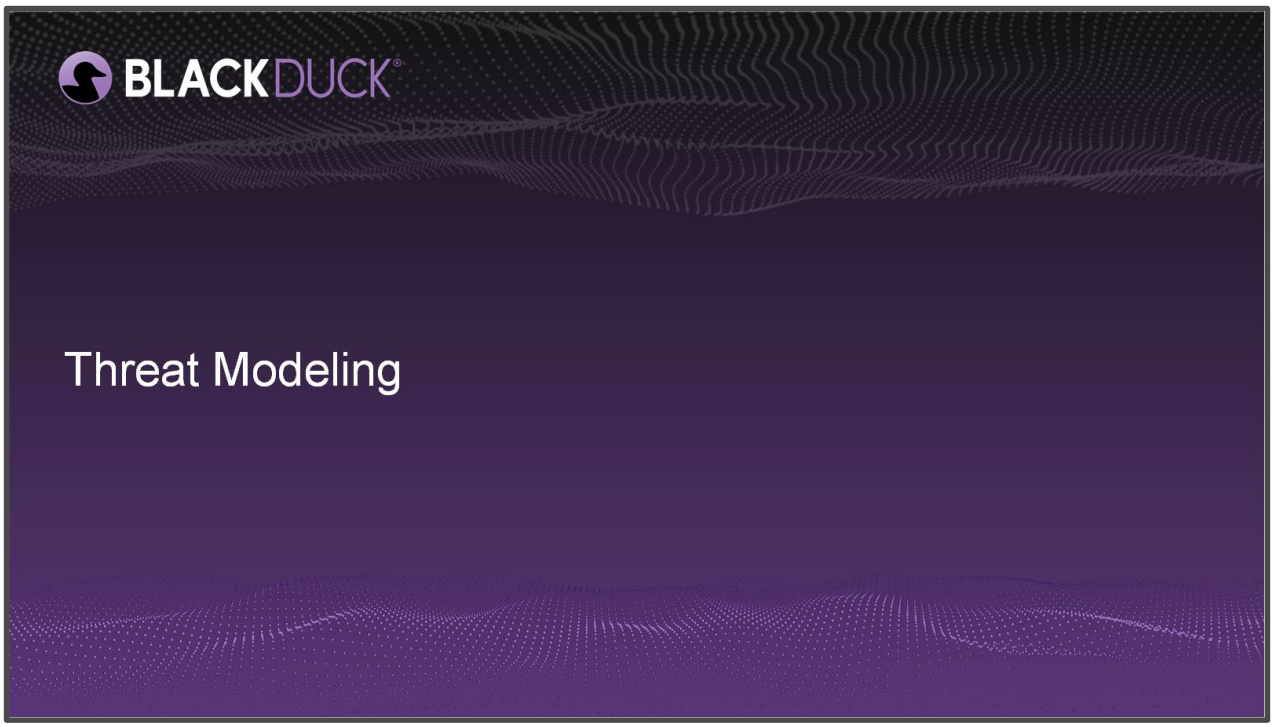
IMPORTANT NOTICE

In the event information in this presentation reflects Black Duck Software's future plans, such plans are as of the date of this presentation and are subject to change. Black Duck Software is not obligated to update this presentation or develop the products with the features and functionality discussed in this presentation. Additionally, Black Duck Software's services and products may only be offered and purchased pursuant to an authorized quote and purchase order or a mutually agreed upon written contract with Black Duck Software.

About Black Duck

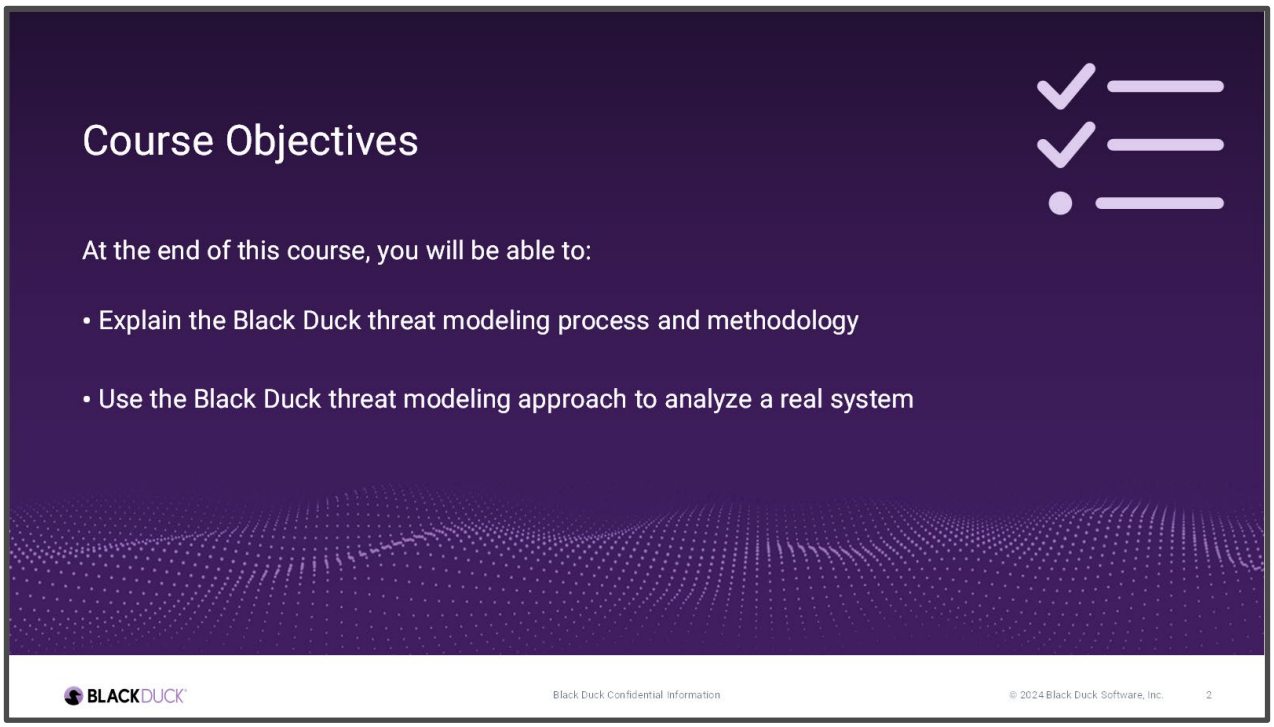
Build trust in your software

Black Duck® offers the most comprehensive, powerful, and trusted portfolio of application security solutions in the industry. We have an unmatched track record of helping organizations around the world secure their software quickly, integrate security efficiently in their development environments, and safely innovate with new technologies. As the recognized leaders, experts, and innovators in software security, Black Duck has everything you need to build trust in your software. Learn more at www.blackduck.com.

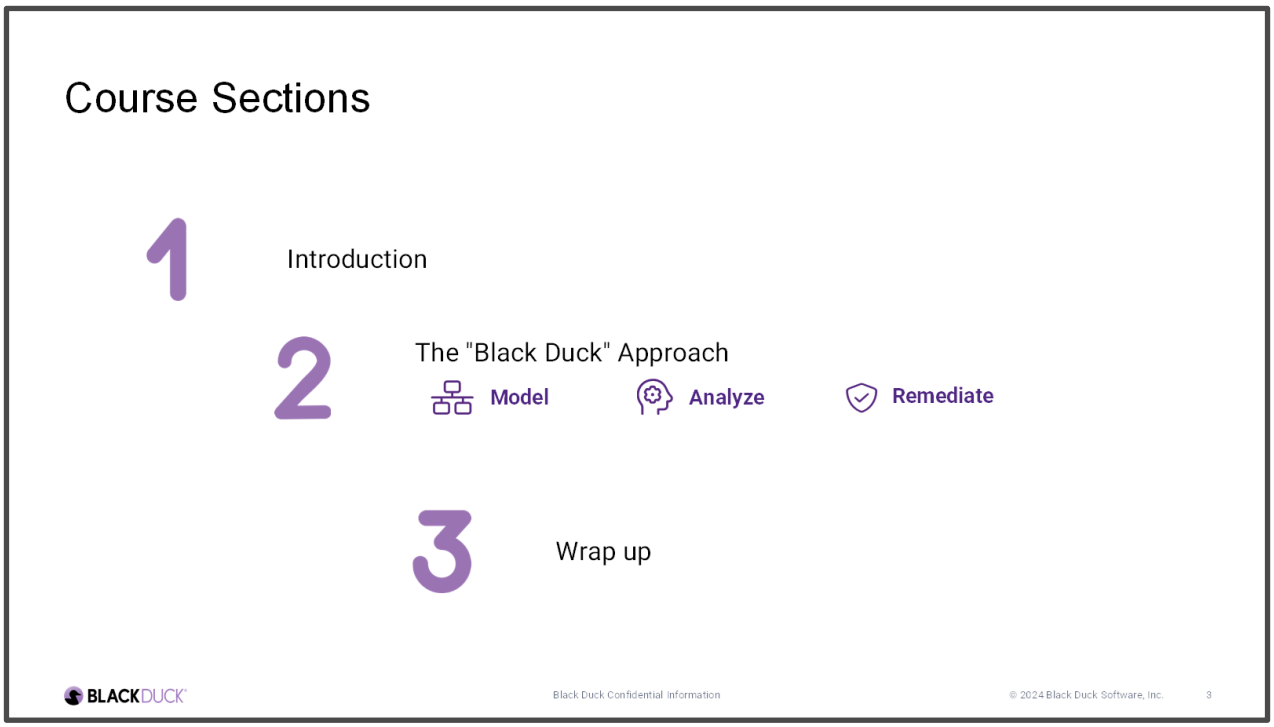


Slide 1

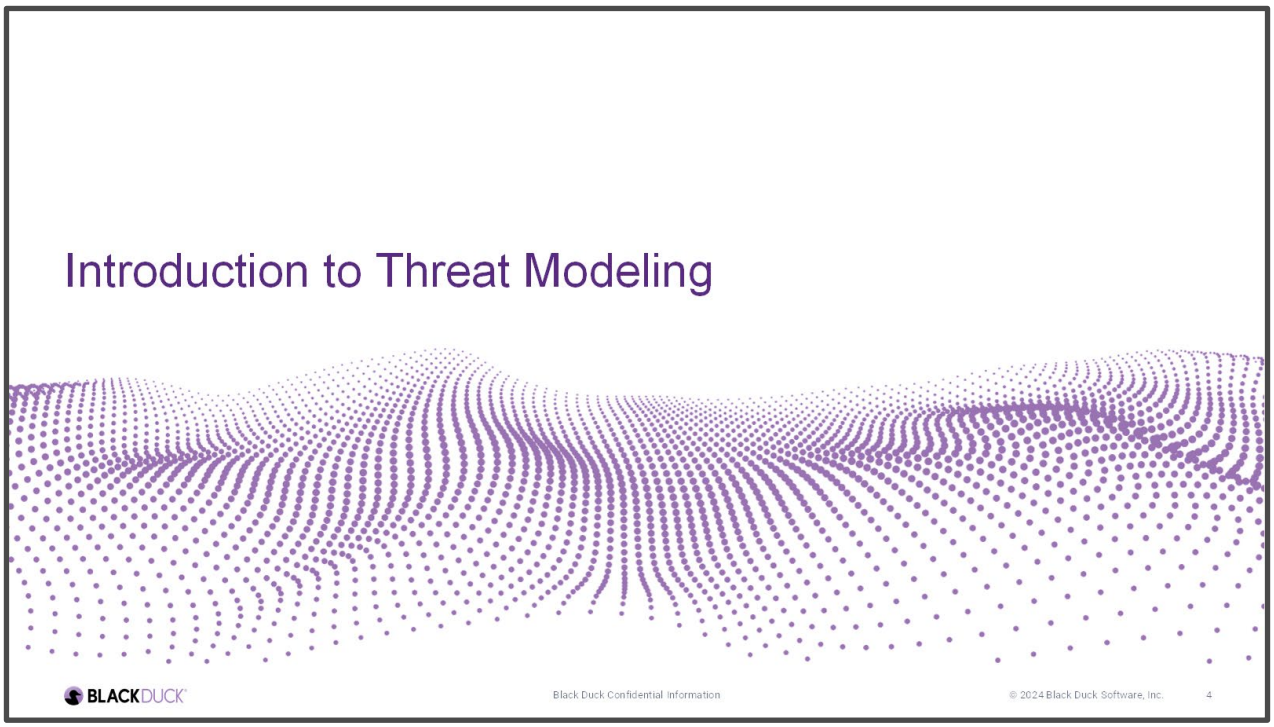
© 2024 Black Duck


Course Objectives
At the end of this course, you will be able to:
• Explain the Black Duck threat modeling process and methodology
• Use the Black Duck threat modeling approach to analyze a real system
BLACKDUCK
Black Duck Confidential Information
© 2024 Black Duck Software, Inc.
2

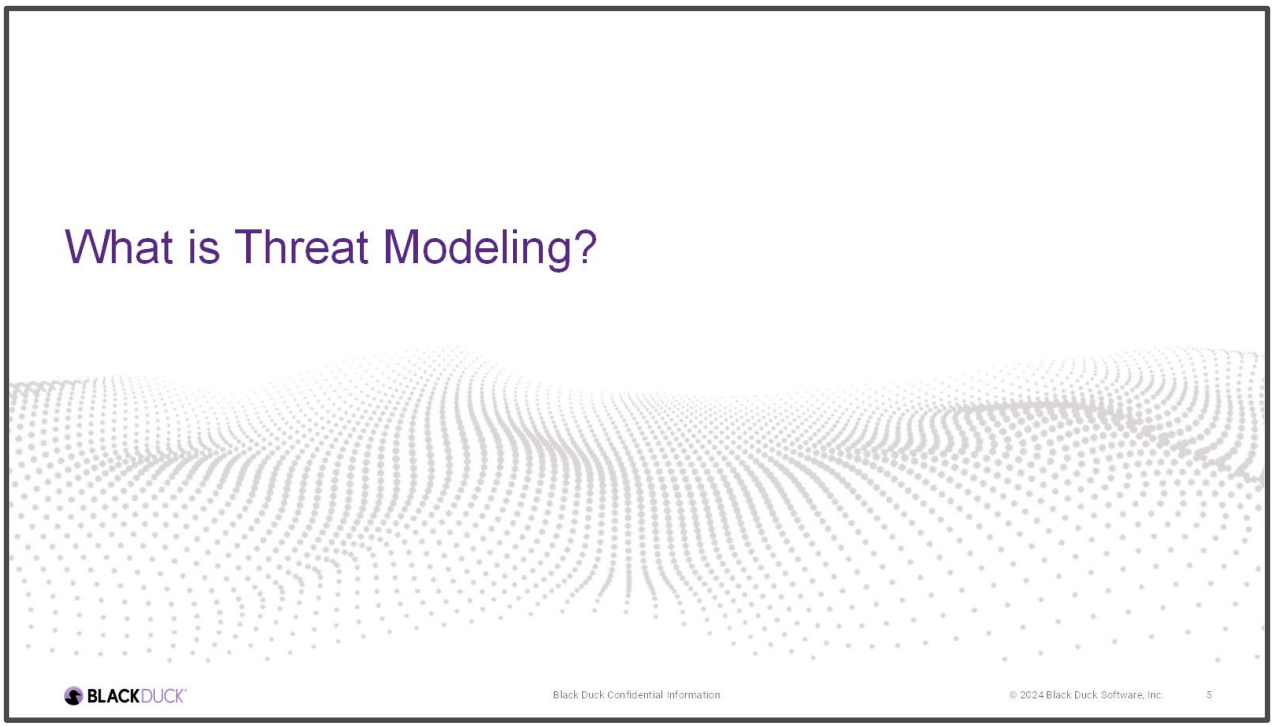
Slide 2


Course Sections
1
Introduction
2
The "Black Duck" Approach
Model
Analyze
Remediate
3
Wrap up
BLACKDUCK
Black Duck Confidential Information
© 2024 Black Duck Software, Inc.
3

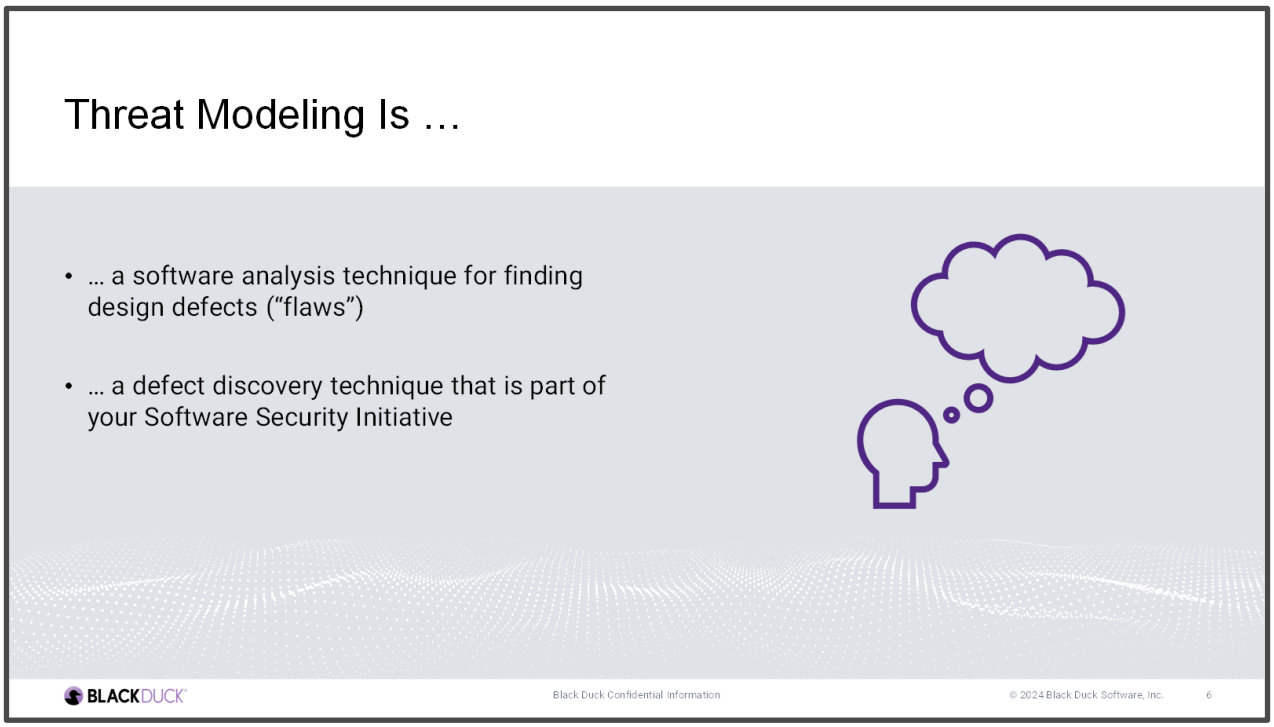
Slide 3


Introduction to Threat Modeling
BLACKDUCK
Black Duck Confidential Information
© 2024 Black Duck Software, Inc.
4

Slide 4


What is Threat Modeling?
BLACKDUCK
Black Duck Confidential Information
© 2024 Black Duck Software, Inc.
5

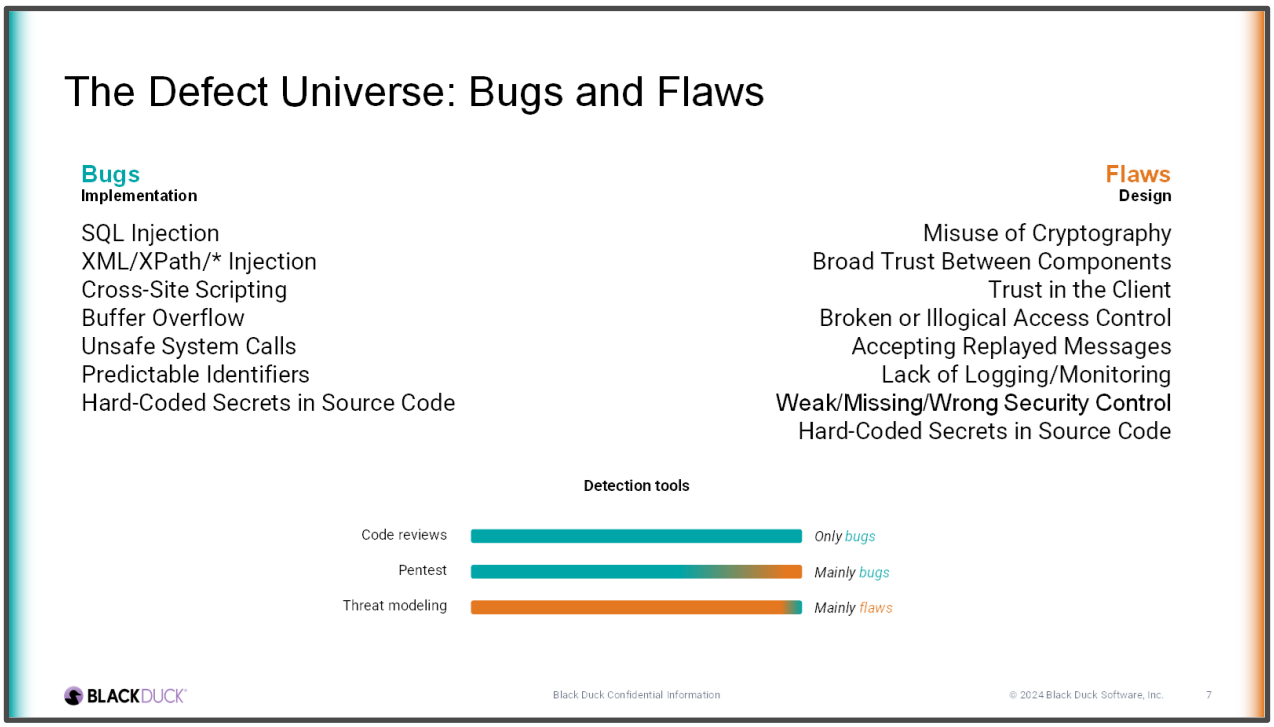
Slide 5



Slide 6

We use threat modeling to identify security weaknesses or flaws in software, such as missing, weak or inappropriate security controls. If a weakness is exploitable by a threat agent, it is a vulnerability. Threat modeling identifies weaknesses but does not evaluate exploitability. Therefore, the weaknesses that it finds may or may not be actual vulnerabilities. Subsequent activities, including penetration testing and code reviews, can evaluate the exploitability of the weaknesses found during threat modeling.

One example of a weakness is an application that does not perform output encoding of untrusted data. The application may or may not contain actual injection vulnerabilities depending on how good its input validation controls are. Threat modeling would simply point out the missing output encoding control as a weakness. Subsequent activities would then determine whether the input validation controls are sufficient to protect against the types of injection vulnerabilities that are relevant to the application.

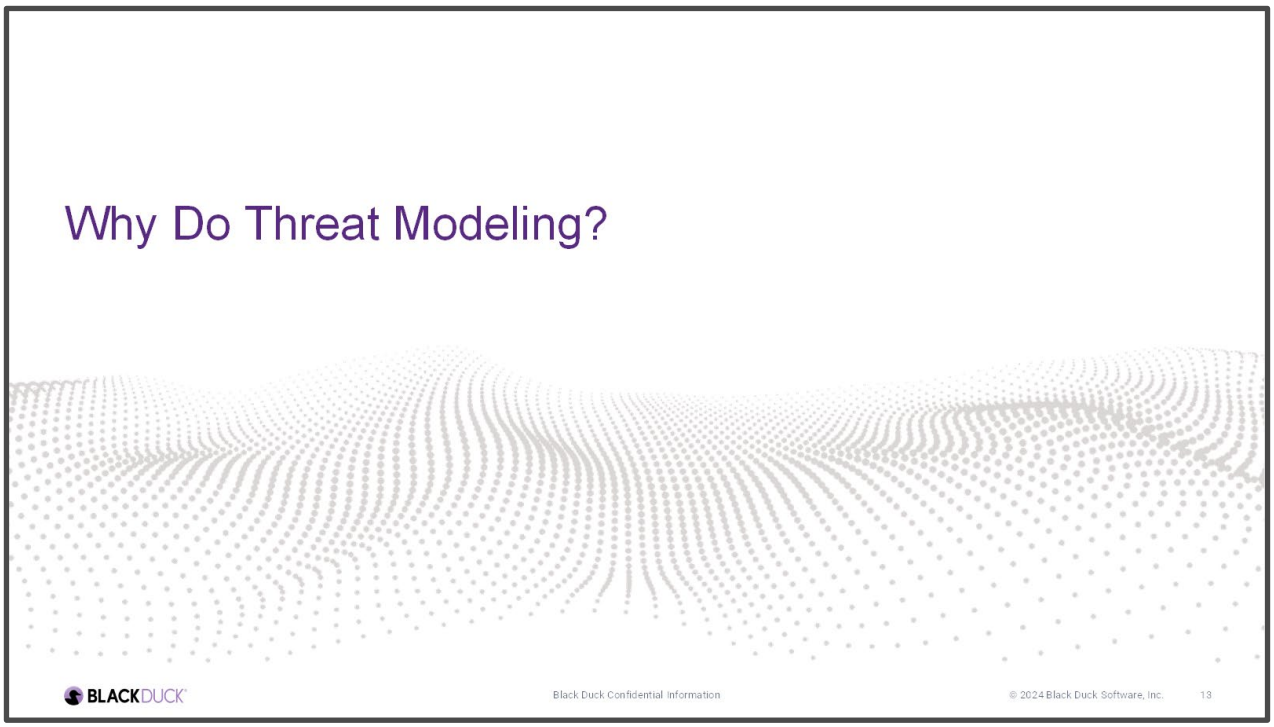


Slide 7

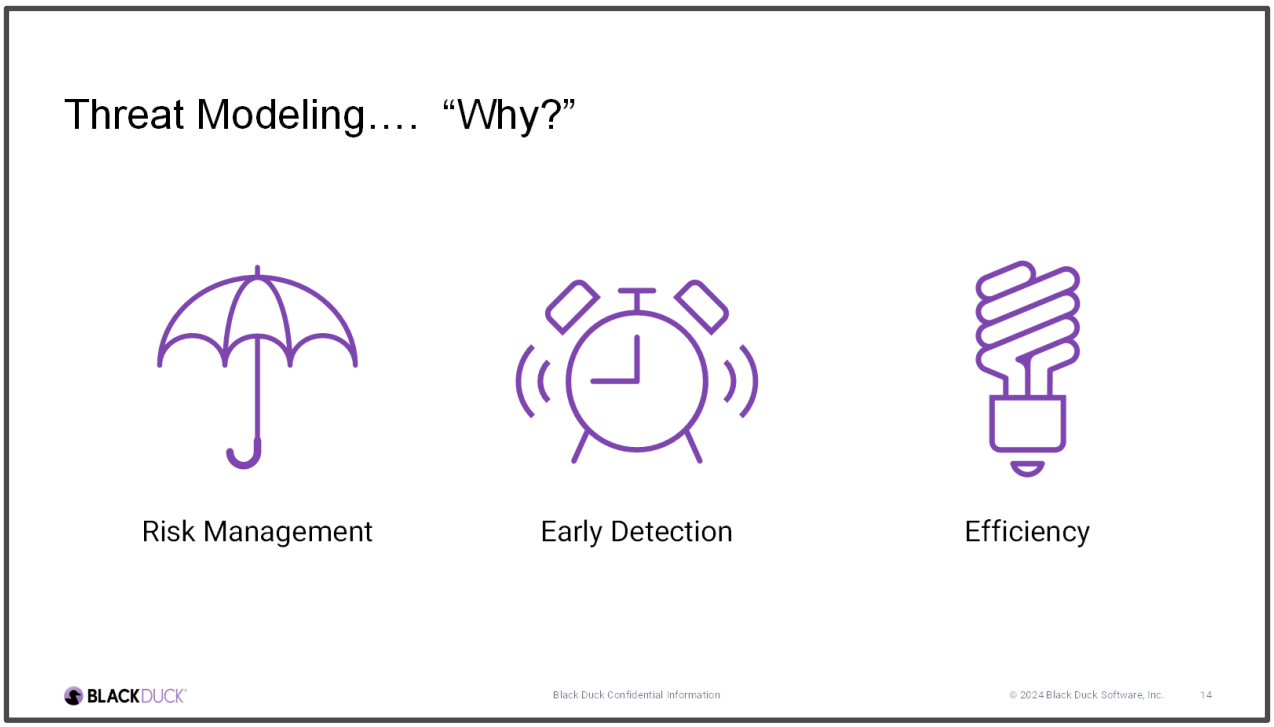
A common word used to think about software issues is "bug". What differentiates a bug from a flaw?

Bugs are usually deviations from the specification–issues in the implementation. Flaws, in contrast, are found in the design. Some issues can be caught in both design and implementation, but some can only be caught at one stage or the other.

Different security activities are effective at discovering issues at different stages. Code reviews and penetration tests are very good at identifying security bugs, while threat models are good at detecting flaws. Sometimes the boundaries mix, but the strengths of each method are clear.


Why Do Threat Modeling?
BLACKDUCK
Black Duck Confidential Information
© 2024 Black Duck Software, Inc.
13

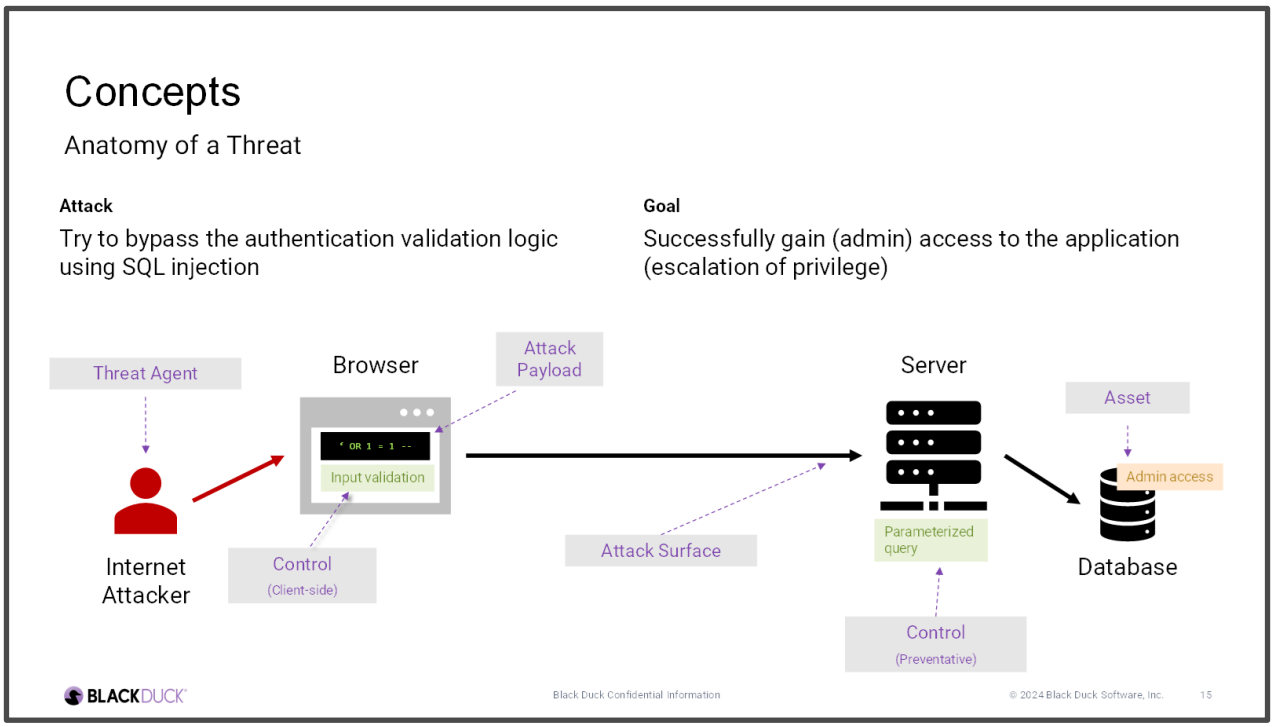
Slide 13



Slide 14

There are many benefits to threat model, and they will trickle down to other security activities. The slide focuses on the three most significant categories:
- **Risk management**: threat modeling provides a better understanding of the system from a security perspective. Knowing what matters, how it can be compromised and what happens if it gets compromised, enables application teams to focus their security efforts. Risk management allows to decide which mitigation strategies to pursue.

- **Early detection:** even if some issues can be caught at different stages, detecting them early significant reduces the cost of mitigating them.
- **Efficiency:** Future activities, like pentests and code reviews, can become much more efficient if they are informed by the threat model. Being usually time-boxed, this time can be used much better if the testers do not have to use part of it to discover the system and, in a way, come up with their own model.



Slide 15

To talk about threats requires to settle on a specific vocabulary. Different sources out there will use somewhat difference words and definitions, but for a company it is only important to agree on a certain terminology.

This slide introduces some terms and advances a definition that will be used throughout the training. The terms are:

- **Attack and goal**: each threat has two dimensions. One is technical, where the intention is to describe the steps or conditions that lead to the situation. In other words, to describe the *attack*. The other dimension focuses on the intention of the attacker: why would an attacker carry on the attack? What is his *goal*?
- **Threat agent**: threat agents are our way to profiling attackers. Different attackers can have different capabilities, motivations, or points of interaction with the system.
- **Assets:** assets are the parts of the system we want to protect. Identifying assets and describing why they matter will be helpful to identify *relevant* threats at a later point.
- **Controls:** controls are security features. These are mechanisms that allow us to make a threat harder or to take measures to reduce their impact.
- **Attack payload:** in very technical threats, it describes the exact interaction that triggers the threat (e.g. the string that would allow to exploit a SQL injection).
- **Attack surface**: the attack surface describes the point where the attack interacts with the system. It is usually discussed in terms of relative size. For example: Adding features increases the attack surface, while implementing a firewall reduces it.



Slide 16

Despite its seemingly complex nature, threat modeling is an intuitive process. We perform threat modeling on a daily basis when we run daily errands such as driving, walking, and shopping. In the context of this class, threat modeling is applied to a system under analysis. The “Black Duck” threat modeling approach performs this analysis in an algorithmic fashion. Some portions of the “Black Duck” threat modeling approach utilize a checklist.

We Must Protect This House

Sample Threat Analysis



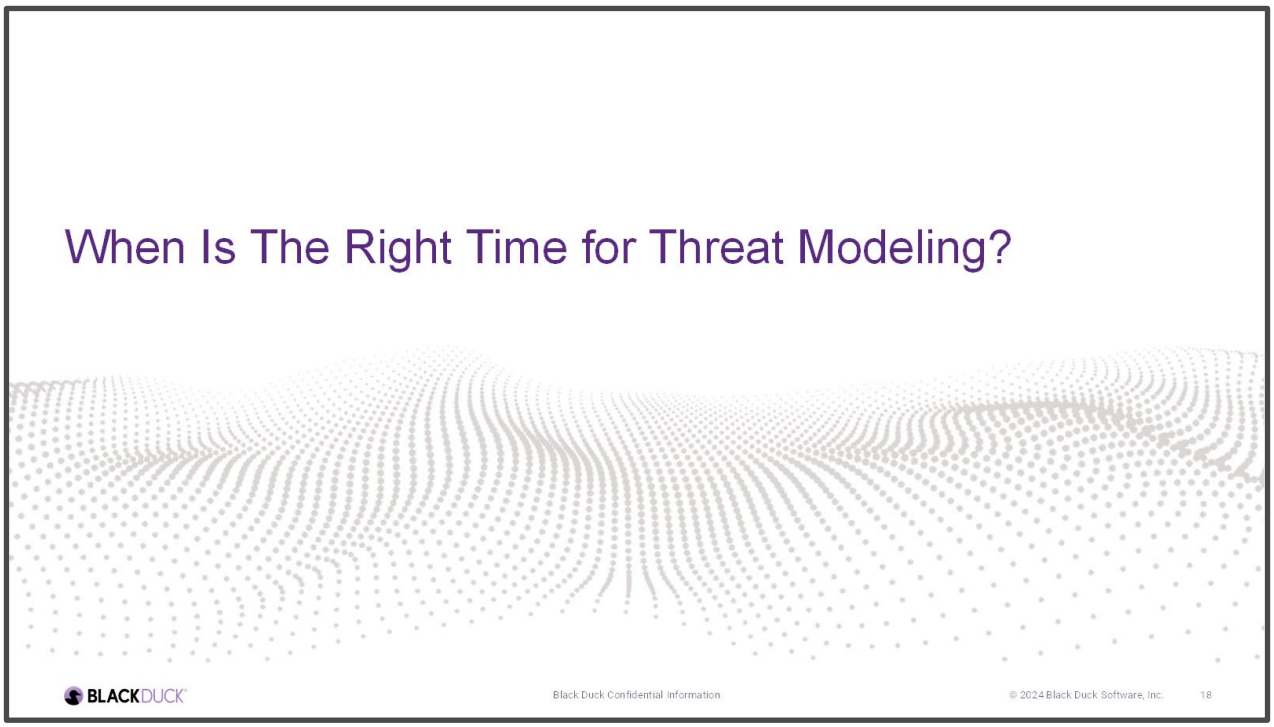
- A burglar may break into the garage and **steal one of the vehicles**.
- If successful, the family may **need to purchase a new vehicle**. The estimated value of a luxury sedan is over $50k.
- The car doors are locked, the garage door is closed and locked, and the homeowner has an alarm system armed.
- Likelihood of threat:
  - Is it likely that the car could be stolen? Why, or why not?
  - What could you do to reduce the likelihood of theft further?
- Impact of threat:
  - How damaging would it be if the car was stolen?
  - What could you do to reduce the impact of theft?

BLACKDUCK

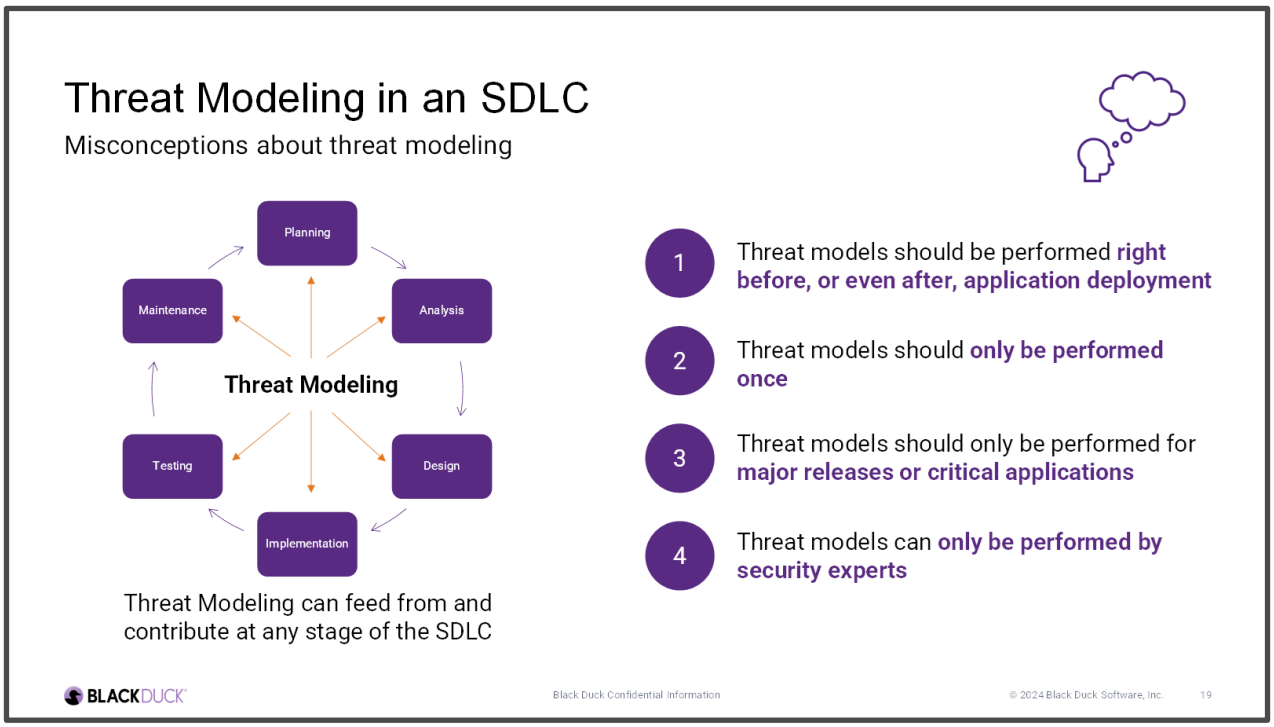
Black Duck Confidential Information

© 2024 Black Duck Software, Inc. 17

Slide 17


When Is The Right Time for Threat Modeling?
BLACKDUCK
Black Duck Confidential Information
© 2024 Black Duck Software, Inc.
18

Slide 18

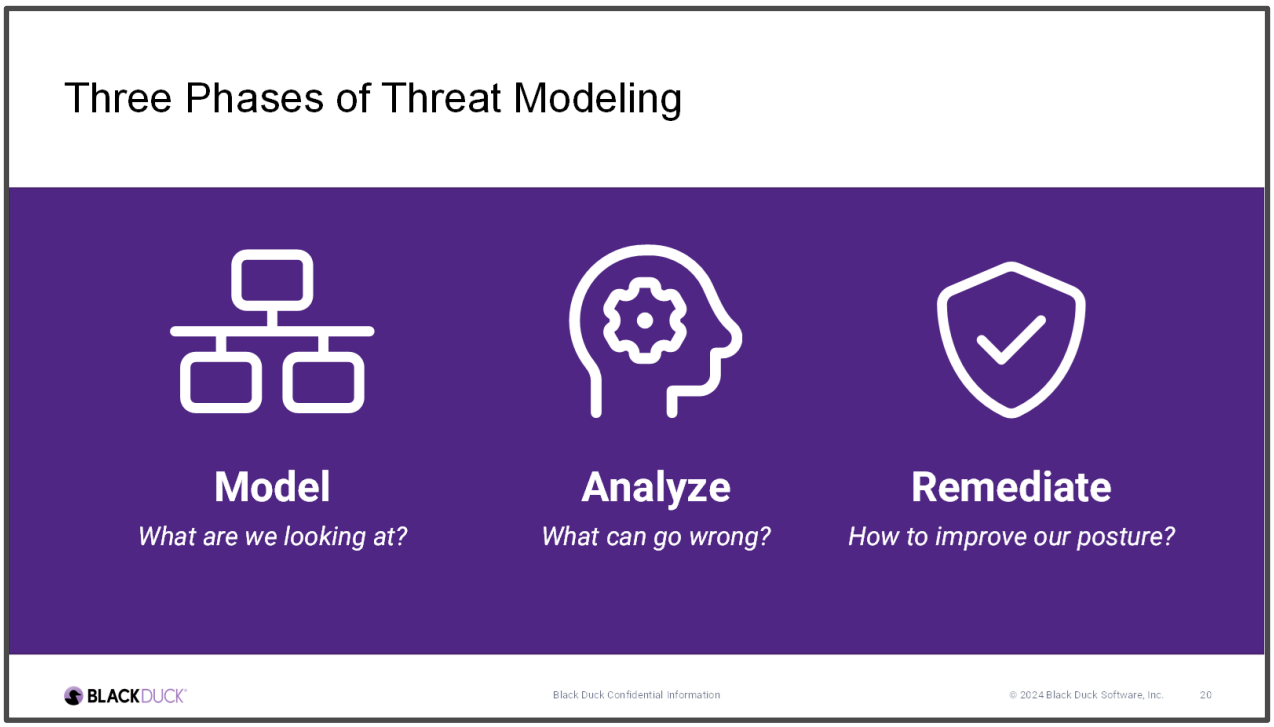


Slide 19

We have already remarked how *early detection* is one of the benefits of threat modeling. This would suggest it is an activity that should happen earlier on in the software development life cycle, but this can lead to some misunderstandings:

- What if you "inherit" an existing application? Is it too late for threat modeling? The answer is a resounding NO. A late threat model is better than no threat model at all. Fixing the findings may be harder, but the insights are still invaluable.
- The idea that threat modeling is a box to be checked is dangerous and misleading. As systems evolve, threat models need to be updated to stay relevant. That said, the question of how often to update it, or to which level of detail, is a major issue each company must address for itself.

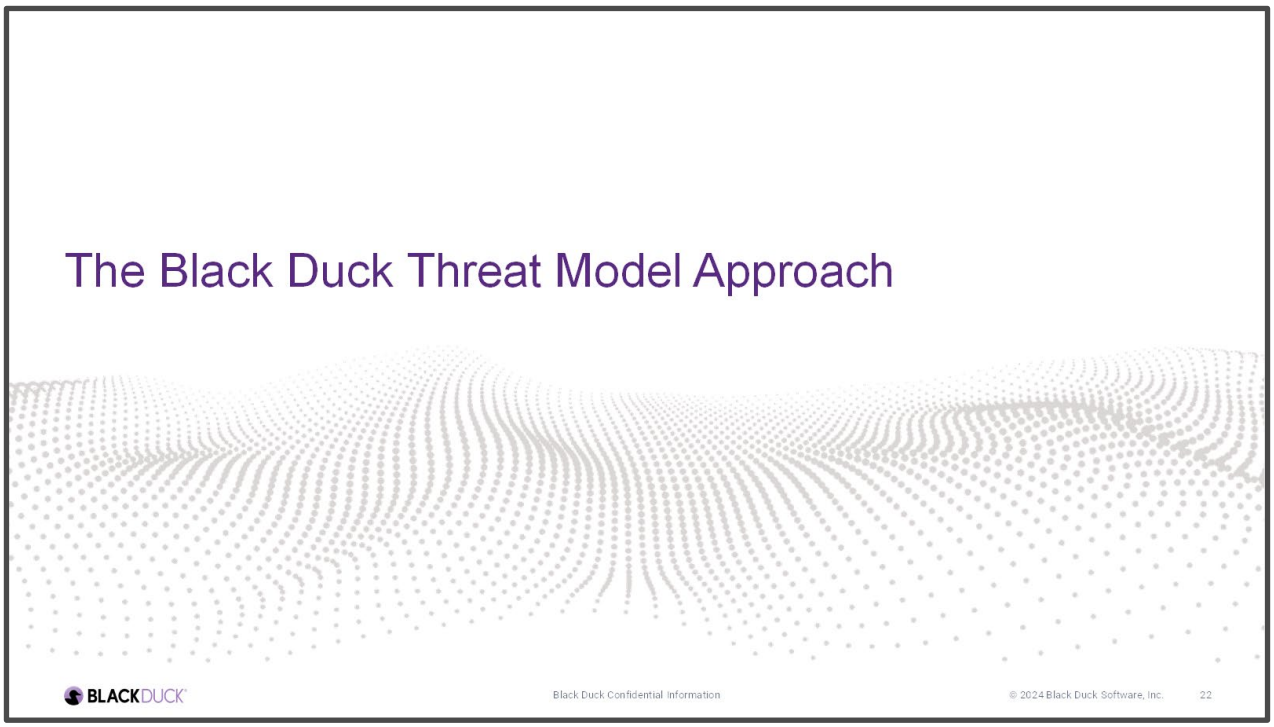
Other dangerous misconceptions stem from an understandable but misguided interest in reducing the cost of threat modeling. For example:
- The idea that only the most critical application or systems deserve a threat model. This leaves many applications unprotected. These applications could themselves keep valuable information, or even become a stepping stone for gaining access into more valuable systems. The mindset should be entirely swapped: instead of being a privilege reserved for a few applications, threat modeling should be part of an overarching baseline. Only the most harmless, minimal applications, should ever be exempt from it.
- Avoiding threat modeling because experts are expensive is another dangerous decision. Provided enough guidance, application teams can often perform threat models. In fact, their knowledge about the system may provide some insights that external reviewers may struggle to reach.



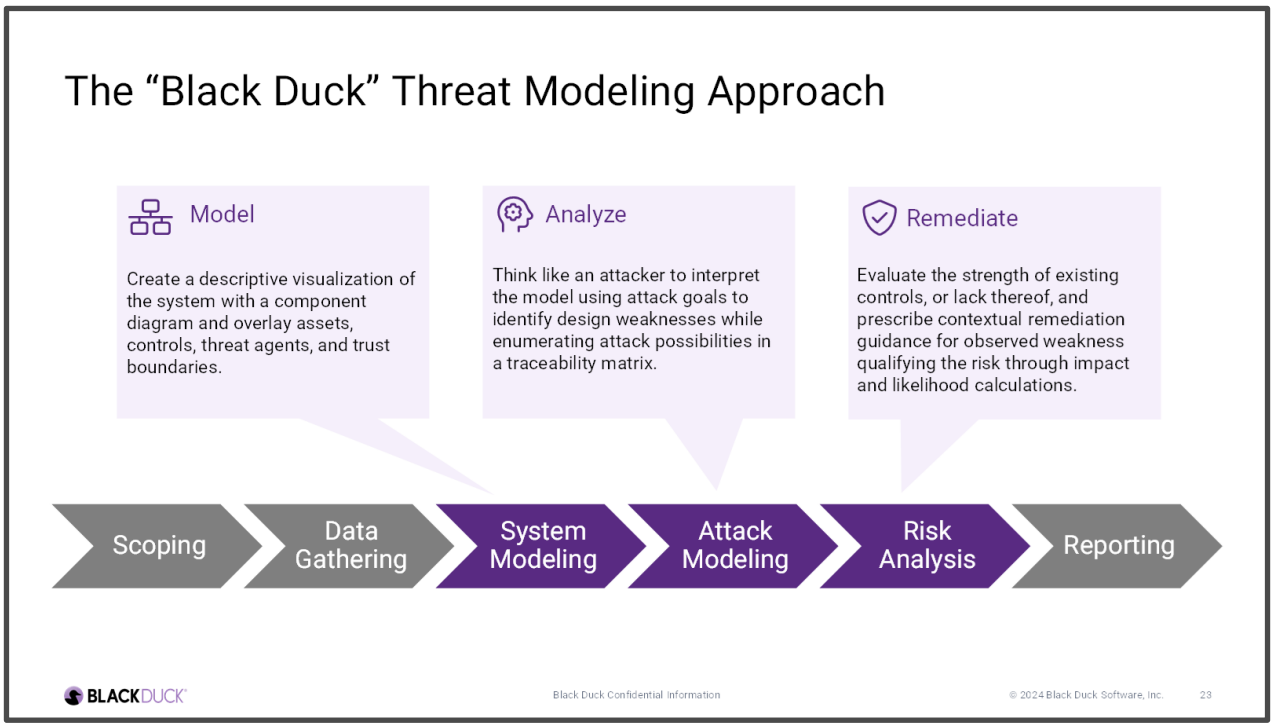
Slide 20

Threat modeling is composed of three basic steps:

- Understand what we are building (or what has been built). This is called “the system.”
- Try to enumerate different things that can go wrong with the system.
- Determine how we can compensate or control the things that can go wrong with the system.


The Black Duck Threat Model Approach
BLACKDUCK
Black Duck Confidential Information
© 2024 Black Duck Software, Inc.
22

Slide 22



Slide 23

The security consulting team at Black Duck (formerly part of Synopsys, and of Cigital before that) has performed threat models for years on a large variety of industries. These years of experience have been reflected in what we call the Black Duck threat modeling approach.

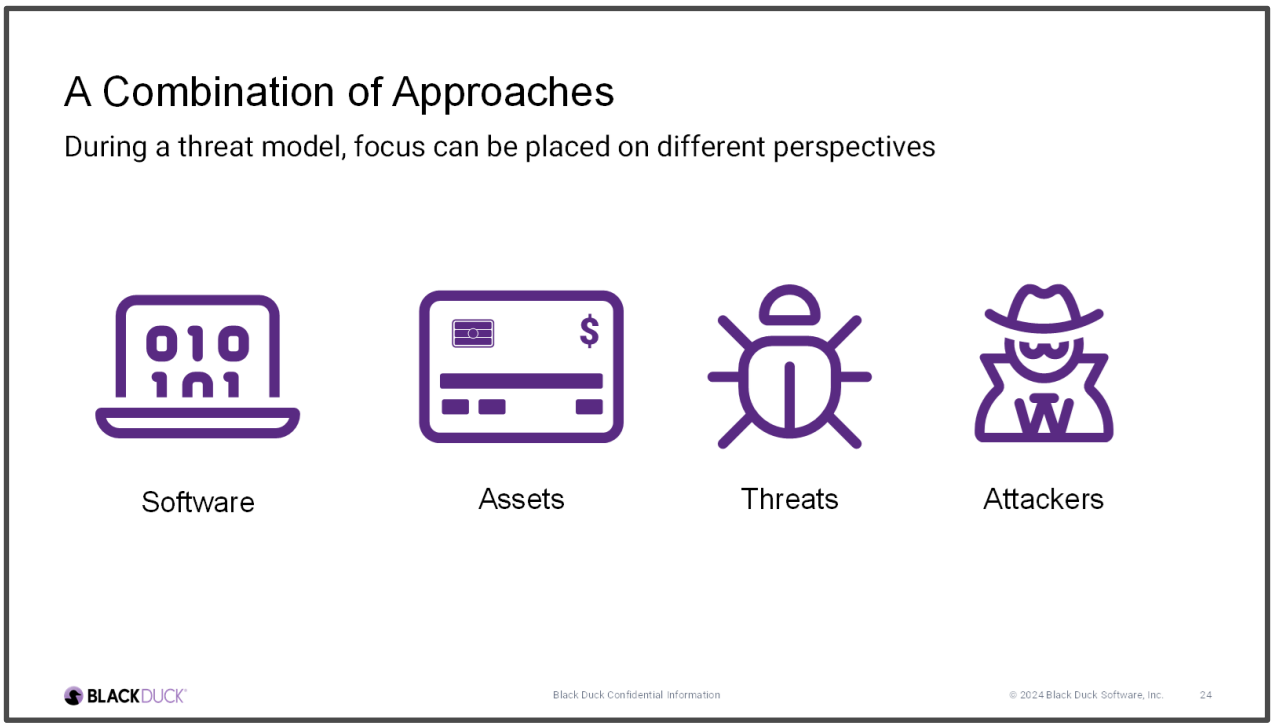
At a high level, threat model tends to follow a very sequential pattern (think of the Model / Analyze / Remediate phases discussed earlier). Most of the differences among methodologies (or rather, *approaches* to that global methodology) lie in specific details like the scales used for rating or the shape of certain deliverables. This course follows the Black Duck approach as a way of looking at this level of detail, but it does not try to lean on it as the best way of threat modeling. Most of the takeaway from this course can be applied to other methodologies just as well.

Altogether, the Black Duck approach is split into six steps:

- Scoping: deciding what parts of the system are in scope and to what extent.
- Data gathering: collecting information about the system (e.g. through interviews or documentation).
- System modeling: summarizing the data gathered into a model that highlights the components of the system and to review its security.
- Attack modeling: identifying possible attacks that could affect the system modeled.
- Risk analysis: assessing the level of exposure to the different threats by understanding their feasibility and impact.
- Report: presenting the result of the previous steps.

Scoping, data gathering and reporting are variable and can be highly influenced by externalities such as company templates or compliance requirements. This course pays attention to the more technical and essential parts of the process: system modeling, attack modeling and risk analysis. Please: **do not understand this to imply that the other phases are unimportant**. They are critical to the success of the threat model, too!

© 2024 Black Duck

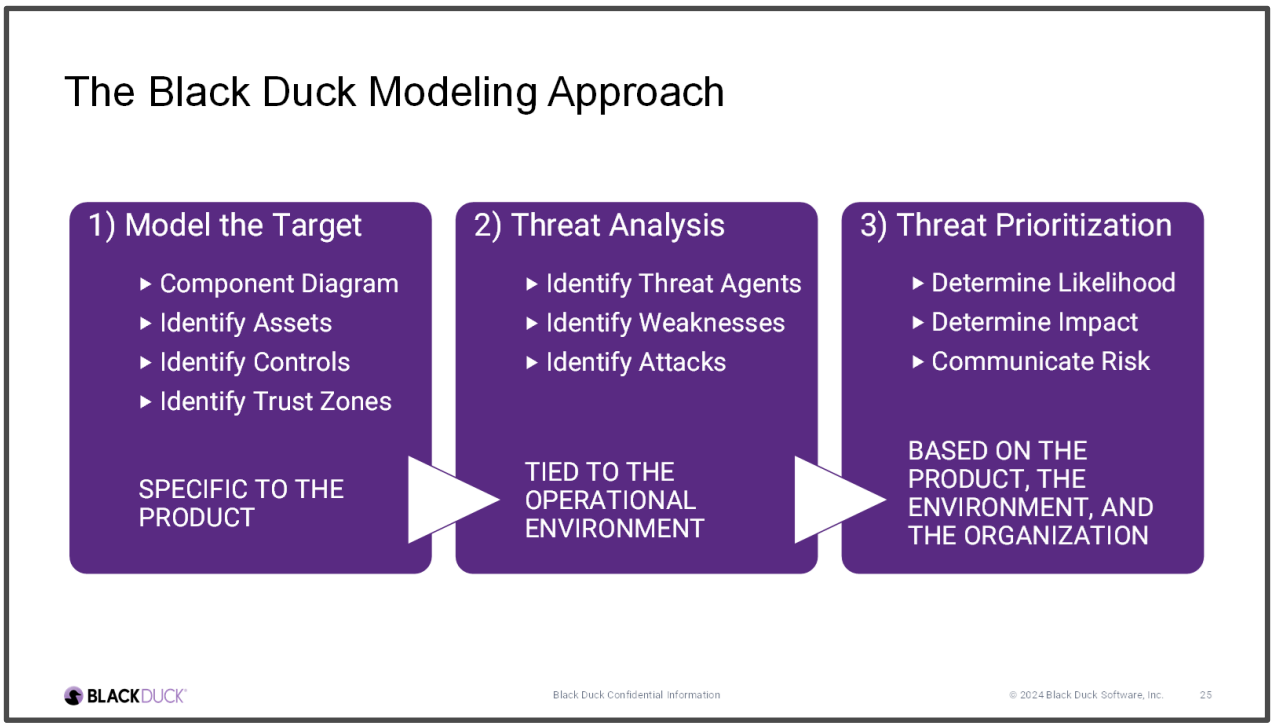


Slide 24

The first threat modeling approaches placed the analyst on a specific perspective that would emphasize one aspect of a system in order to identify threats based on it. For example:

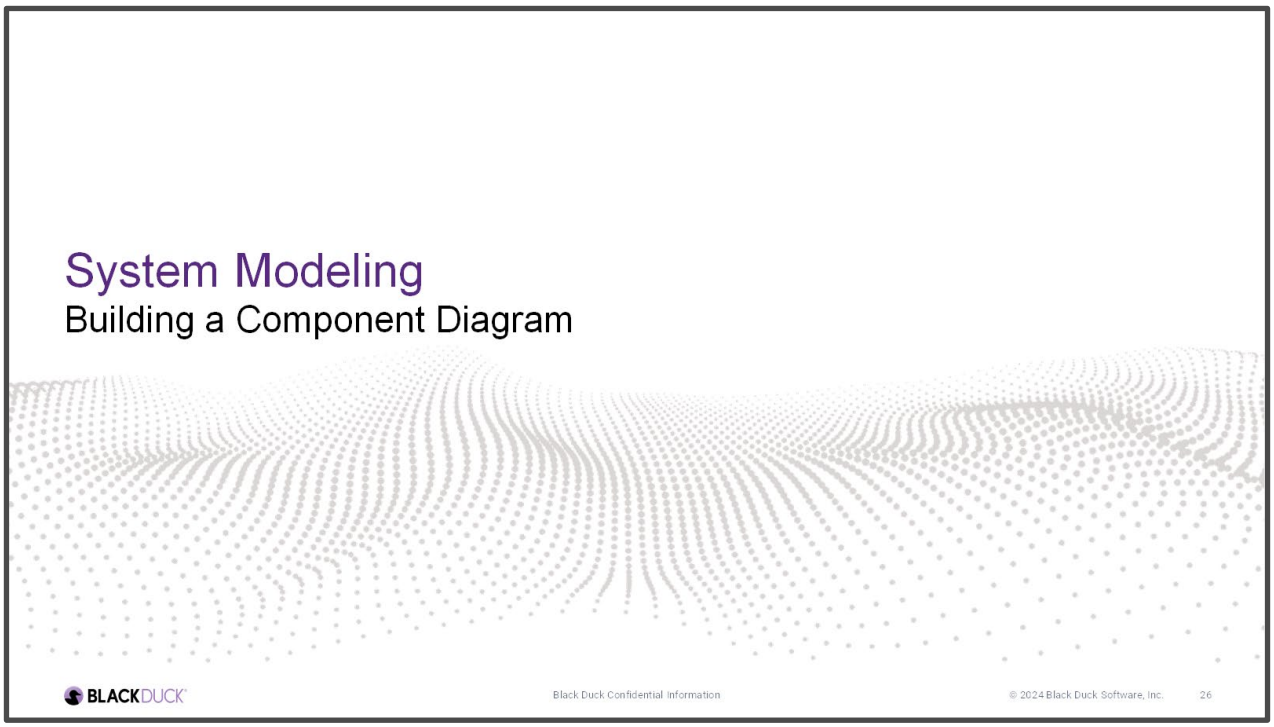
- Software: Identify the software components and connections in the system. Then, look at potential threats (e.g. "what are common threats for SQL Server? And for TLS?")
- Assets: Identify valuable elements within the system (assets) and analyze how they are exposed. For each asset, think what an attacker may want to do and why it is bad. (e.g. "how can someone steal the social security numbers of my users?")
- Threats: Look at different types of threats based on existing taxonomies. Decide how they apply (e.g. "what connections can be *spoofed*?", "in which way is data tampering a concern?")
- Attackers: Define threat agents, including their motivations and capabilities. What may each of them attempt? (e.g. "what can someone do connected from the internet?", "if an admin had the wrong incentives, what could he cause?")

It quickly became apparent that there is no reason to restrict oneself to one specific focus. Quite on the contrary: switching among points of emphasis throughout the assessment allows for a more comprehensive approach. The Black Duck approach is a testament to this insight.

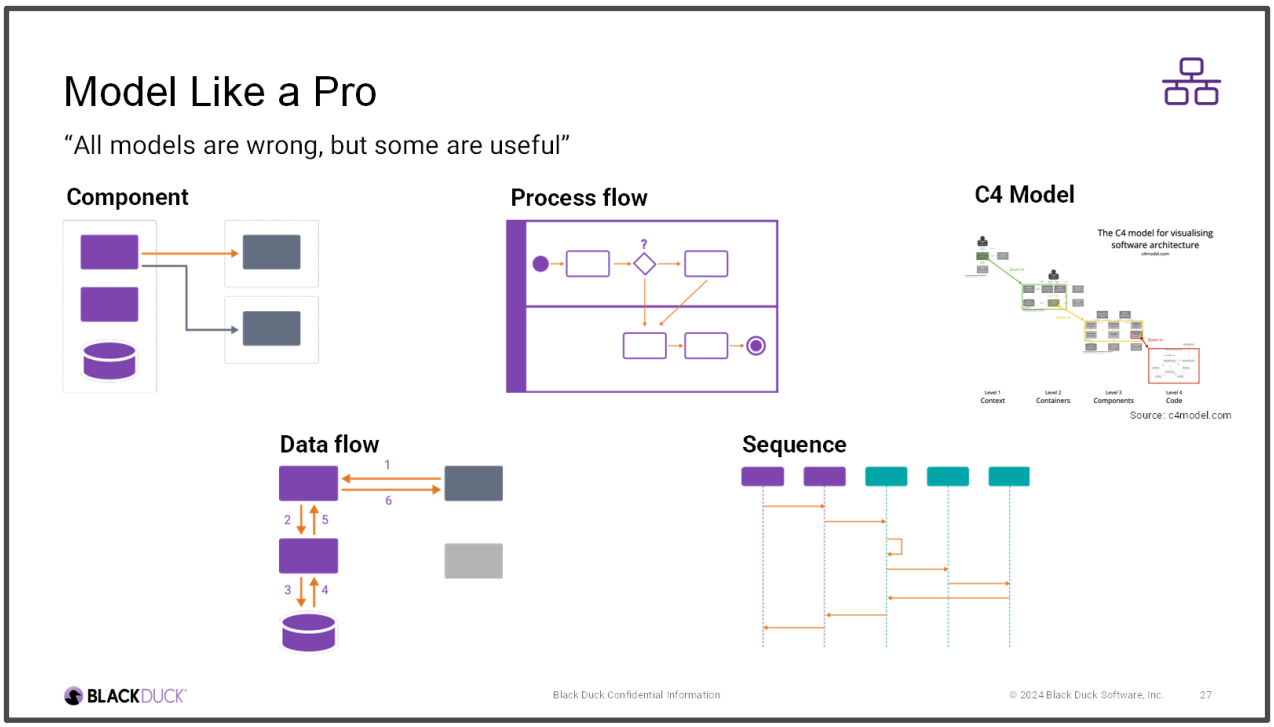


Slide 25

Look carefully – notice that the model step is entirely dependent on the design and the implementation while the analysis step depends on the world around the target. Threat analysis is the interaction of your product with the world. The prioritization step on the other hand is a mix of your product, the world around it, and the organization that owns the risk for the product.


System Modeling
Building a Component Diagram
BLACKDUCK
Black Duck Confidential Information
© 2024 Black Duck Software, Inc.
26

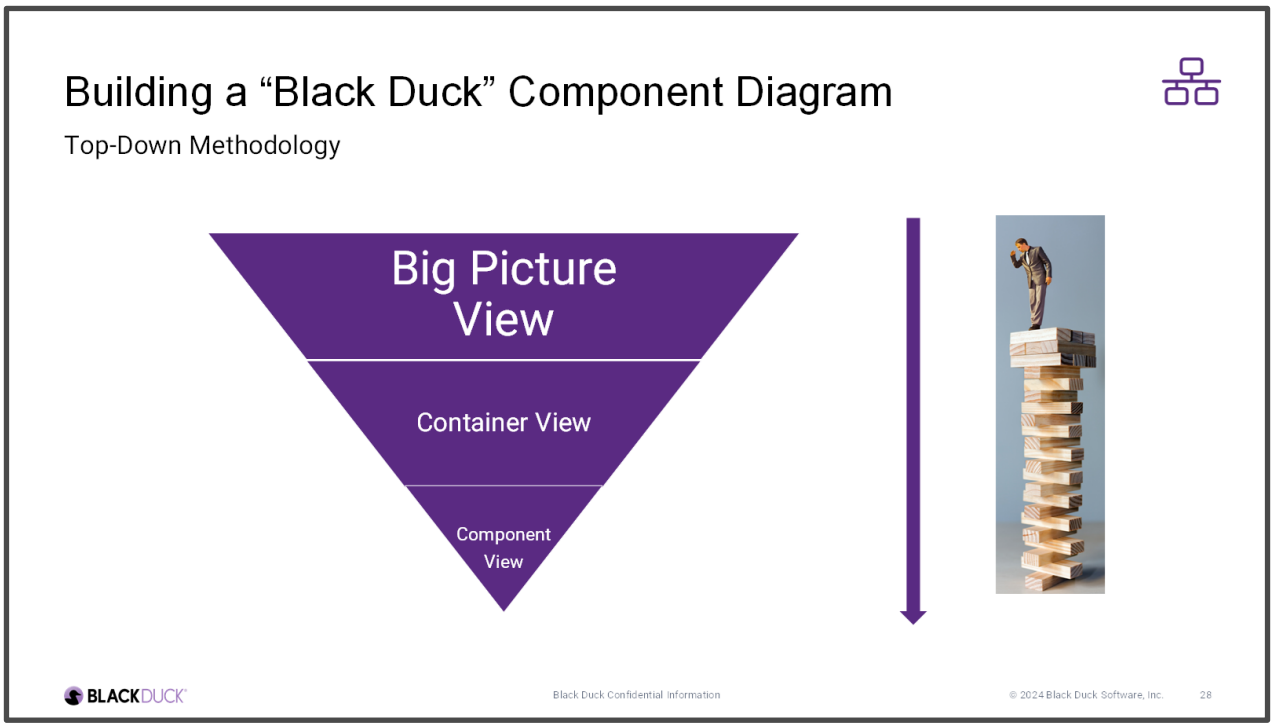
Slide 26



Slide 27

Models are built on abstractions and are typically logical representations – they reflect in human form how software systems work.

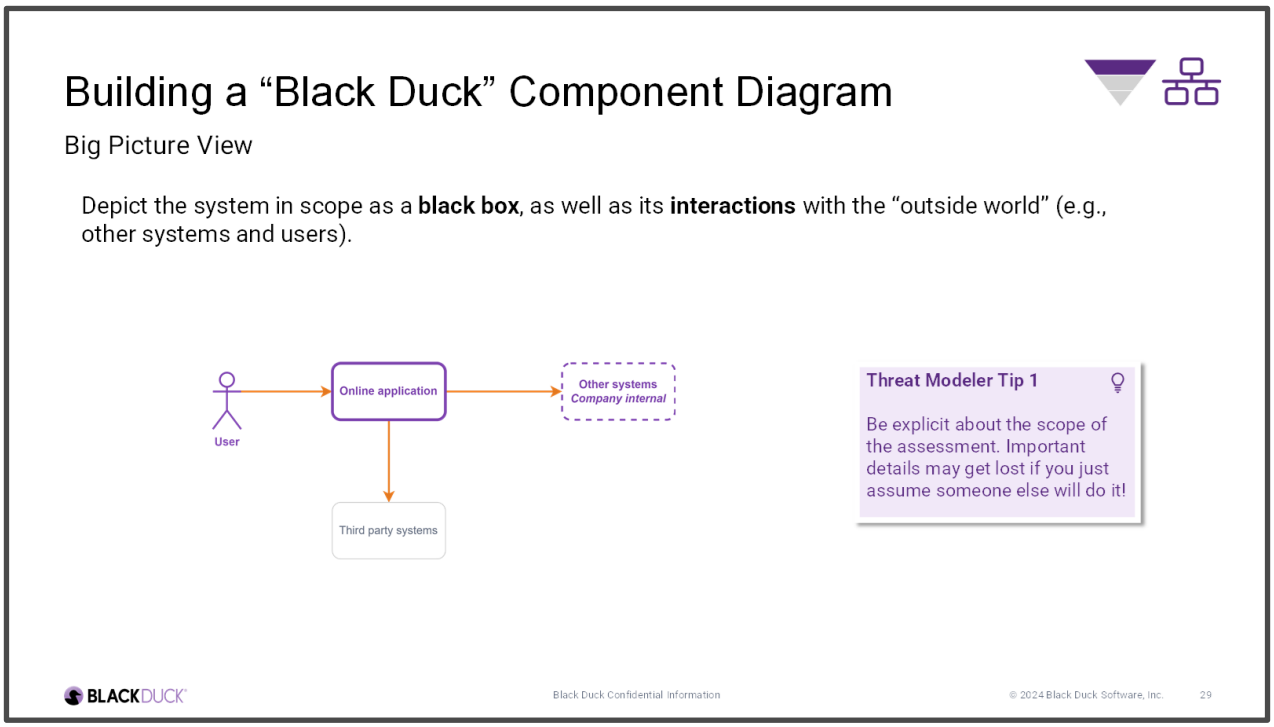
No model is perfect or complete. The aim of system modeling is not to create a 100% model of the system but something that is "good enough" and useful for creating a shared view.



Slide 28

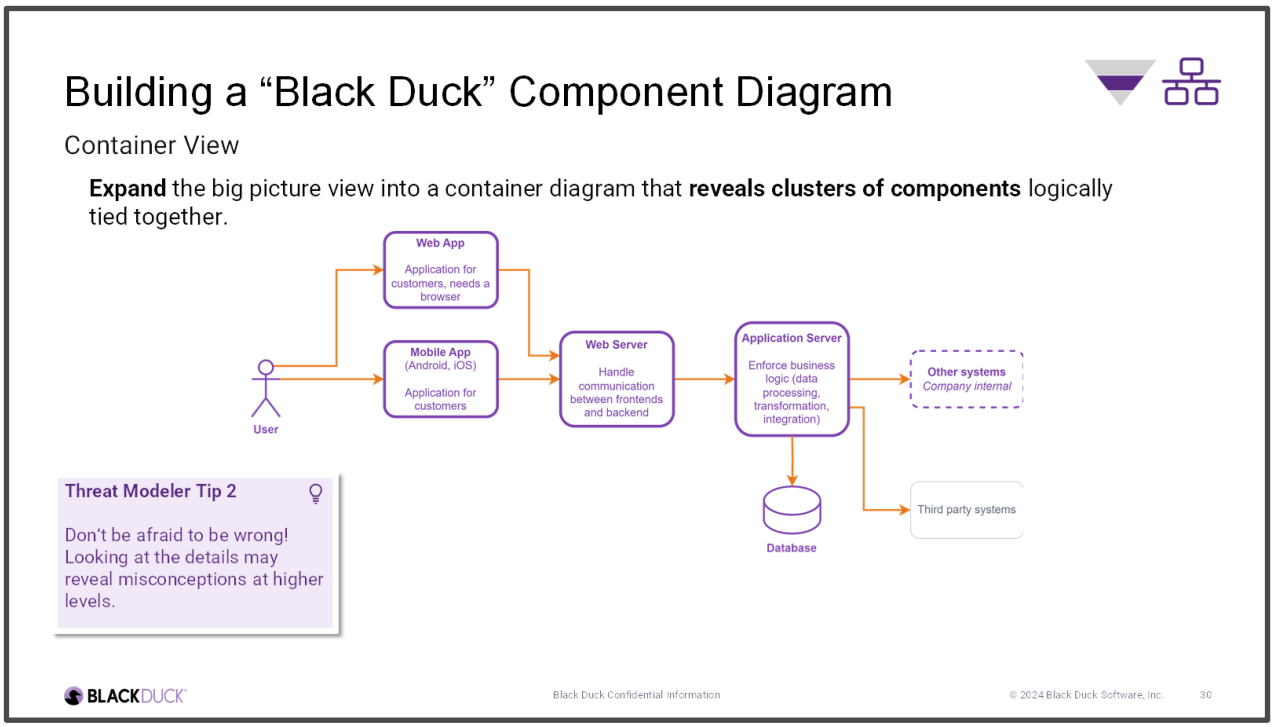
The Black Duck approach has been geared towards consultants. Most of the time, a consultant starts a threat model with nothing but the faintest idea about the target system. His process of data gathering and system model is not just a formality: it is his way to gain an understanding of a complex system. This is a process of iterative discovery that starts with a coarse description of a system and ends in a highly detailed exploration of its design and the motivations behind it.

The way the approach creates a diagram is a reflection of this process: it starts with a high level representation and it refines it until sufficient detail is attained. The following slides will exemplify this process.



Slide 29

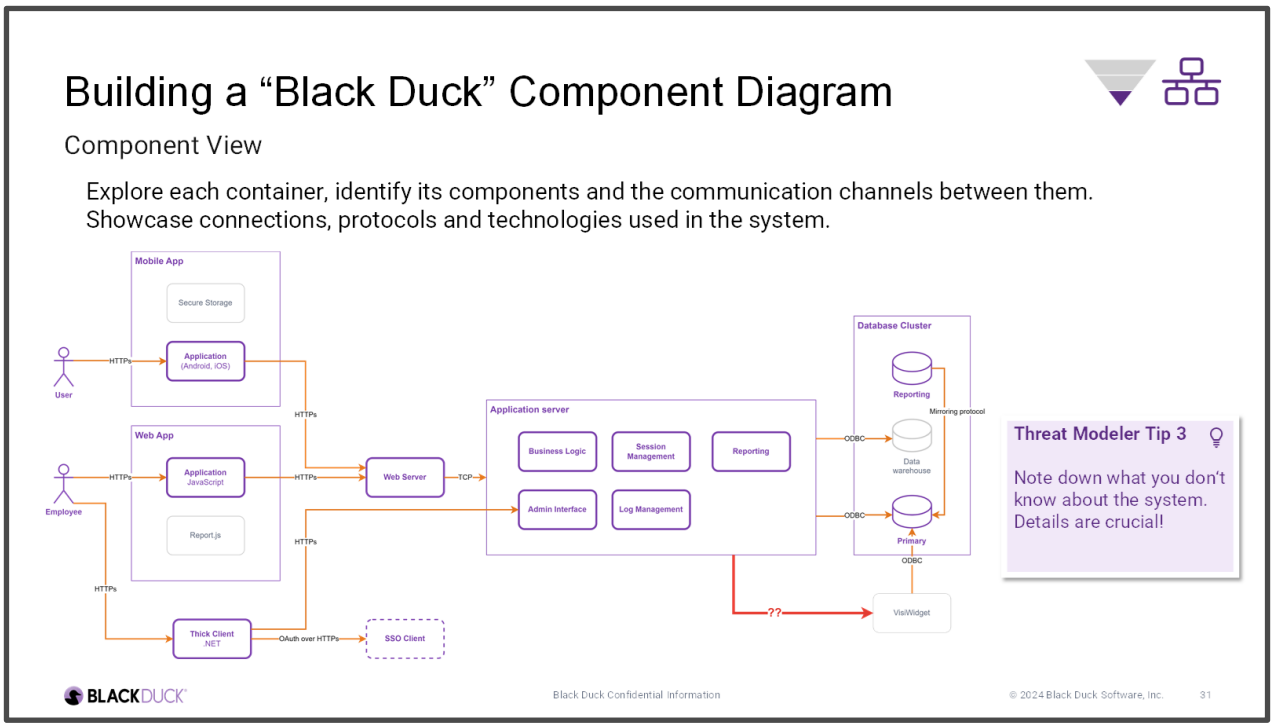
If we take our initial understanding of the system under review, we know that the primary target is an online banking application. This application connects to other internal systems and some 3rd party systems to operate on a normal basis.



Slide 30

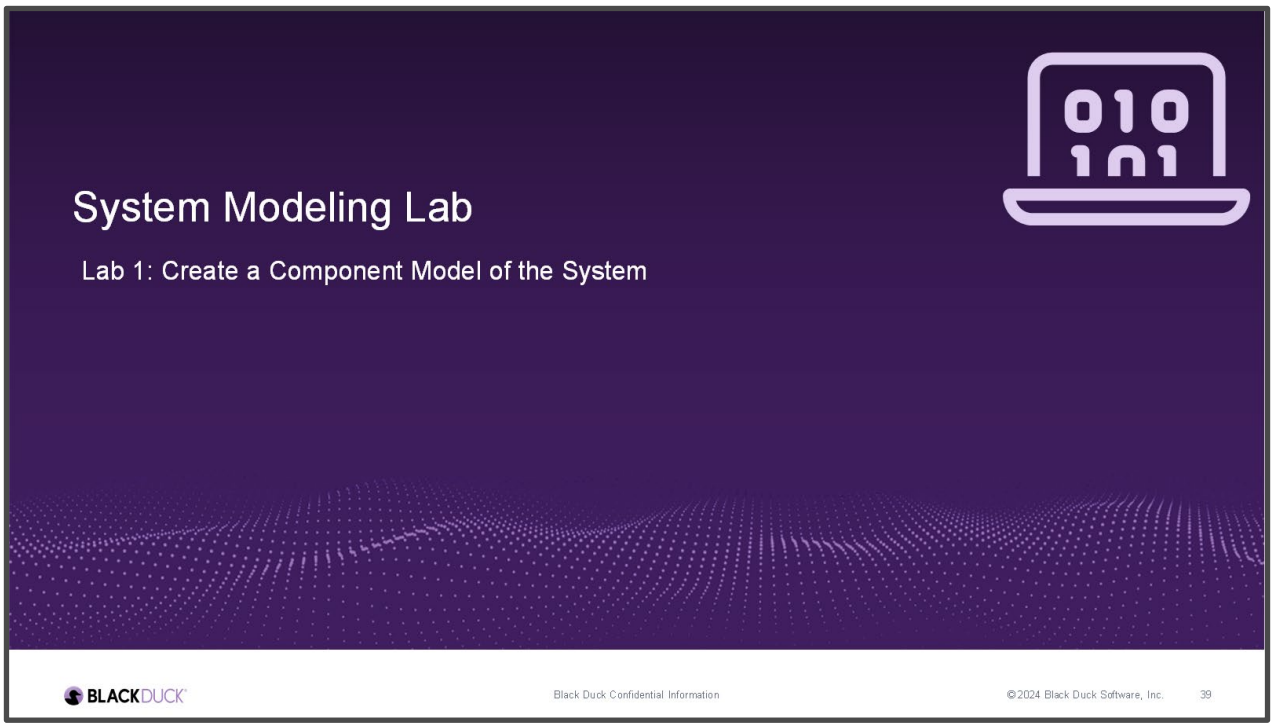
The container view is the next level of detail where we break down are high-level components into groups of containers. These containers typically represent major components of the system. Upon completion of our interviews, we discover the application is made up of some mobile application, and web applications that both process data in a backend database.

Oftentimes, these containers highlight the roles of people we need to interview during the data-gathering phase.

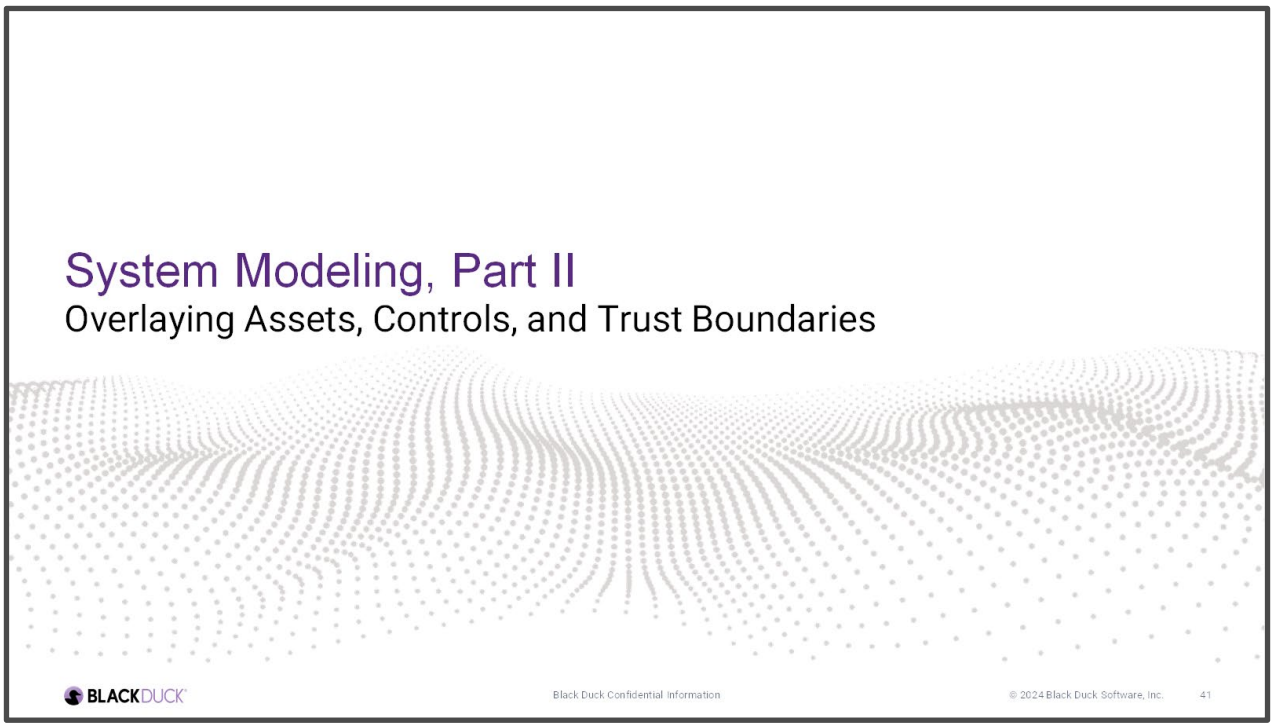


Slide 31

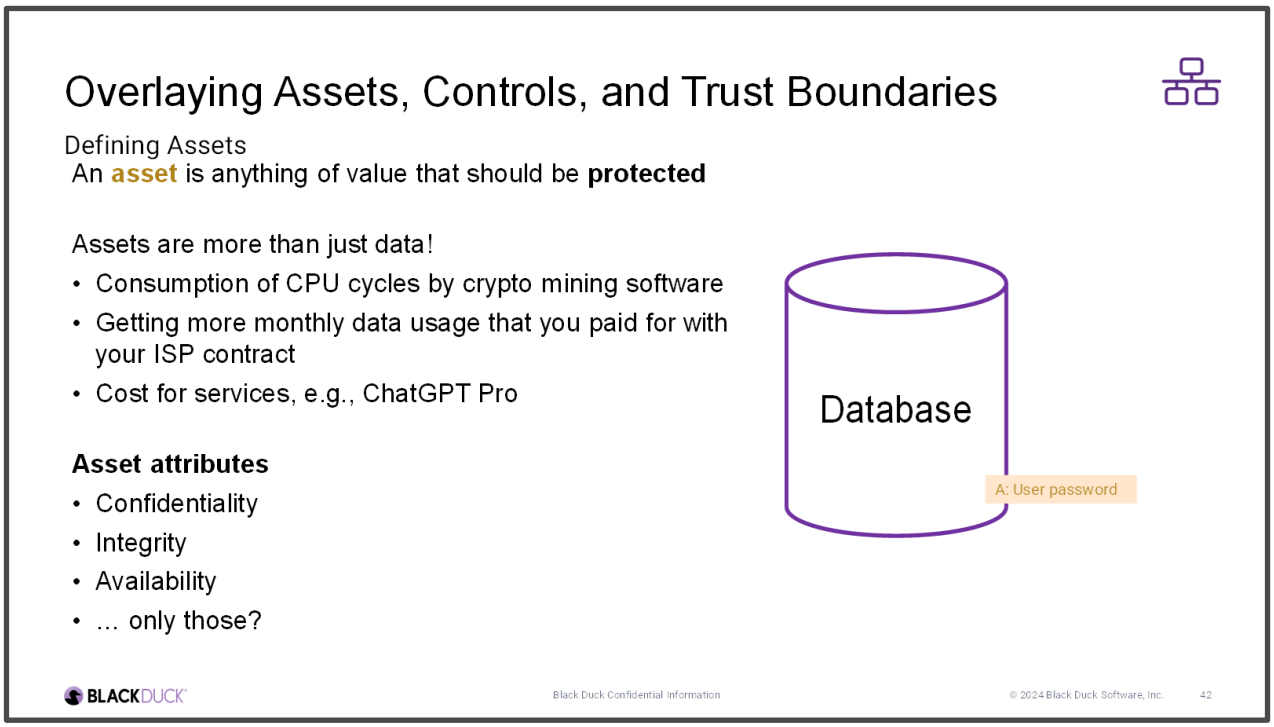
The component view is often the most granular level of detail provided to build a diagram. The level is often dictated by the client and the interviews but should show enough information to showcase major functionalities, features, and technologies the make up the system. This view should showcase how all the components communicate with each other.


System Modeling Lab
Lab 1: Create a Component Model of the System
BLACKDUCK
Black Duck Confidential Information
©2024 Black Duck Software, Inc.
39

Slide 39


System Modeling, Part II
Overlaying Assets, Controls, and Trust Boundaries
BLACKDUCK
Black Duck Confidential Information
© 2024 Black Duck Software, Inc.
41

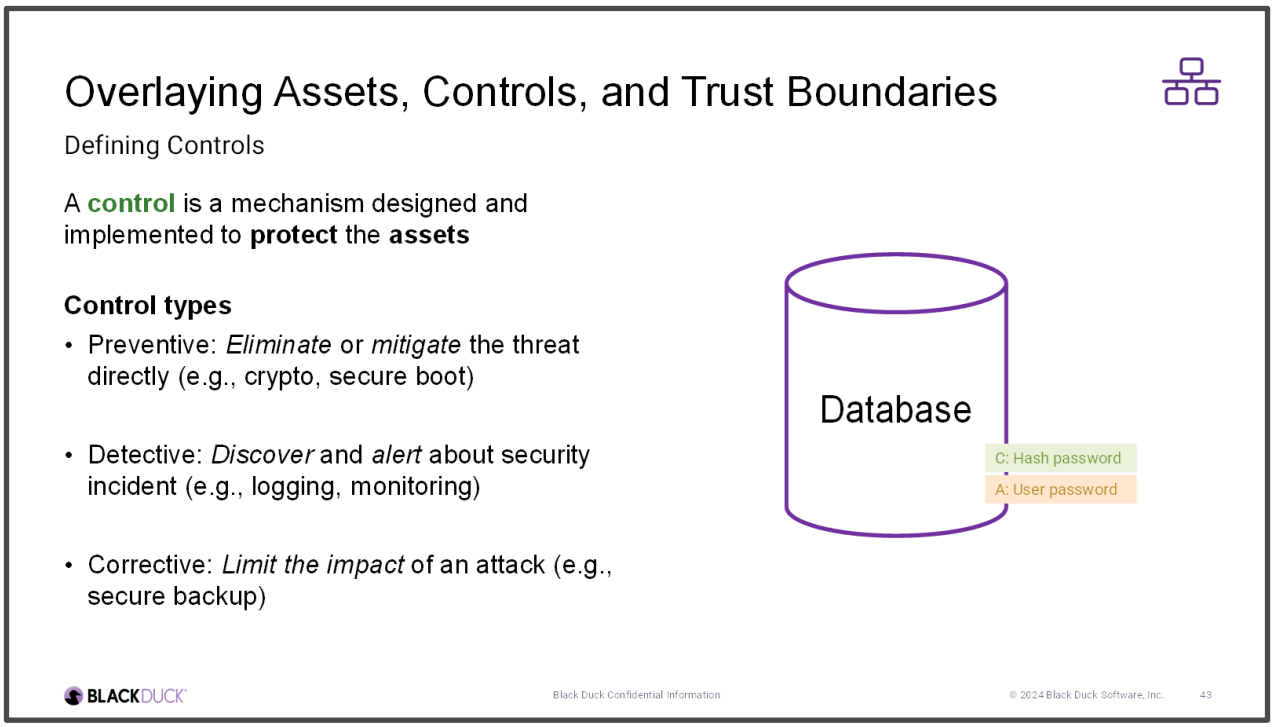
Slide 41



Slide 42

An asset is something that you are trying to protect. An asset could be data, functional (e.g., API), or even a system property (e.g., availability) that you want to protect against abuse or compromise.

You will need to understand business risks to ensure you've identified assets that are in scope for your analysis.

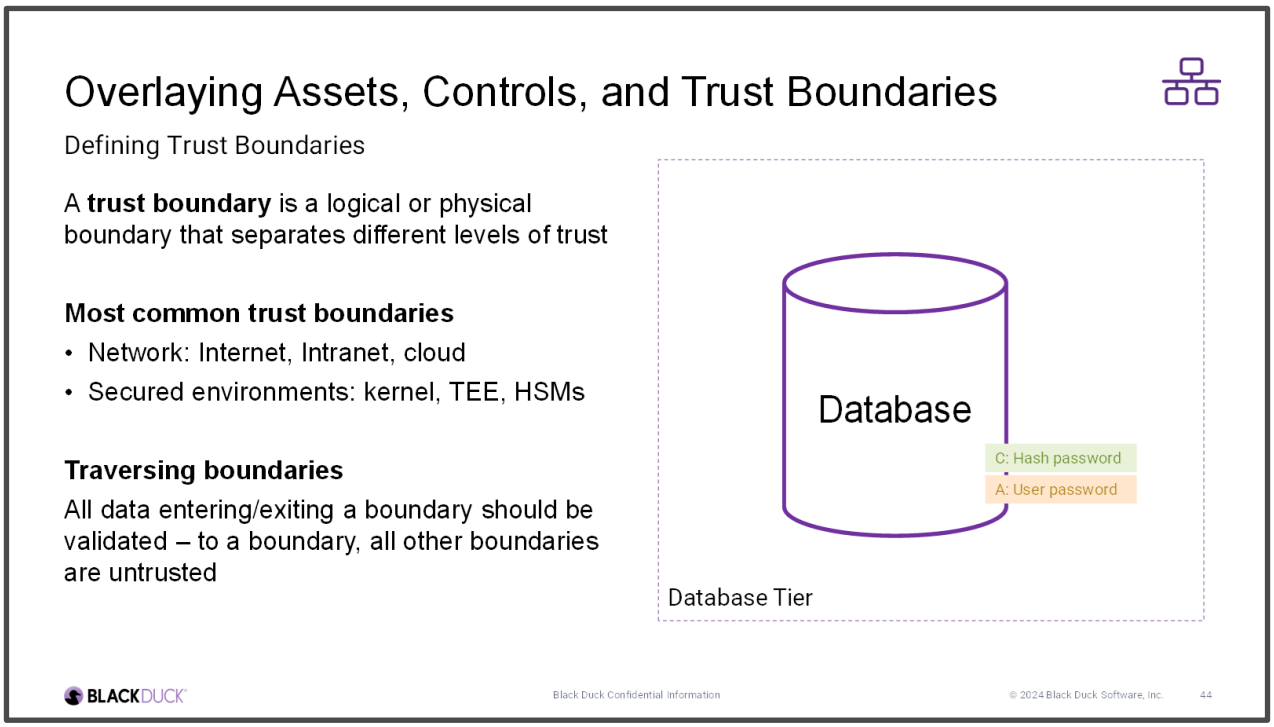


Slide 43

A security control is a mechanism that helps protect assets from one or more threat agents (aka attackers). The most common controls can be an application-level control or a network-level control. It is important to note that a control may or may not provide complete protection to an asset. A control considered secure at one point in time may fail in the future (due to the disclosure of new vulnerabilities). It is recommended that you do not rely on a single security control while protecting an important asset (defense-in-depth security principle).

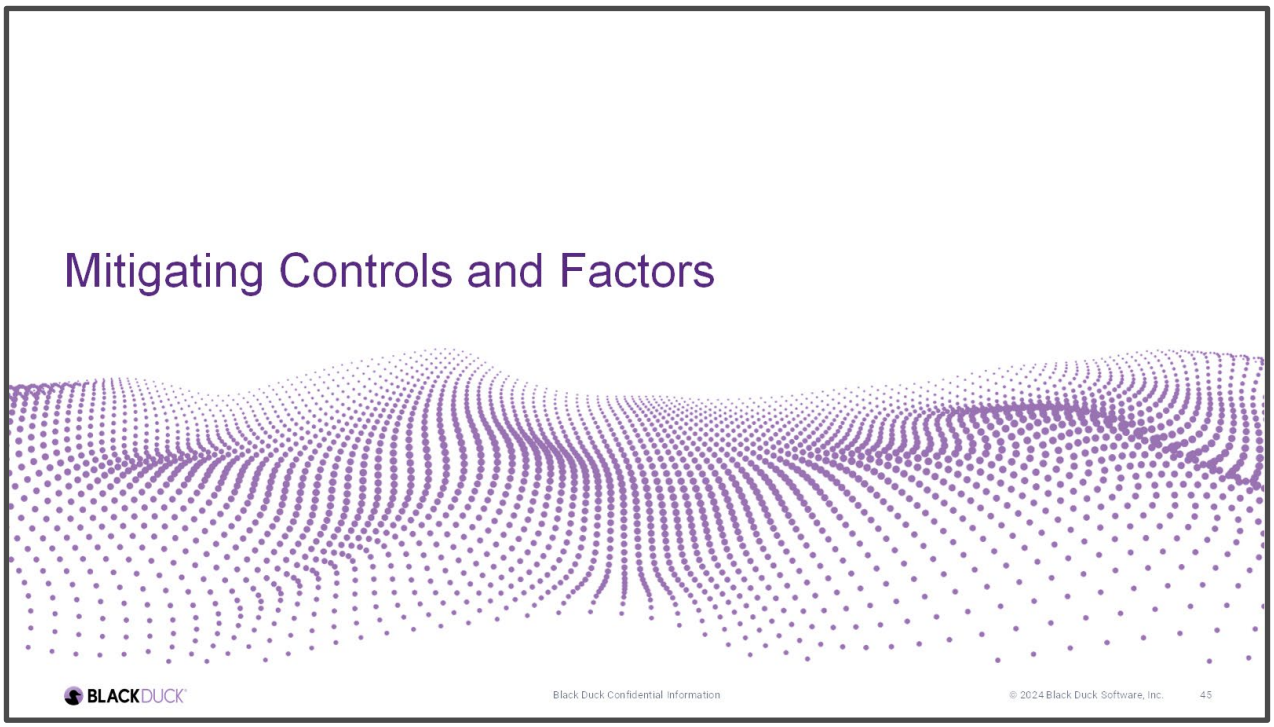
Generally, you are concerned about security controls at the network and application level.

Examples of security controls include:
- Authentication and Authorization
- Data confidentiality
- Message authentication
- Monitoring
- Edge security controls like WAF and BOT managers
- Network segmentation controls like subnets
- Malware detection controls

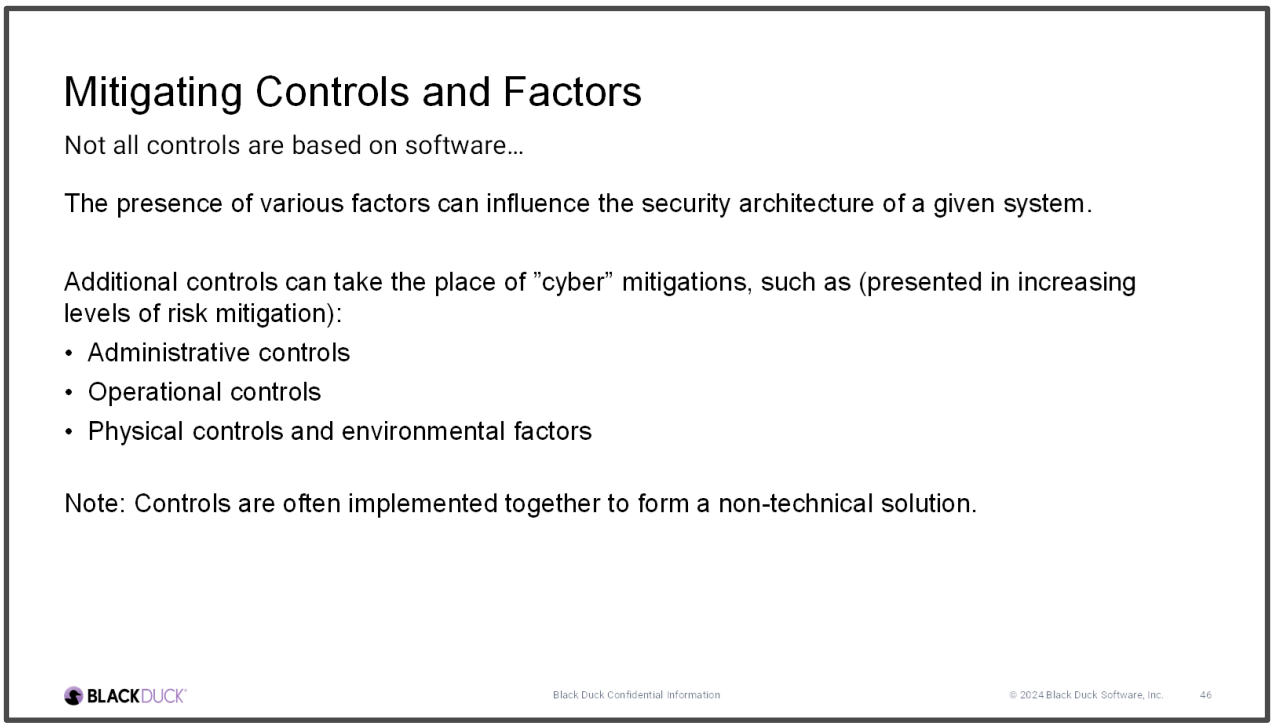


Slide 44

A trust boundary or a zone in security is used to describe a boundary where all the internal components / functions share a common set of security controls and security posture. Each zone has an implied level of risk tolerance defined by the security controls placed on the elements within the boundary. The term refers to any distinct boundary within which a system trusts all sub-systems (including data). You must authenticate senders across trust boundary, apply authorization control, and all the data validation controls. This is required as when you pass from one trust boundary to another, the default control set changes, and therefore the level of vetting of risk has changed.

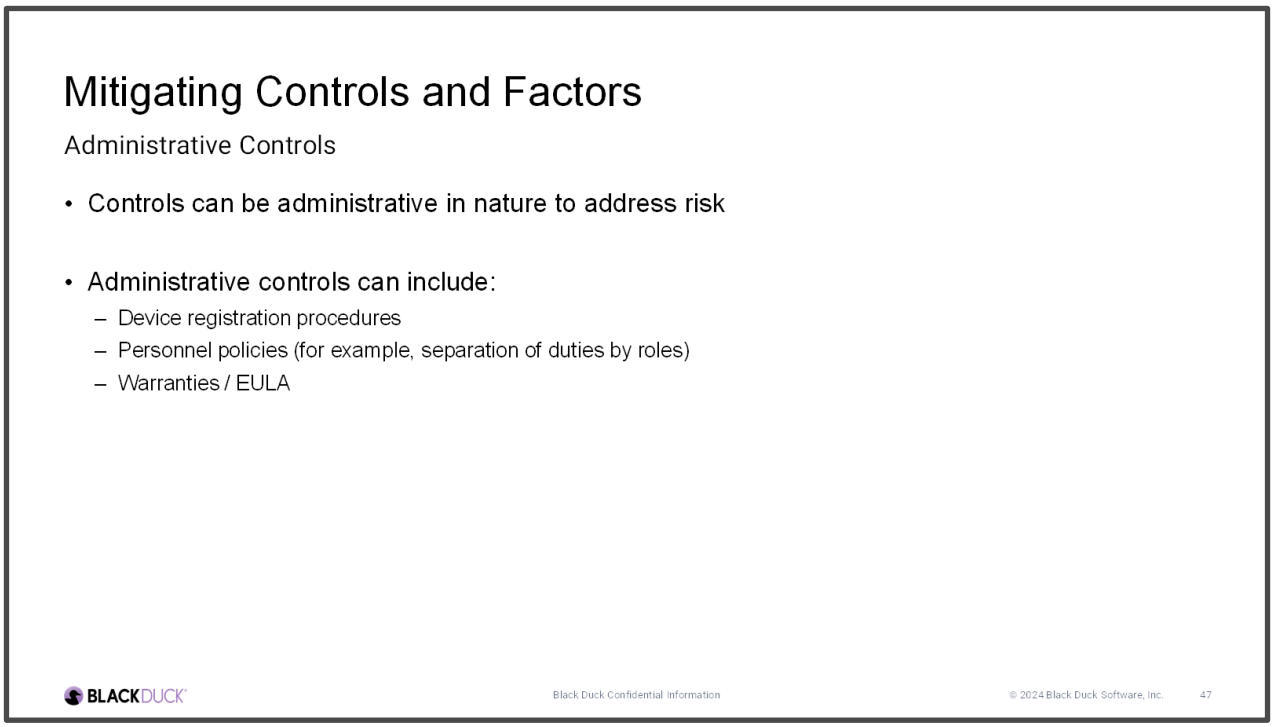

Mitigating Controls and Factors
BLACKDUCK
Black Duck Confidential Information
© 2024 Black Duck Software, Inc.
45

Slide 45



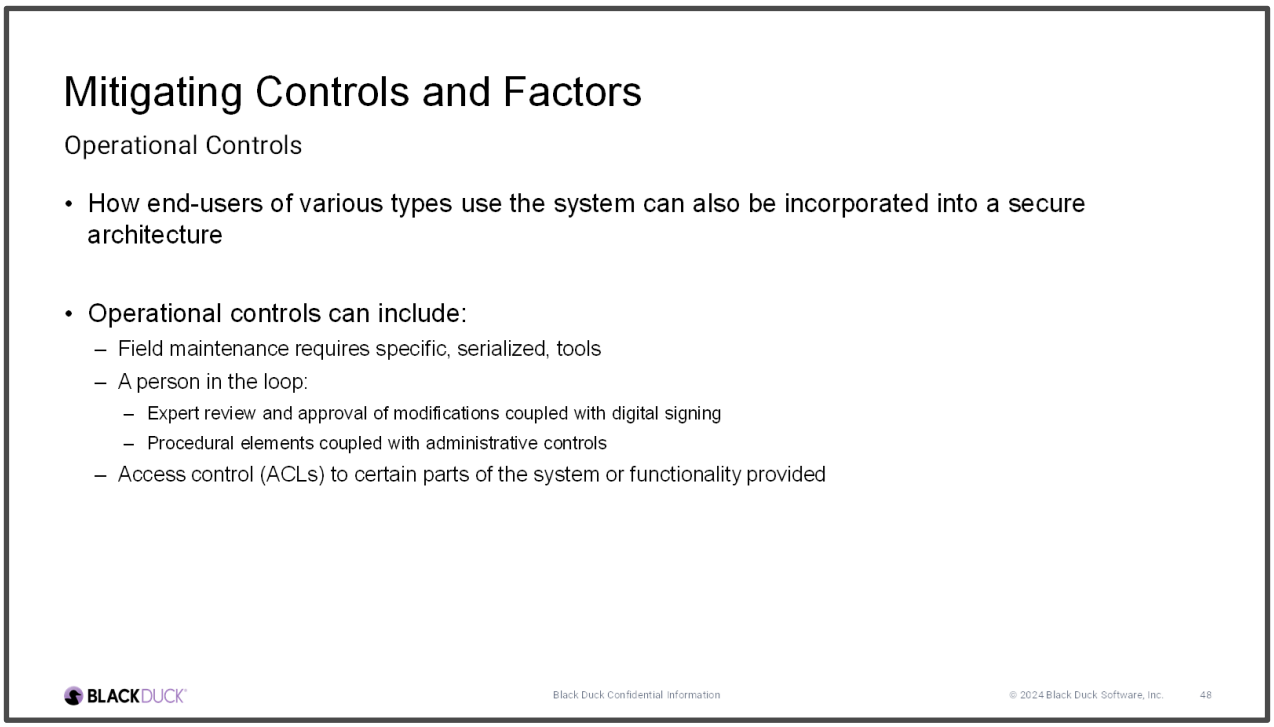
Slide 46

There are different ways to create and apply security controls to mitigate risk. For embedded work, often the environment itself may be a control.
NOTE: even though these other controls and factors may be referenced, their impact MUST BE DOCUMENTED in the Threat Model.



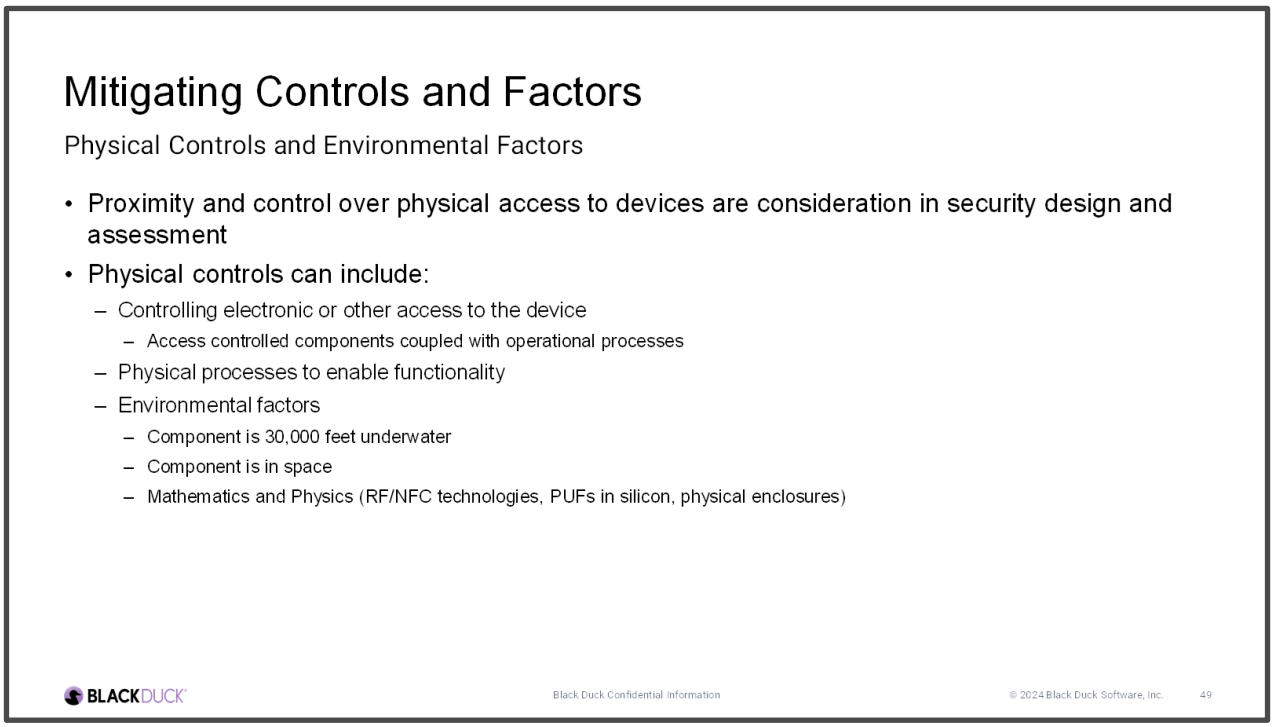
Slide 47

Administrative controls are weak in that they are reliant on user organizations to implement, track, and enforce, but they are still part of the security picture.



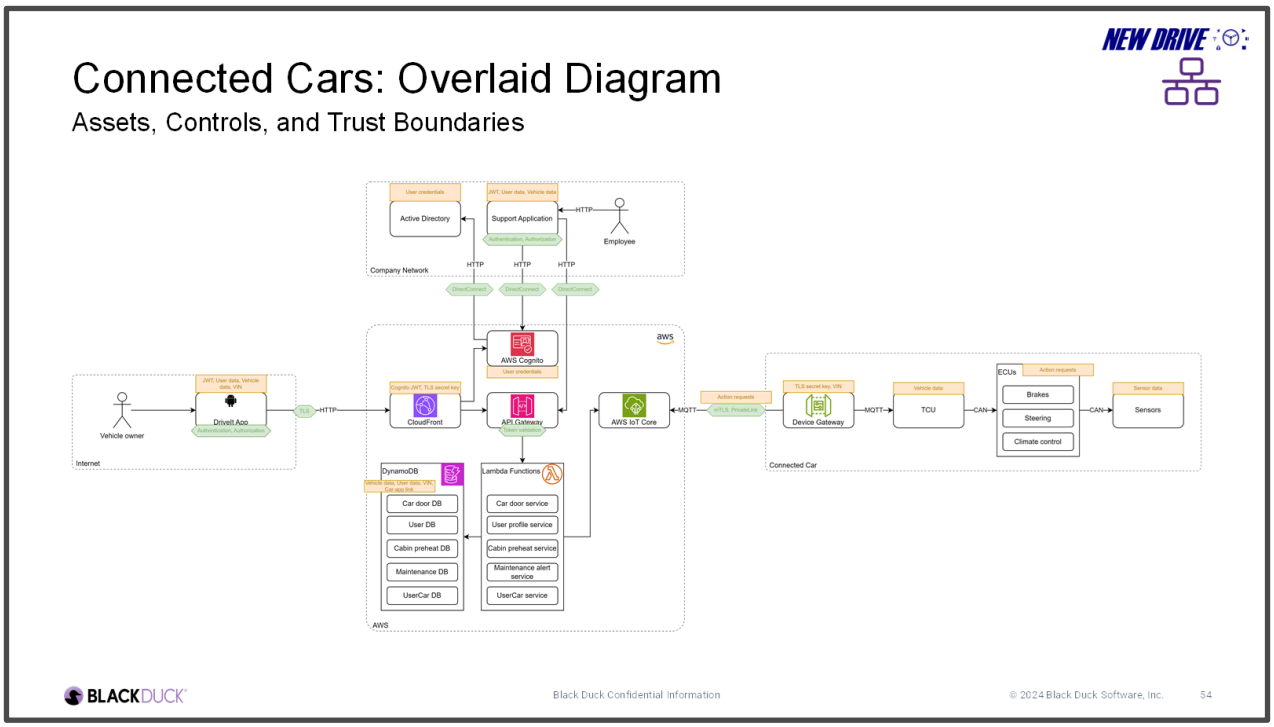
Slide 48

Operational controls are stronger than administrative, as they depend on configuration, but still may be bypassed by privileged individuals. Operational controls are strongest when coupled with tight technical controls.

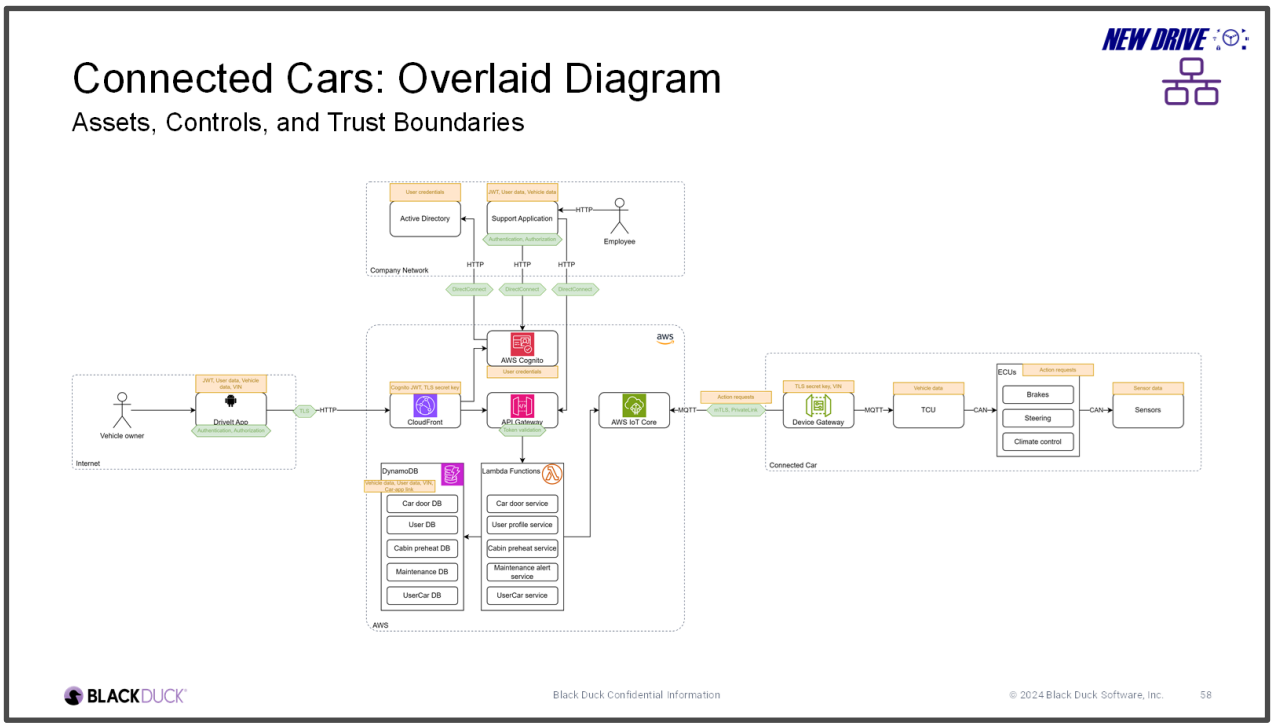


Slide 49

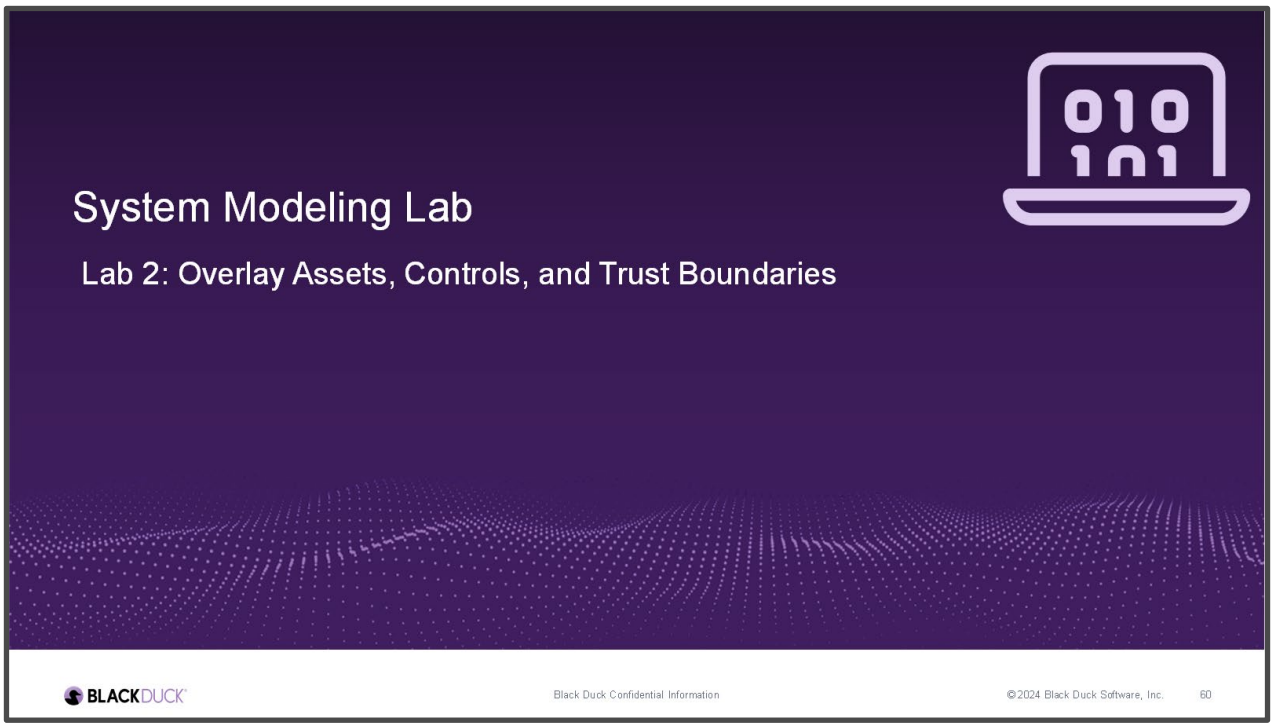
50cm of cold, hardened steel will beat a firewall every time.


NEW DRIVE
Connected Cars: Overlaid Diagram
Assets, Controls, and Trust Boundaries
Active Directory
Support Application
Employee
HTTP
Company Network
AWS Cognito
CloudFront
API Gateway
AWS IoT Core
Vehicle owner
DriveIt App
Internet
MQTT
Device Gateway
TCU
CAN
ECUs
Brakes
Steering
Climate control
Sensors
Connected Car
DynamoDB
Car door DB
User DB
Cabin preheat DB
Maintenance DB
UserCar DB
Lambda Functions
Car door service
User profile service
Cabin preheat service
Maintenance alert service
UserCar service
AWS
BLACKDUCK
Black Duck Confidential Information
© 2024 Black Duck Software, Inc.
54

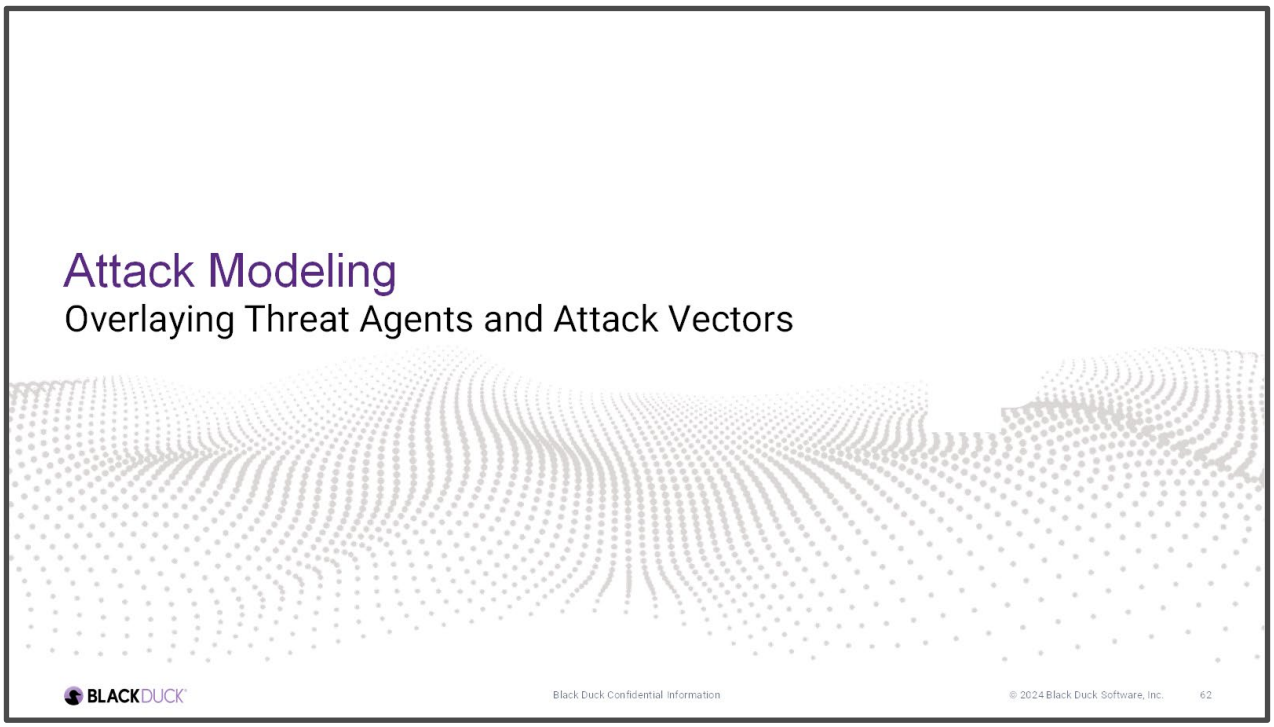
Slide 54


NEW DRIVE
Connected Cars: Overlaid Diagram
Assets, Controls, and Trust Boundaries
Active Directory
Support Application
Employee
Company Network
HTTP
AWS Cognito
CloudFront
API Gateway
AWS IoT Core
Vehicle owner
DriveIt App
Internet
DynamoDB
Lambda Functions
Car door DB
User DB
Cabin preheat DB
Maintenance DB
UserCar DB
Car door service
User profile service
Cabin preheat service
Maintenance alert service
UserCar service
AWS
MQTT
Device Gateway
TCU
CAN
ECUs
Brakes
Steering
Climate control
Sensors
Connected Car
BLACKDUCK
Black Duck Confidential Information
© 2024 Black Duck Software, Inc.
58

Slide 58


System Modeling Lab
Lab 2: Overlay Assets, Controls, and Trust Boundaries
BLACKDUCK
Black Duck Confidential Information
©2024 Black Duck Software, Inc.
60

Slide 60


Attack Modeling
Overlaying Threat Agents and Attack Vectors
BLACKDUCK
Black Duck Confidential Information
© 2024 Black Duck Software, Inc.
62

Slide 62

Think Like an Attacker

Threat agents

Threats

Attack goals

Attack surface

Attack vectors

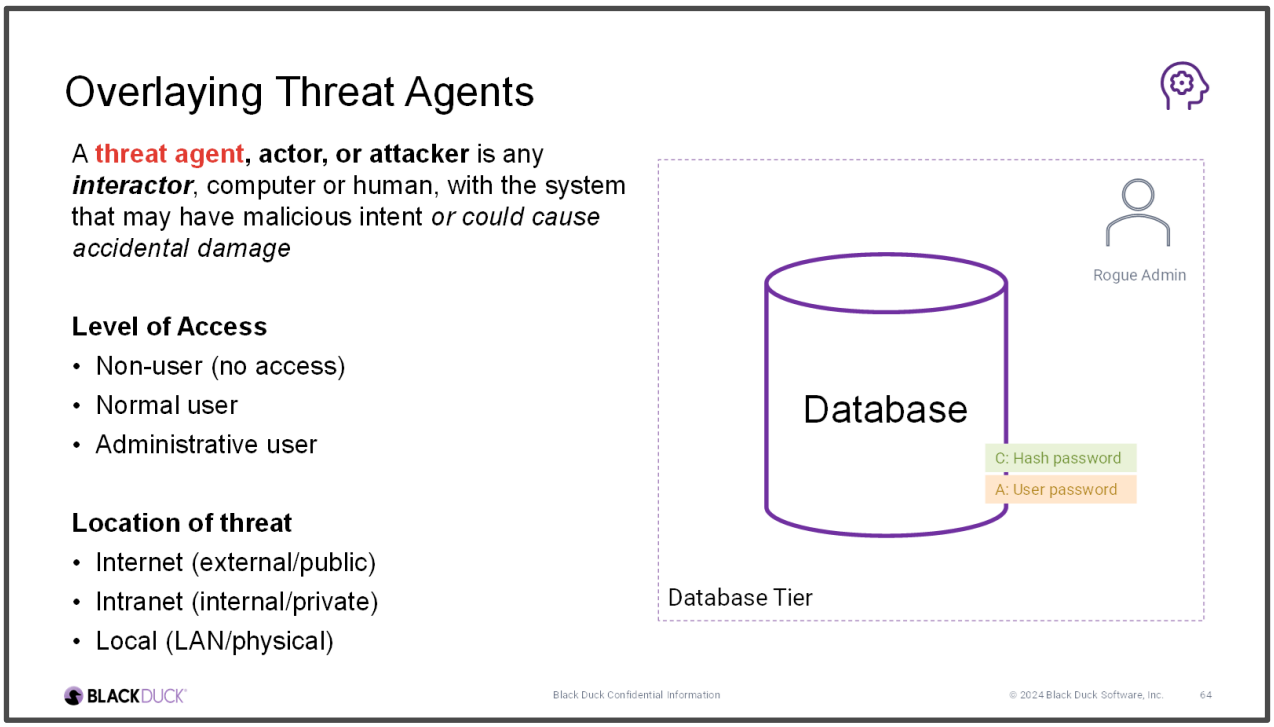


BLACKDUCK

Black Duck Confidential Information

© 2024 Black Duck Software, Inc. 63

Slide 63



Slide 64

Here are four canonical threat agents to start with for web applications:

- **Internet-based attacker:** somebody without credentials for accessing the app; can typically attack functionality that does not require authentication
- **LAN-based attacker:** somebody on the same LAN as an app user; can also carry out attacker-in-the-middle attacks
- **Malicious user:** somebody with app-layer credentials for the app; can attack functionality that requires authentication
- **Malicious app/system admin:** somebody with access to the data center network

It is difficult to protect against malicious internal attackers; however, numerous studies have shown that the majority of attacks that cause real damage are carried out by insiders (e.g., admins, developers, and customer service representatives). Think about how you would protect against malicious insiders. You often cannot implement preventative controls, so what can you do?

If you have an application hosted in a public cloud, you also have to worry about malicious cloud provider administrators. What do these threat agents have access to? How can you ensure that your code/data is safe?

With mobile devices, you should account for a threat agent with a jailbroken/rooted device. There are at least two different reasons why a device may be jailbroken/rooted:

- The device may have been jailbroken/rooted by malware. In this case, the malware could steal the user's data from your application.
- The user may have intentionally jailbroken/rooted their device. Now, code from an unknown source is running on the device with kernel/root privileges. This code could contain malware or serious vulnerabilities. Also, the user may have installed potentially malicious apps on the device and granted root privileges to them. Malicious apps could be stealing the user's data from your application.

© 2024 Black Duck

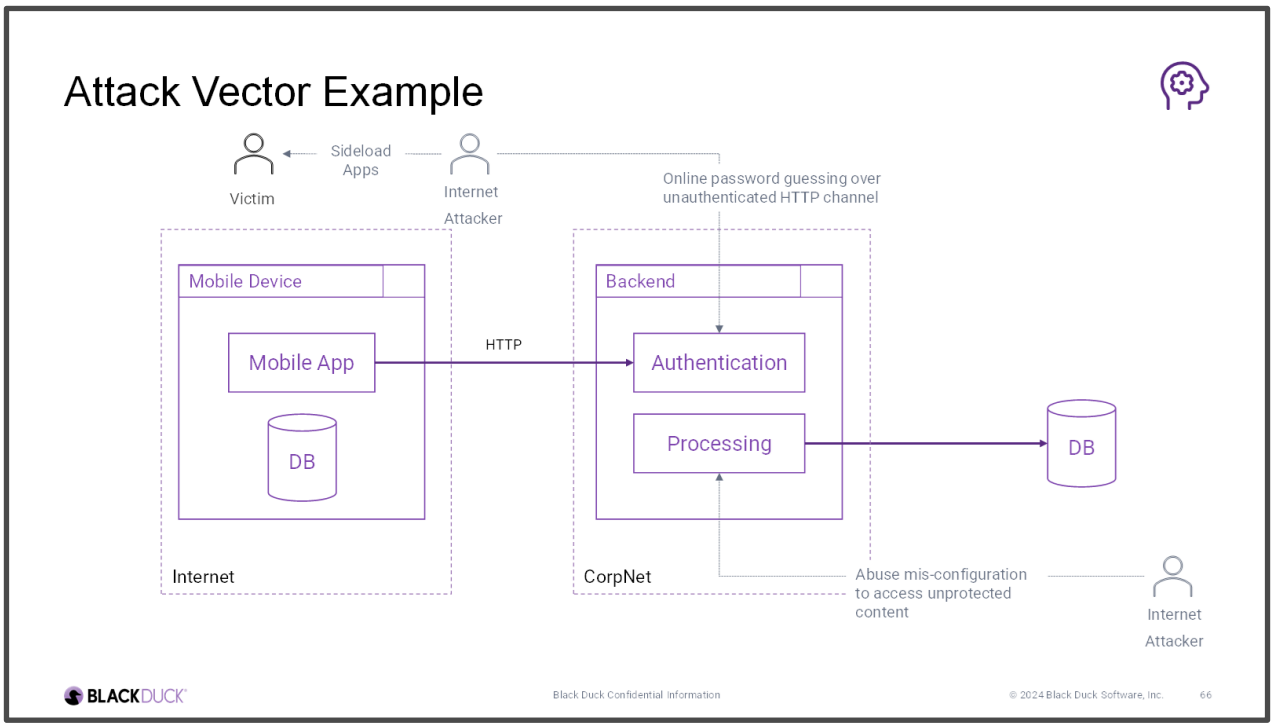
Attack Terminology



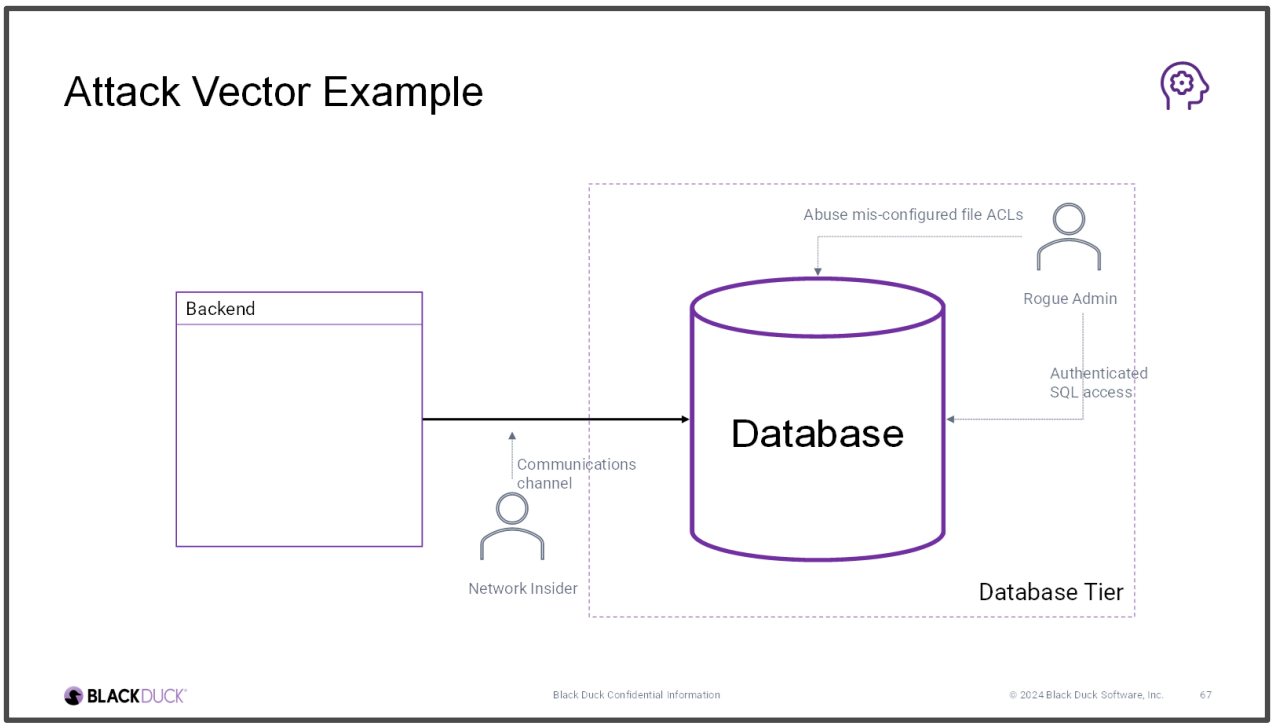
- An **attack** is any kind of malicious activity that has some effect in a system.
- An **attack vector** is the *means* and *method* used by an attacker to attempt an exploit.
- The **attack surface** is the union of all interaction points for a system.

BLACKDUCK Black Duck Confidential Information © 2024 Black Duck Software, Inc. 65

Slide 65


Attack Vector Example
Victim
Sideload Apps
Internet Attacker
Online password guessing over unauthenticated HTTP channel
Mobile Device
Mobile App
DB
HTTP
Backend
Authentication
Processing
DB
Internet
CorpNet
Abuse mis-configuration to access unprotected content
Internet Attacker
BLACKDUCK
Black Duck Confidential Information
© 2024 Black Duck Software, Inc.
66

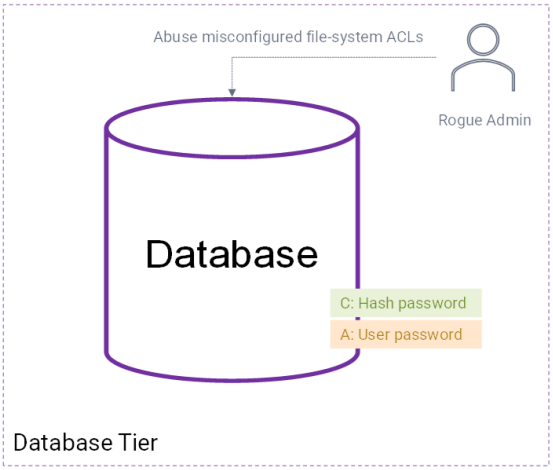
Slide 66


Attack Vector Example
Abuse mis-configured file ACLs
Rogue Admin
Backend
Authenticated
SQL access
Database
Communications
channel
Network Insider
Database Tier
BLACKDUCK
Black Duck Confidential Information
© 2024 Black Duck Software, Inc.
67

Slide 67

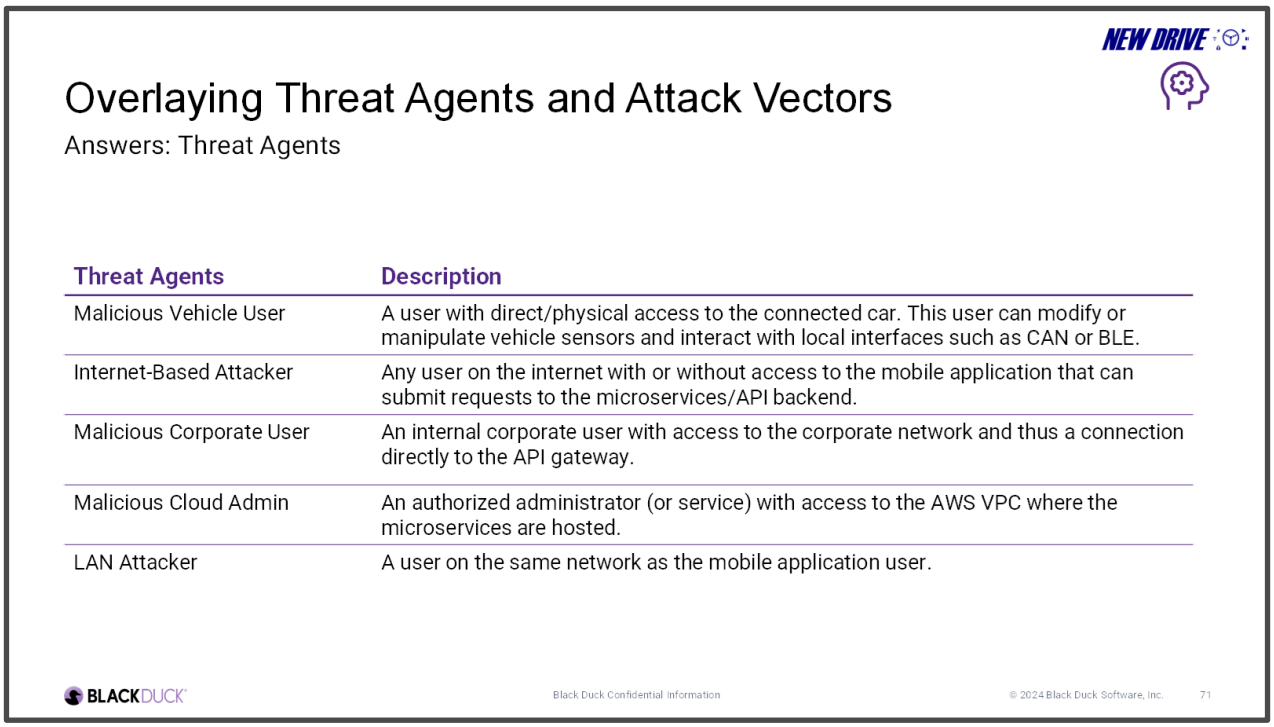
Overlaying Threat Agents and Attack Vectors

- Overlaying threat artifacts into a single model facilitates reasoning about the system.
- Does the system protect against this attack vector?

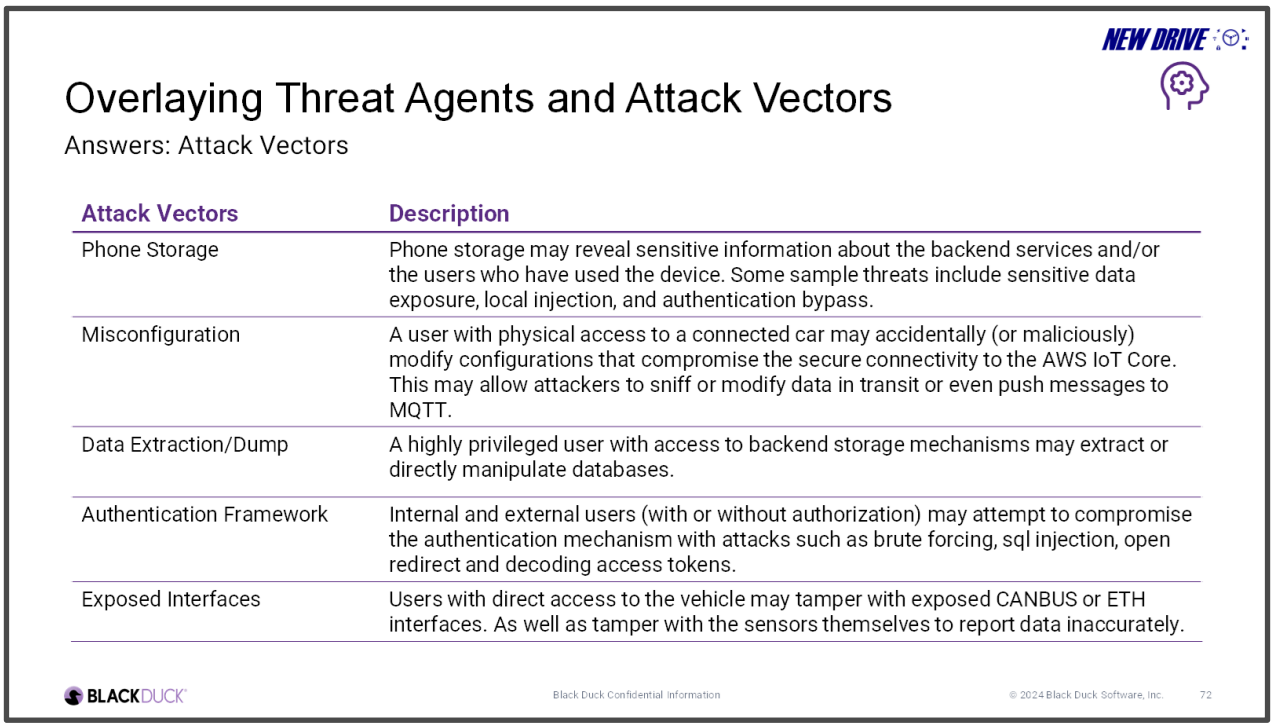


BLACKDUCK
Black Duck Confidential Information
© 2024 Black Duck Software, Inc.

Slide 68


NEW DRIVE
Overlaying Threat Agents and Attack Vectors
Answers: Threat Agents
Threat Agents
Description
Malicious Vehicle User
A user with direct/physical access to the connected car. This user can modify or manipulate vehicle sensors and interact with local interfaces such as CAN or BLE.
Internet-Based Attacker
Any user on the internet with or without access to the mobile application that can submit requests to the microservices/API backend.
Malicious Corporate User
An internal corporate user with access to the corporate network and thus a connection directly to the API gateway.
Malicious Cloud Admin
An authorized administrator (or service) with access to the AWS VPC where the microservices are hosted.
LAN Attacker
A user on the same network as the mobile application user.
BLACKDUCK
Black Duck Confidential Information
© 2024 Black Duck Software, Inc.
71

Slide 71



NEW DRIVE

Overlaying Threat Agents and Attack Vectors

Answers: Attack Vectors

Attack Vectors	Description
Phone Storage	Phone storage may reveal sensitive information about the backend services and/or the users who have used the device. Some sample threats include sensitive data exposure, local injection, and authentication bypass.
Misconfiguration	A user with physical access to a connected car may accidentally (or maliciously) modify configurations that compromise the secure connectivity to the AWS IoT Core. This may allow attackers to sniff or modify data in transit or even push messages to MQTT.
Data Extraction/Dump	A highly privileged user with access to backend storage mechanisms may extract or directly manipulate databases.
Authentication Framework	Internal and external users (with or without authorization) may attempt to compromise the authentication mechanism with attacks such as brute forcing, sql injection, open redirect and decoding access tokens.
Exposed Interfaces	Users with direct access to the vehicle may tamper with exposed CANBUS or ETH interfaces. As well as tamper with the sensors themselves to report data inaccurately.

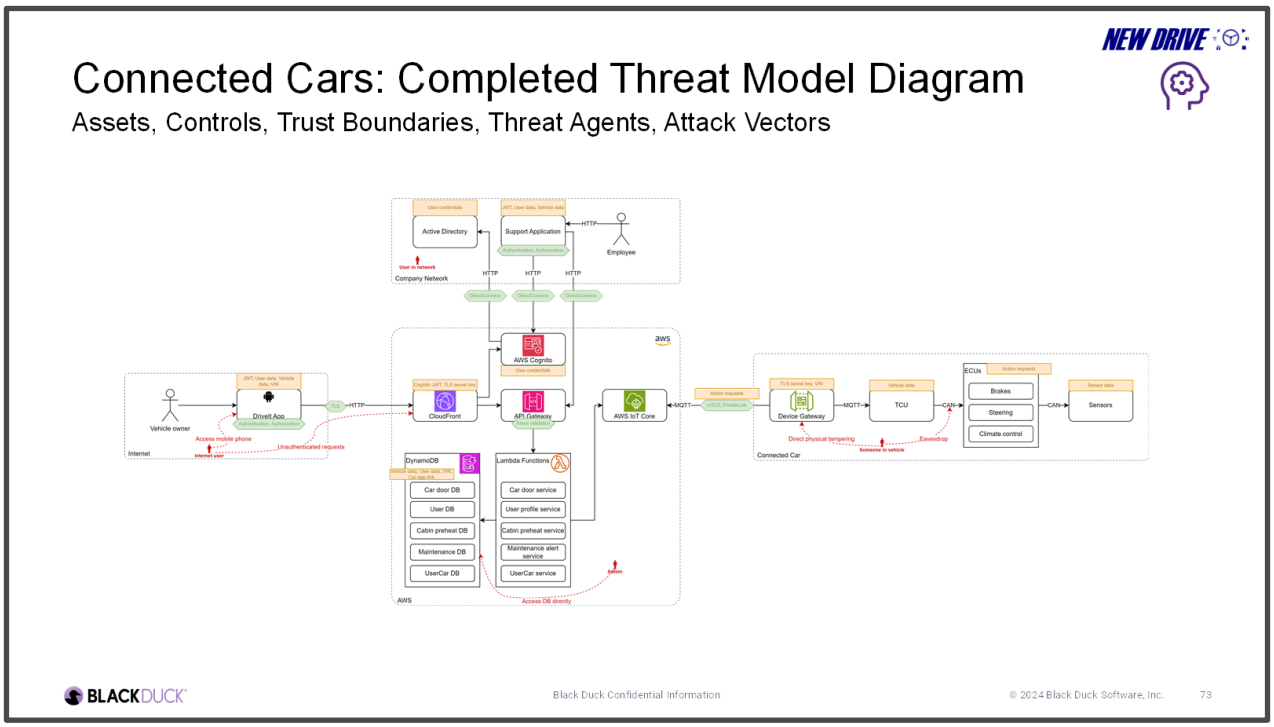
BLACKDUCK

Black Duck Confidential Information

© 2024 Black Duck Software, Inc.

72

Slide 72



Slide 73

Here is what a component diagram with flows for this application could look like.


Attack Modeling Lab
Lab 3: Overlay Threat Agents and Attack Vectors
BLACKDUCK
Black Duck Confidential Information
©2024 Black Duck Software, Inc.
75

Slide 75

Lab Instructions

Overlay Threat Agents and Attack Vectors on Component Diagram

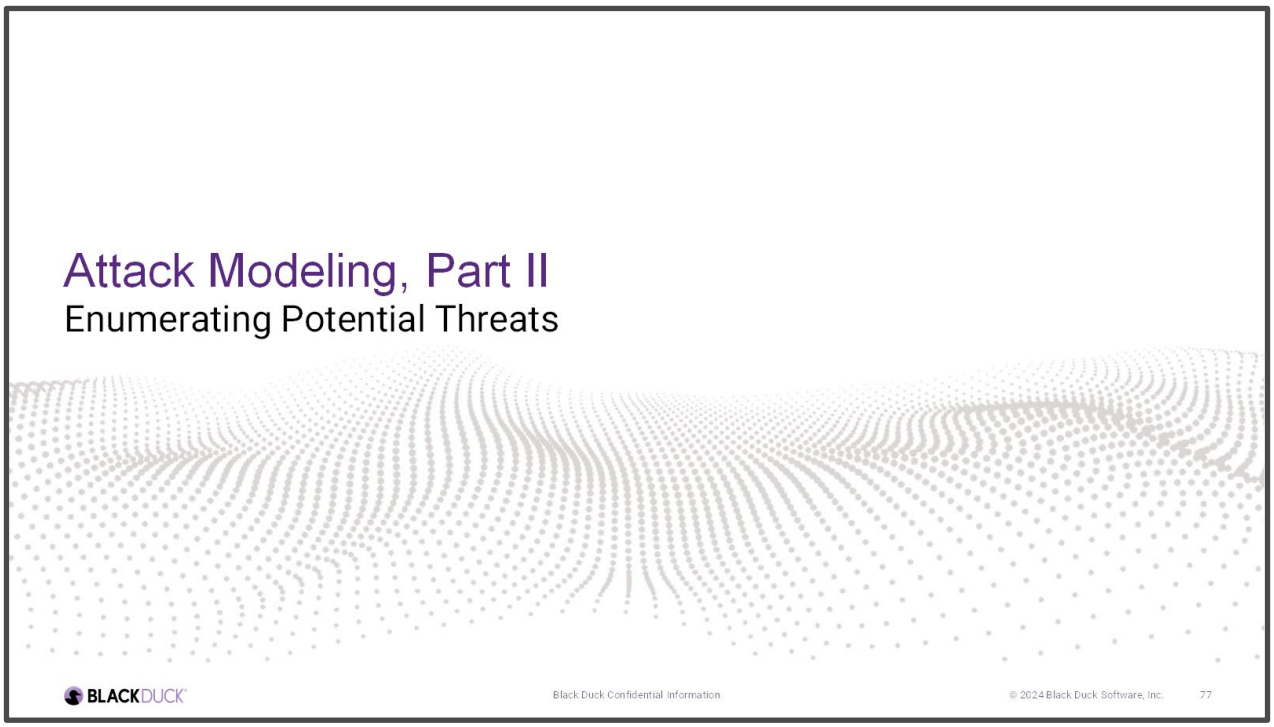


Complete the Threat Model Diagram

- Using the Black Duck Stencil, update the component diagram with threat agents and attack vectors
- It should account for realistic threat agents and the most common attack vectors

BLACKDUCK Black Duck Confidential Information © 2024 Black Duck Software, Inc. 76

Slide 76


Attack Modeling, Part II
Enumerating Potential Threats
BLACKDUCK
Black Duck Confidential Information
© 2024 Black Duck Software, Inc.
77

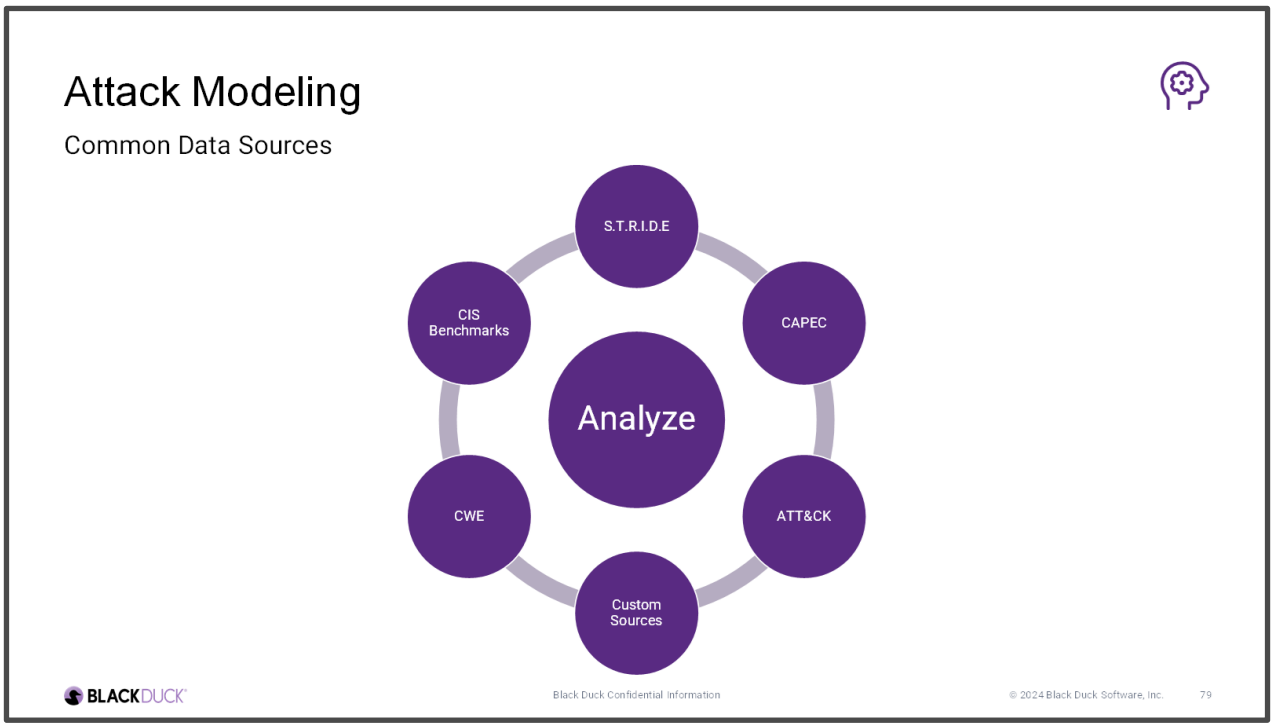
Slide 77



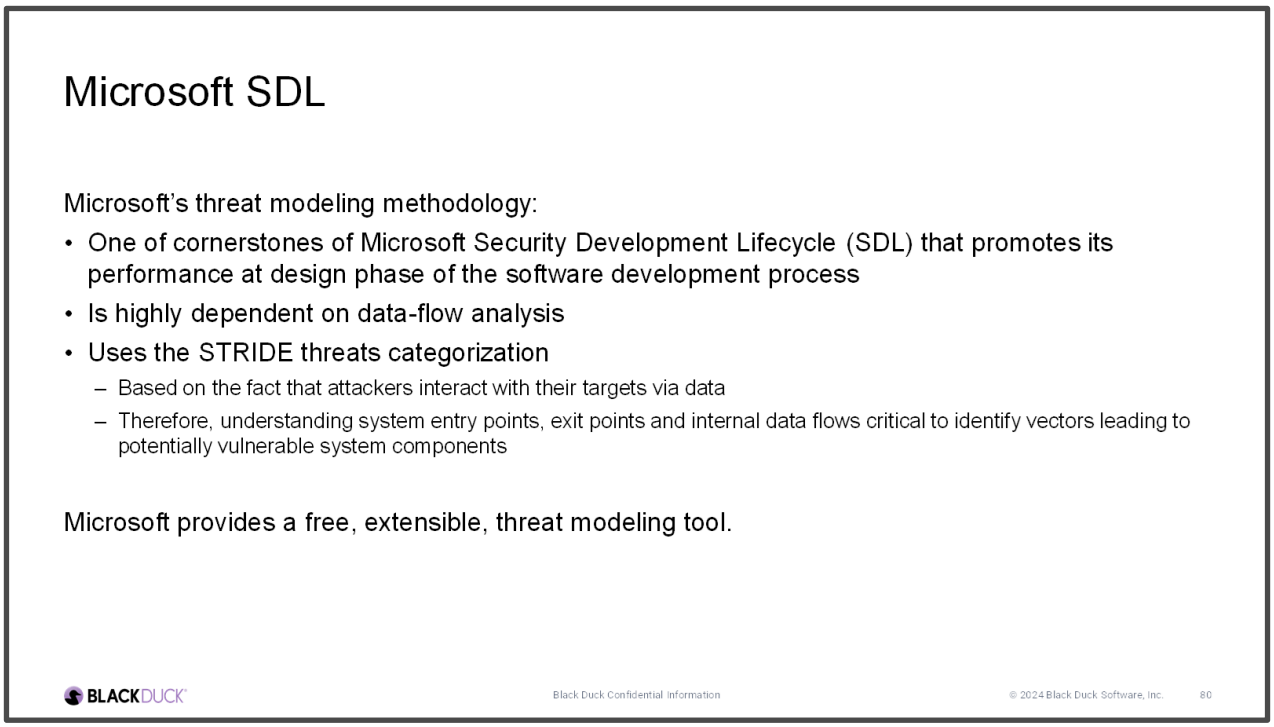
Slide 78

There are many threat modeling methodologies. More information can be found by visiting the following resources:

- Security CARDS: https://securitycards.cs.washington.edu/
- PnG: https://resources.sei.cmu.edu/library/asset-view.cfm?AssetID=516617
- TVR: https://www.etsi.org/deliver/etsi_ts/102100_102199/10216501/05.02.03_60/ts_10216501v050203p.pdf
- STRIDE, based on the OWASP page: https://www.owasp.org/index.php/Threat_Risk_Modeling

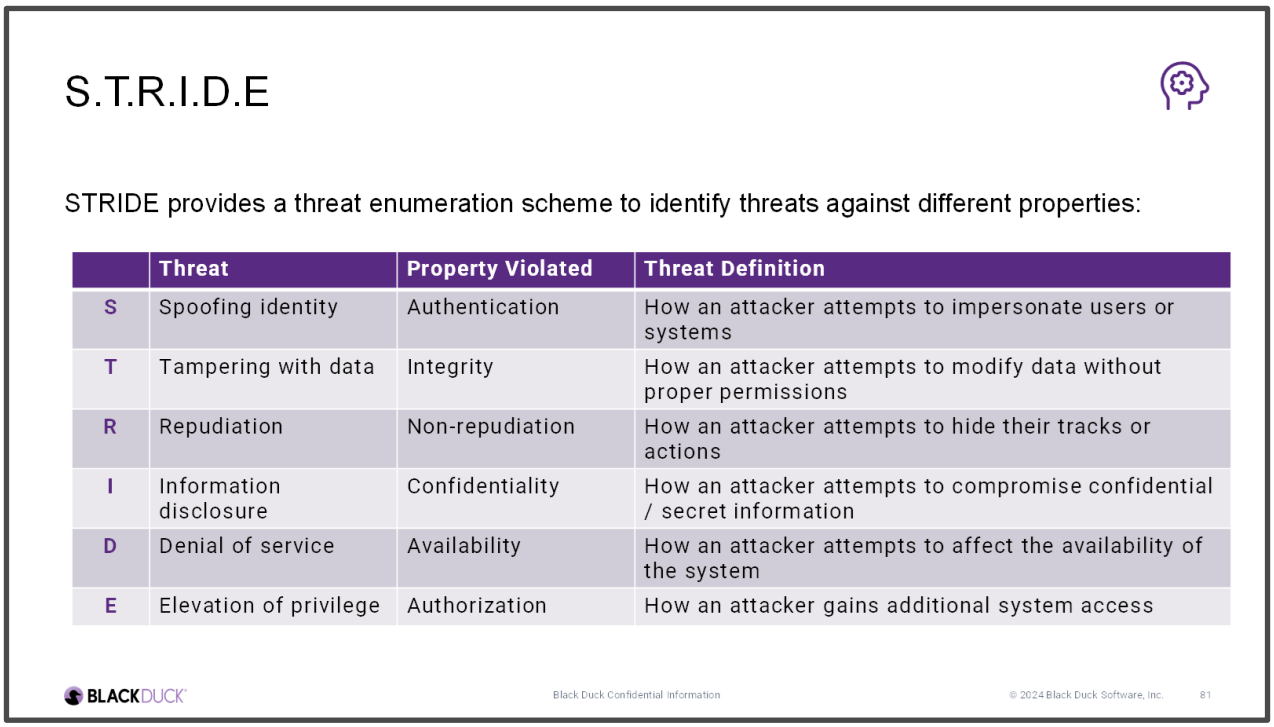

Attack Modeling
Common Data Sources
S.T.R.I.D.E
CAPEC
ATT&CK
Custom Sources
CWE
CIS Benchmarks
Analyze
BLACKDUCK
Black Duck Confidential Information
© 2024 Black Duck Software, Inc.
79

Slide 79



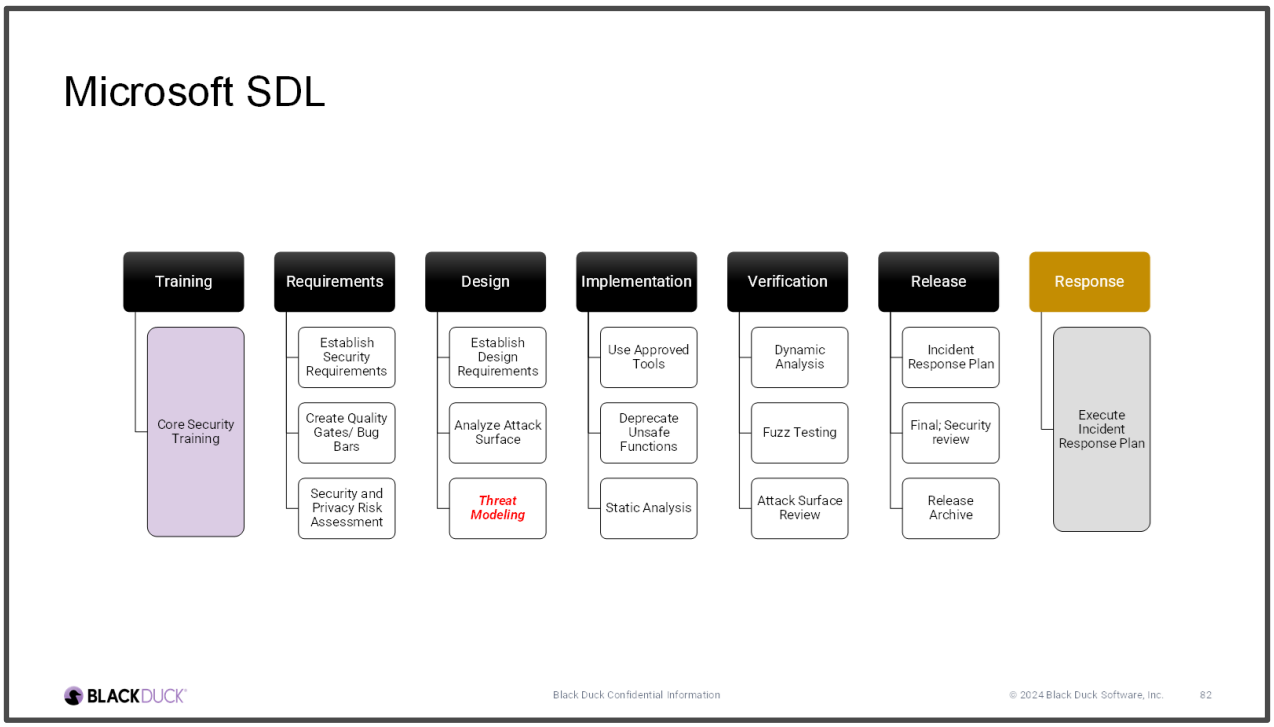
Slide 80

Microsoft's STRIDE, in the context of their SDL, is a popular way to break down the analysis process for finding design and implementation weaknesses.



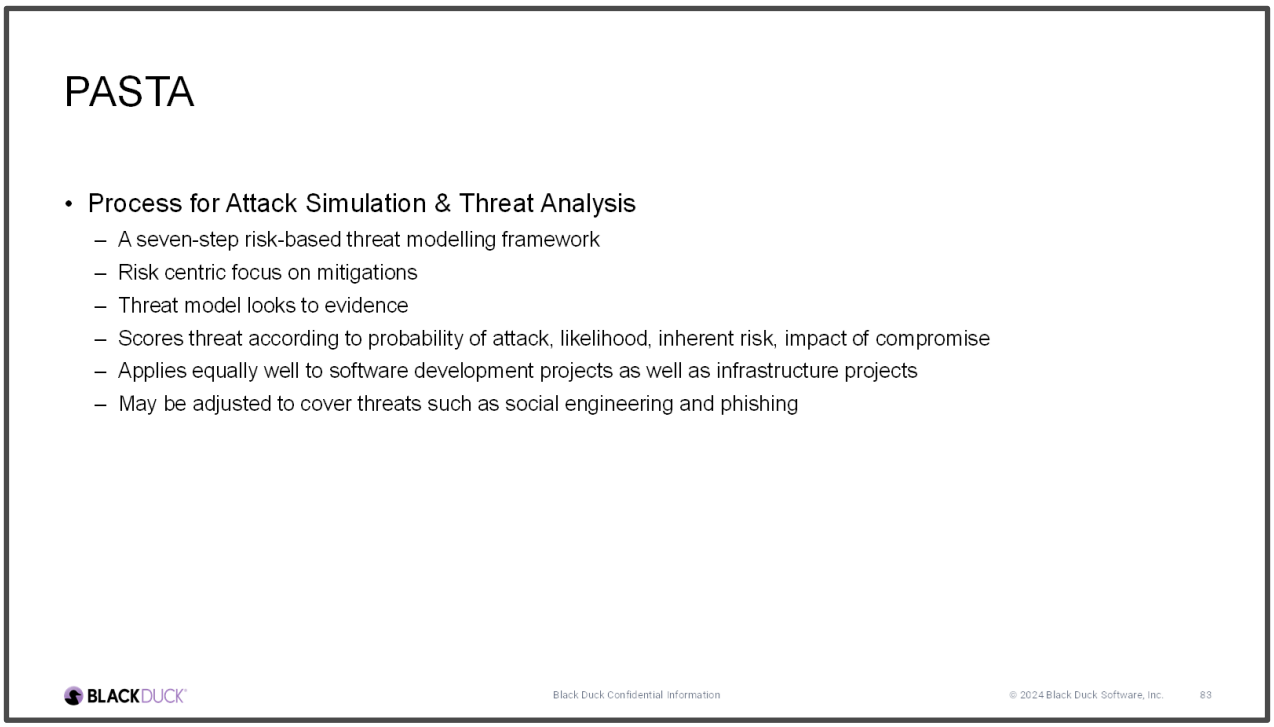
Slide 81

This article from STRIDE explains the method in more detail: https://docs.microsoft.com/en-us/archive/msdn-magazine/2006/november/uncover-security-design-flaws-using-the-stride-approach.



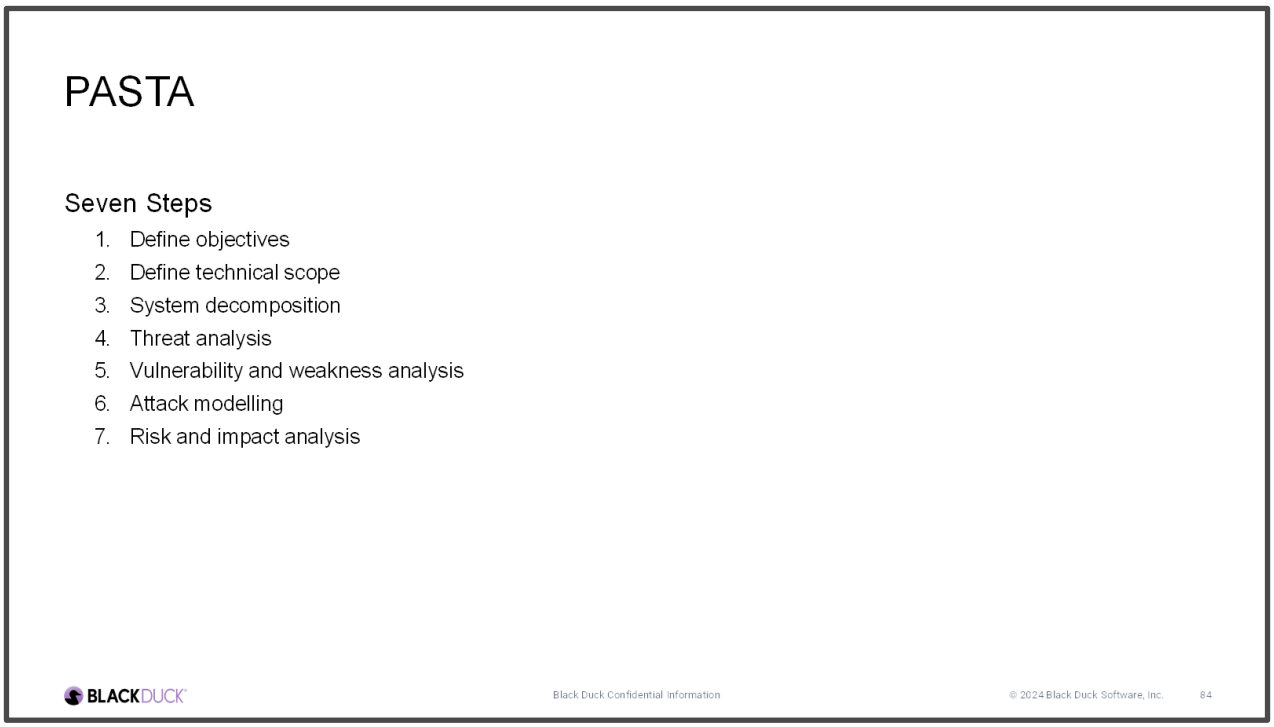
Slide 82

No matter what the process for Threat Modeling, the work has to be conducted in the context of a larger holistic approach. Creating a threat model by itself and then walking away is of little value.



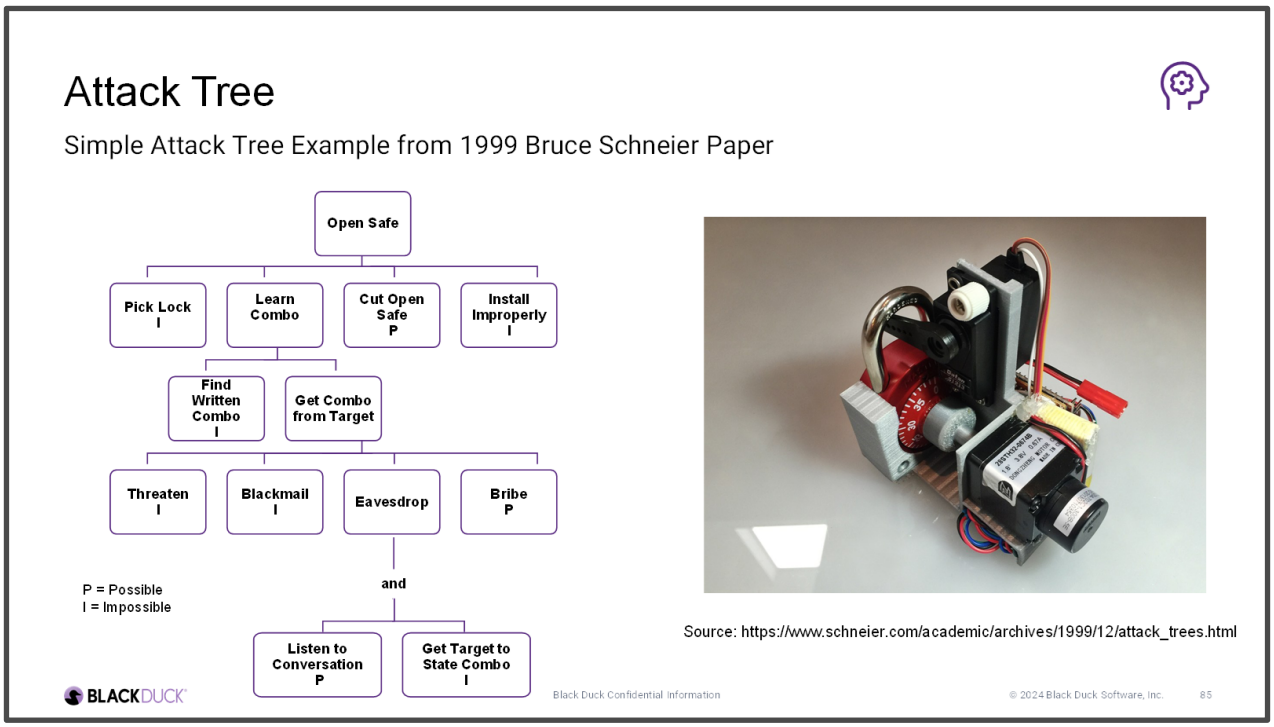
Slide 83

PASTA is another way to break down the process and maintain consistent results.



Slide 84

Note that PASTA is really just a logical way to break the problem down. When first approaching Threat Modeling, a consistent approach helps to ensure that the execution of the work doesn't get ahead of itself. There is a reason experienced professional pilots always execute a checklist before a flight.



Slide 85

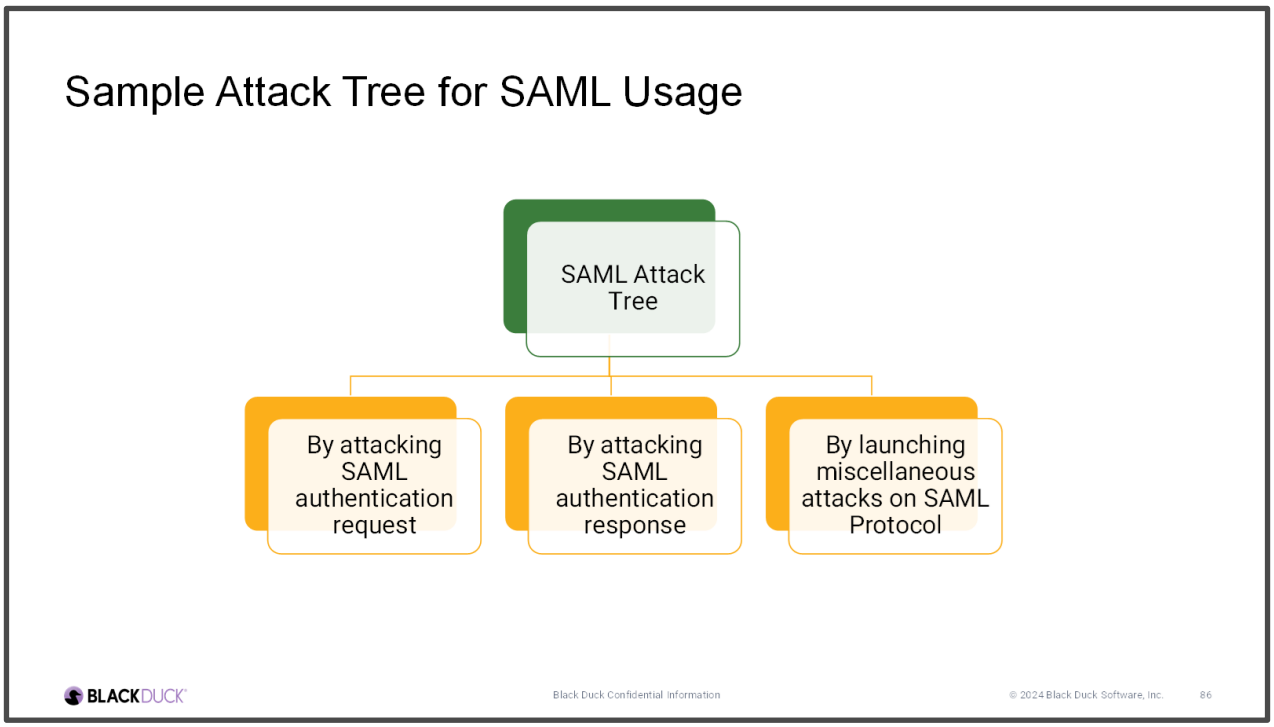
The attack tree on this slide illustrates how to attack a safe. It has been created by Bruce Schneier: https://www.schneier.com/academic/archives/1999/12/attack_trees.html.

The goal of the attack is to open the safe. To do so, attackers can pick the lock, learn the combination, cut the safe open, or install the safe improperly so that they can easily open it later.

To learn the combination, they have to find the combination written down or obtain the combination from the safe owner. To get the combination from the safe owner, the attacker can either threaten, blackmail, eavesdrop or bribe the safe owner. To eavesdrop, the attacker needs to listen to the conversation of the safe owner and get the combination that was said during such conversation. Note that each node is a subgoal and that children of that node achieve that subgoal.

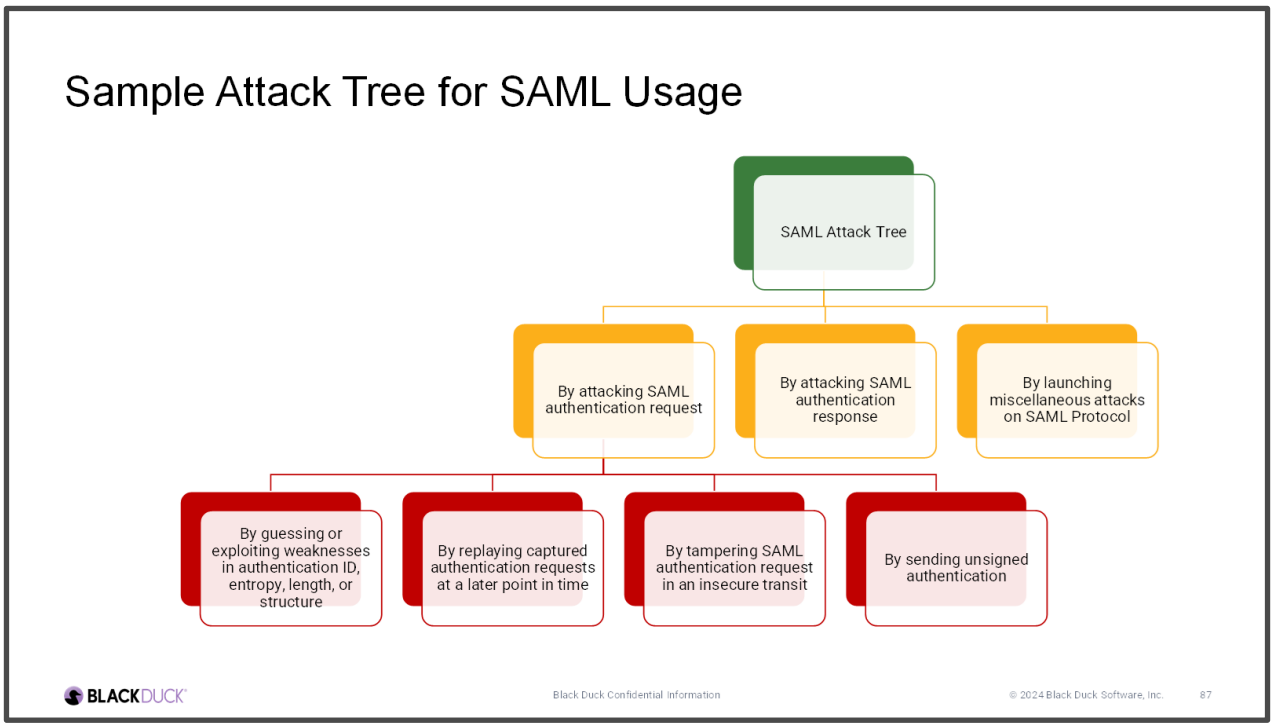
As you can see in the picture, there are AND nodes and OR nodes. OR nodes are alternatives. For instance, the four ways to open a safe are alternatives. AND nodes are the different steps that need to be met to achieve a goal. For instance, to eavesdrop on the safe-owner saying the safe combination, attackers have to eavesdrop on the conversation AND get safe owners to say the combination. Thus, both subgoals need to be satisfied.

During the next step of the analysis, you validate how secure the safe is by assigning each of the leaf nodes a value of I (impossible) or P (Possible) and using those values to calculate the overall security of the goal. Note that the value of an OR node is possible if any of its children are possible, while the value of an AND node is possible only if all children are possible.



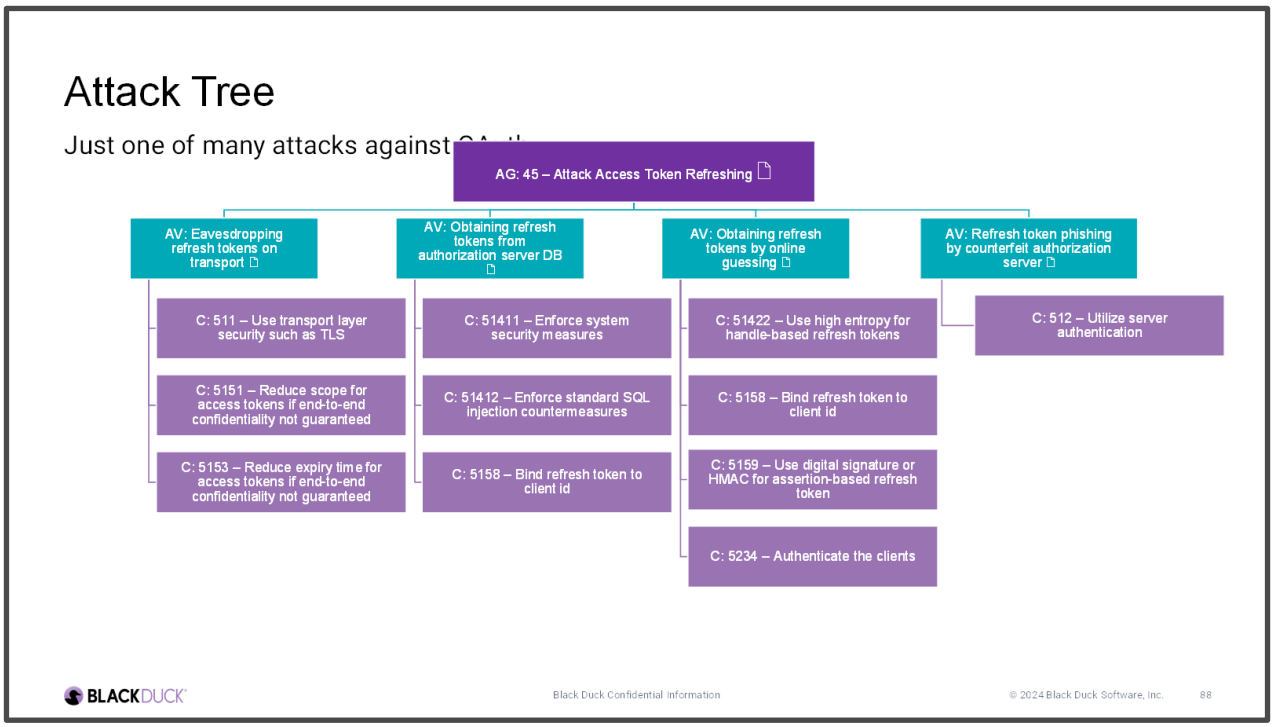
Slide 86

A security panel for SAML could break down SAML attacks into the attack tree listed on this page.



Slide 87

The attack against the SAML authentication request can be grouped together into four different kinds of attacks.



Slide 88

The previous example showed some simple examples. On this slide, we perform this technique on a real-world system that uses OAuth2 with refresh tokens. As a quick reminder, OAuth2 is a delegation framework that enables users to give a third party access to his resources without sharing the user's password to those third parties. For instance, a user can give a printing website (third party, OAuth client) access to his pictures (resource) that are hosted on google Picasa (Service Provider) without giving the printing website the user's password of Google Picasa. As part of the OAuth2 flow, Google Picasa (Service Provider) gives the printing website (OAuth2 Client) access tokens and refresh tokens. The access tokens are short-lived and the printing website (OAuth2 client) can use them to access the pictures (resources) at Google Picasa. The refresh-tokens are valid for a much longer time and the printing website (OAuth2 Client) can exchange them for new access tokens by contacting Google Picasa (service provider). Naturally, the printing website should protect the refresh token as anyone with access to it can use them to obtain new access tokens and use those access tokens to access the user's resources (pictures).

The attack tree itself shows how one can obtain such refresh tokens. The refresh tokens can be obtained by either eavesdropping on them during communication between the Service Provider and the OAuth2 Client, by obtaining them from the database of the server, or by guessing them. As you can see, creating an attack tree for a complicated system gets very large very quickly **and** it requires you understand and know about all attacks (in this case OAuth2).

CAPEC

Common Attack Pattern Enumeration and Classification

Provide a comprehensive dictionary of known patterns of attack employed by adversaries to exploit known weaknesses in cyber-enabled capabilities.

Mechanisms of Attacks	Focus
Engage in Deceptive Interactions	Spoofing of data interactions between principles
Abuse Existing Functionality	Weaknesses in business logic and application functionality
Manipulate Data Structure	Data structure weakness such as buffer manipulation
Manipulate System Resources	The adversary's ability to manipulate one or more resources
Inject Unexpected Items	Weaknesses with data input and installation
Employ Probabilistic Techniques	Fuzzing and brute forcing
Manipulate Timing and State	Race conditions and timing weaknesses
Collect and Analyze Information	Gathering, collection, and theft of information
Subvert Access Control	Weaknesses to identity, authentication and authorization

BLACKDUCK Black Duck Confidential Information © 2024 Black Duck Software, Inc. 89

Slide 89

MITRE ATT&CK

Knowledge base of adversary tactics and techniques based on real-world observations

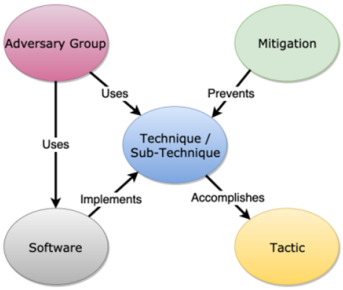
Name	The adversary is trying to…
Reconnaissance	… gather information they can use to plan future operations
Resource Development	…establish resources they can use to support operations
Initial Access	…get into your network
Execution	…run malicious code
Persistence	…maintain their foothold
Privilege Escalation	…gain higher-level permissions
Defense Evasion	…avoid being detected
Credential Access	…steal account names and passwords
Discovery	…figure out your environment
Lateral Movement	…move through your environment
Collection	…gather data of interest to their goal
Command and Control	…communicate with compromised systems to control them
Exfiltration	…steal data
Impact	…manipulate, interrupt, or destroy your systems and data

BLACKDUCK Black Duck Confidential Information © 2024 Black Duck Software, Inc. 90

Slide 90

MITRE ATT&CK®

All ATT&CK tables based on a standard model relationship

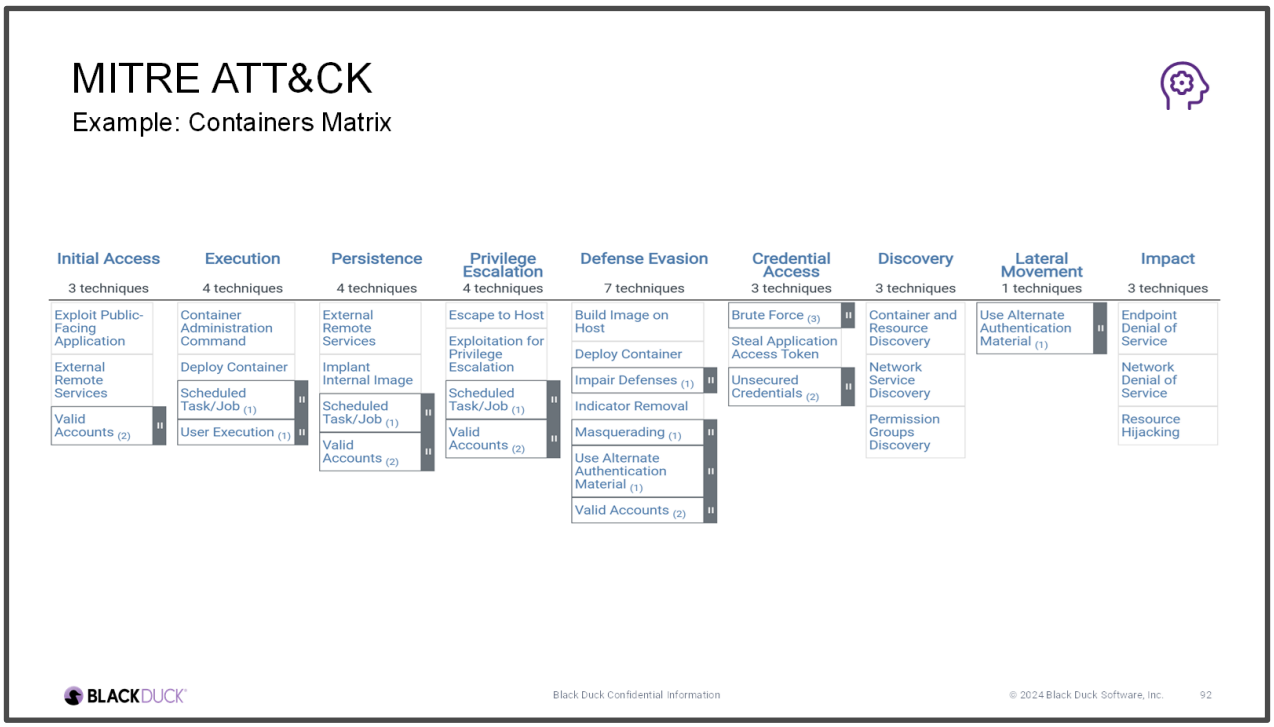


From: MITRE ATT&CK®: Design and Philosophy

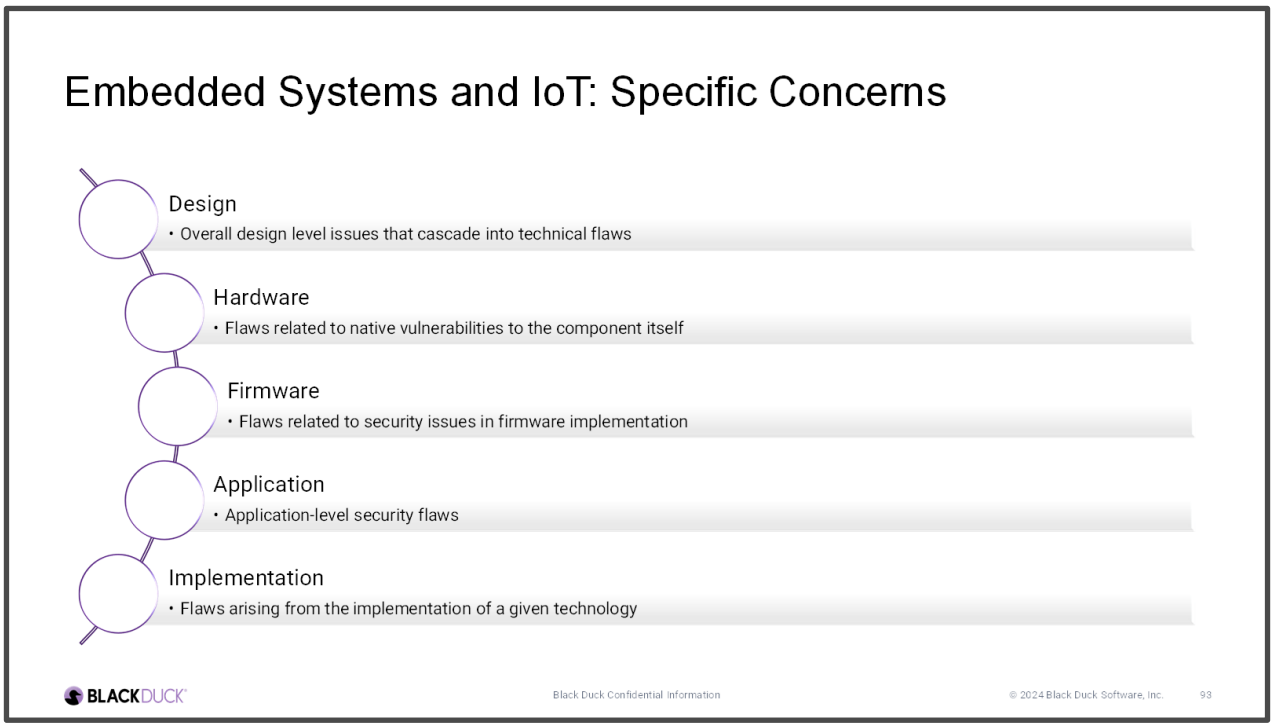
BLACKDUCK
Black Duck Confidential Information
© 2024 Black Duck Software, Inc. 91

Slide 91

Keep this in mind as you navigate the MITRE ATT&CK framework. This diagram helps to keep some clarity to the complex taxonomy the framework uses.


MITRE ATT&CK
Example: Containers Matrix
Initial Access
3 techniques
Exploit Public-Facing Application
External Remote Services
Valid Accounts (2)
Execution
4 techniques
Container Administration Command
Deploy Container
Scheduled Task/Job (1)
User Execution (1)
Persistence
4 techniques
External Remote Services
Implant Internal Image
Scheduled Task/Job (1)
Valid Accounts (2)
Privilege Escalation
4 techniques
Escape to Host
Exploitation for Privilege Escalation
Scheduled Task/Job (1)
Valid Accounts (2)
Defense Evasion
7 techniques
Build Image on Host
Deploy Container
Impair Defenses (1)
Indicator Removal
Masquerading (1)
Use Alternate Authentication Material (1)
Valid Accounts (2)
Credential Access
3 techniques
Brute Force (3)
Steal Application Access Token
Unsecured Credentials (2)
Discovery
3 techniques
Container and Resource Discovery
Network Service Discovery
Permission Groups Discovery
Lateral Movement
1 techniques
Use Alternate Authentication Material (1)
Impact
3 techniques
Endpoint Denial of Service
Network Denial of Service
Resource Hijacking
BLACKDUCK
Black Duck Confidential Information
© 2024 Black Duck Software, Inc.
92

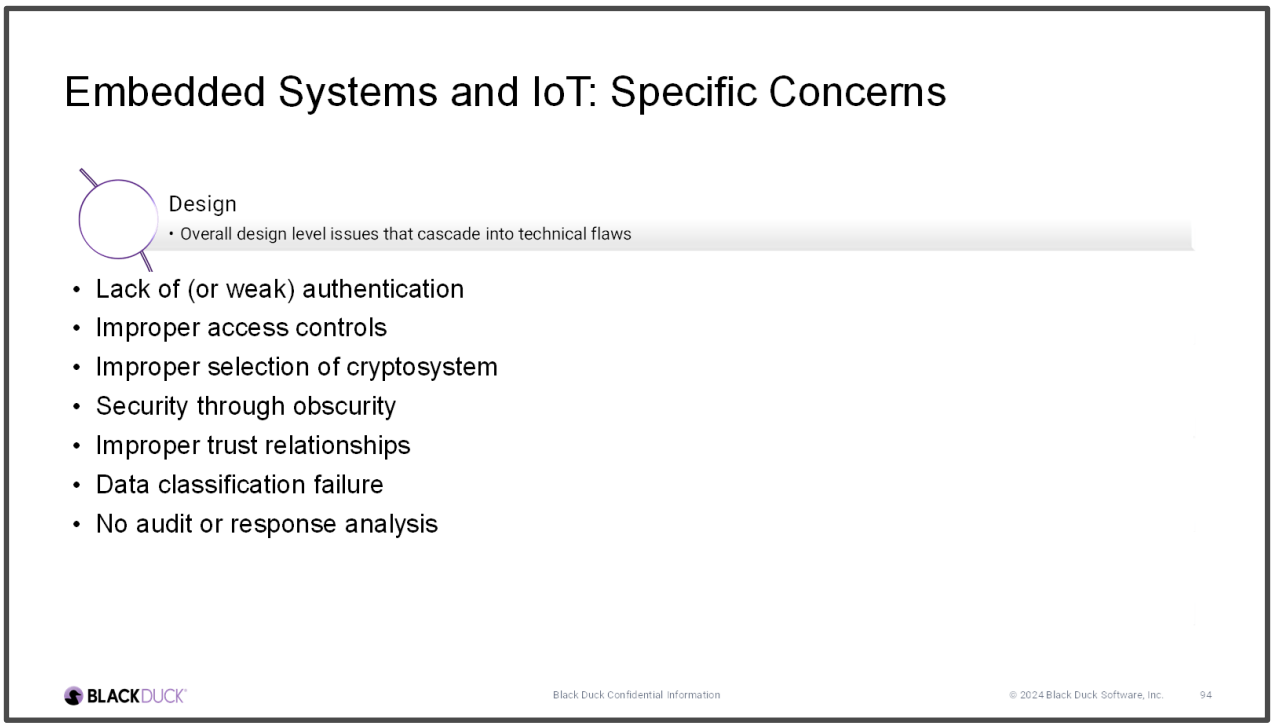
Slide 92



Slide 93

The lines between these categorizations can become blurry when viewed as an entire implementation of an embedded or IoT system.

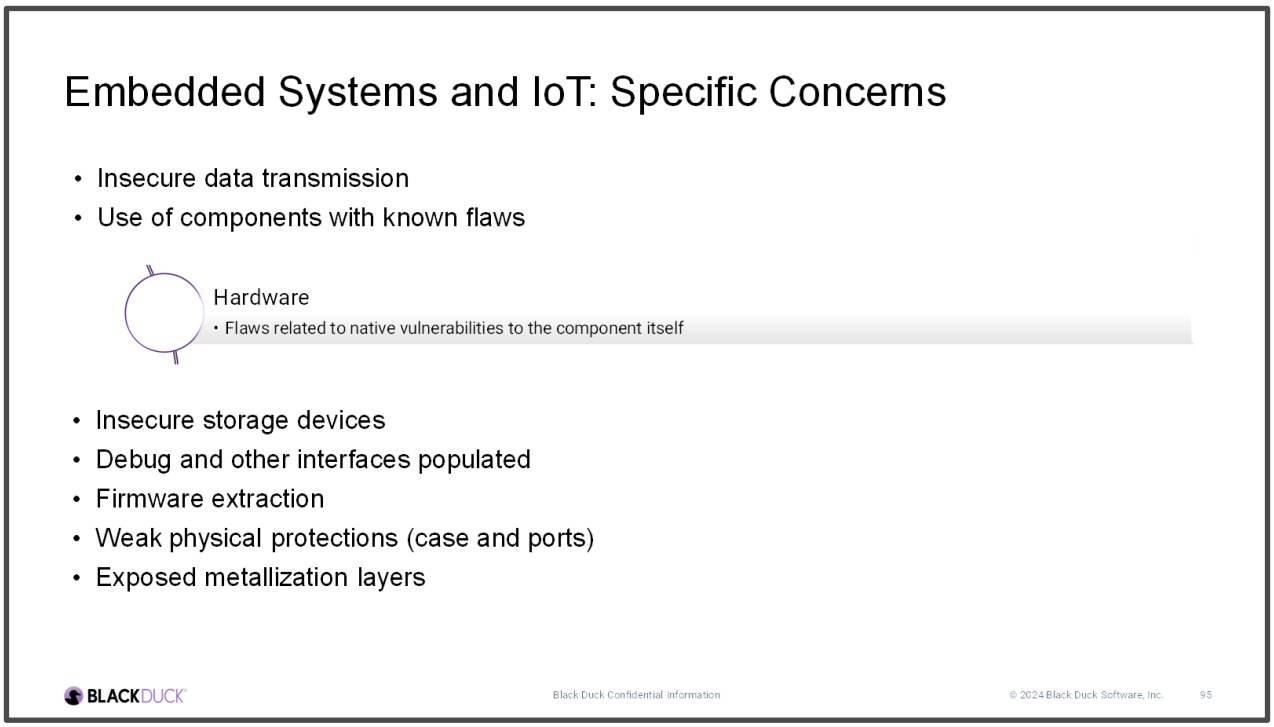
An important note to remember as we continue throughout the module is that many of these issues are systems design oversights that manifested themselves in technical vulnerabilities.



Slide 94

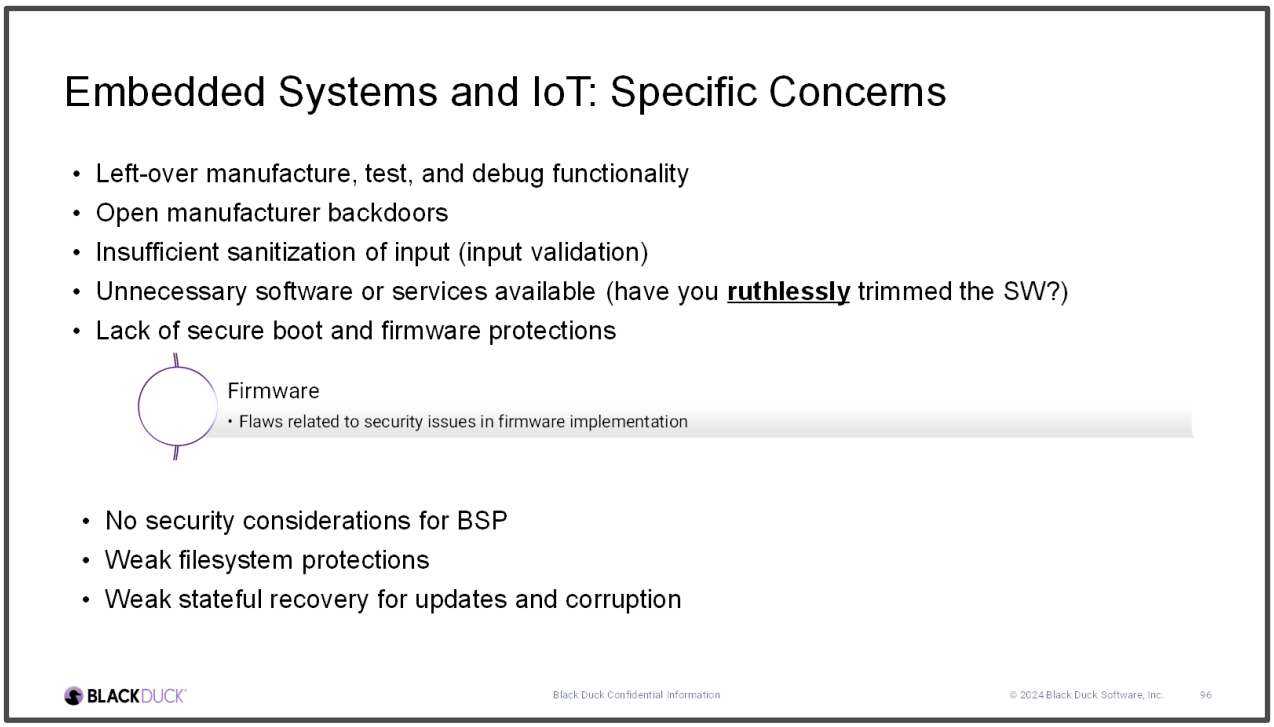
Many design-level issues, in addition to some of the other classifications presented in this module, stem from a lack of clarity or rigor placed on designing security into the system.

Many of these issues can be remediated or addressed by the proper inclusion of security requirements and analysis in the initial stages of system development.



Slide 95

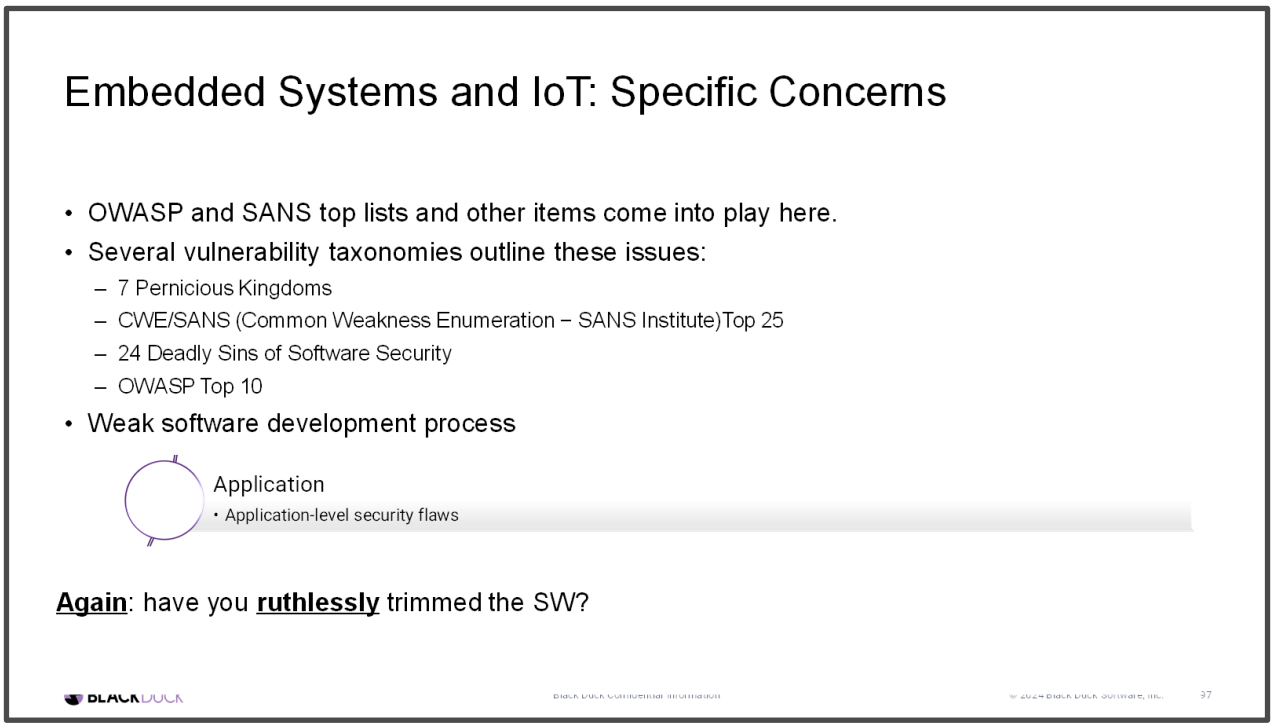
As we observed in the last group, often errors in requirements specification, design decisions, and data classifications cascade to hardware decisions that affect the security of a device or system. Security must be considered during the hardware selection process and in the use of its features.



Slide 96

In the firmware category, you see many vulnerabilities related to functionality that can be exploited or misused. This encompasses intentional functionality that may have been left on a production image, or functionality that remains based on a failure to enumerate the software on a purchased build before use.

Firmware is also where we find our more traditional software vulnerabilities, such as buffer overflows, off-by-one, and integer wraparounds.

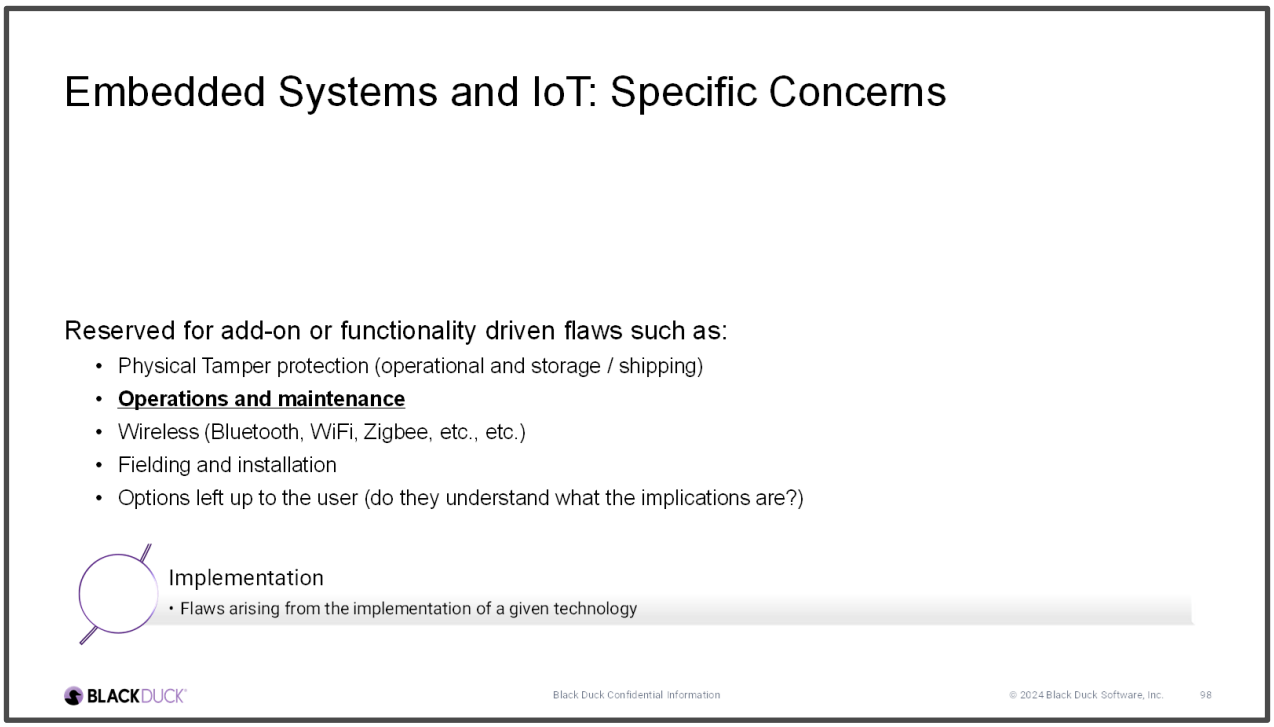


Slide 97

There is a common thought that an evaluation of software vulnerabilities is not relevant to those that develop some types of embedded systems.

As we have seen however, these well-documented issues in the application security domain are analogous to embedded systems' vulnerabilities and much can be gained through their study.

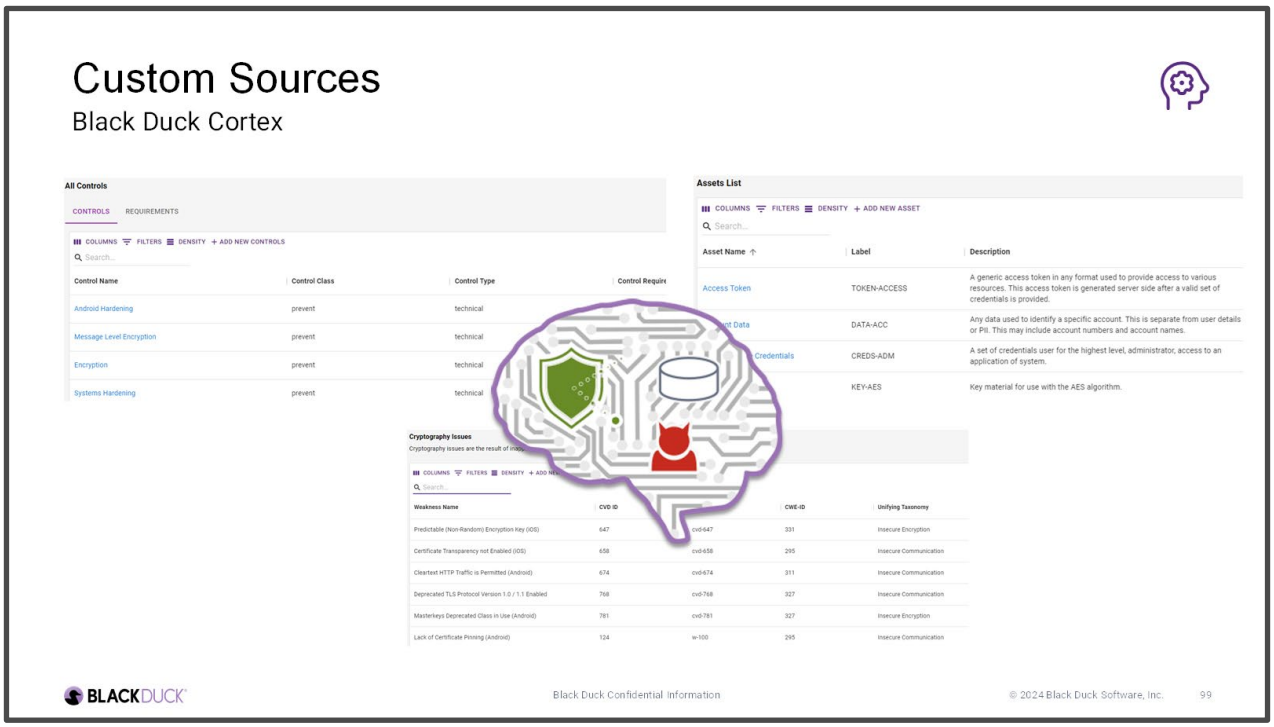
Remember as well, an embedded product is not a desktop system. The functions should be trimmed to match the need.



Slide 98

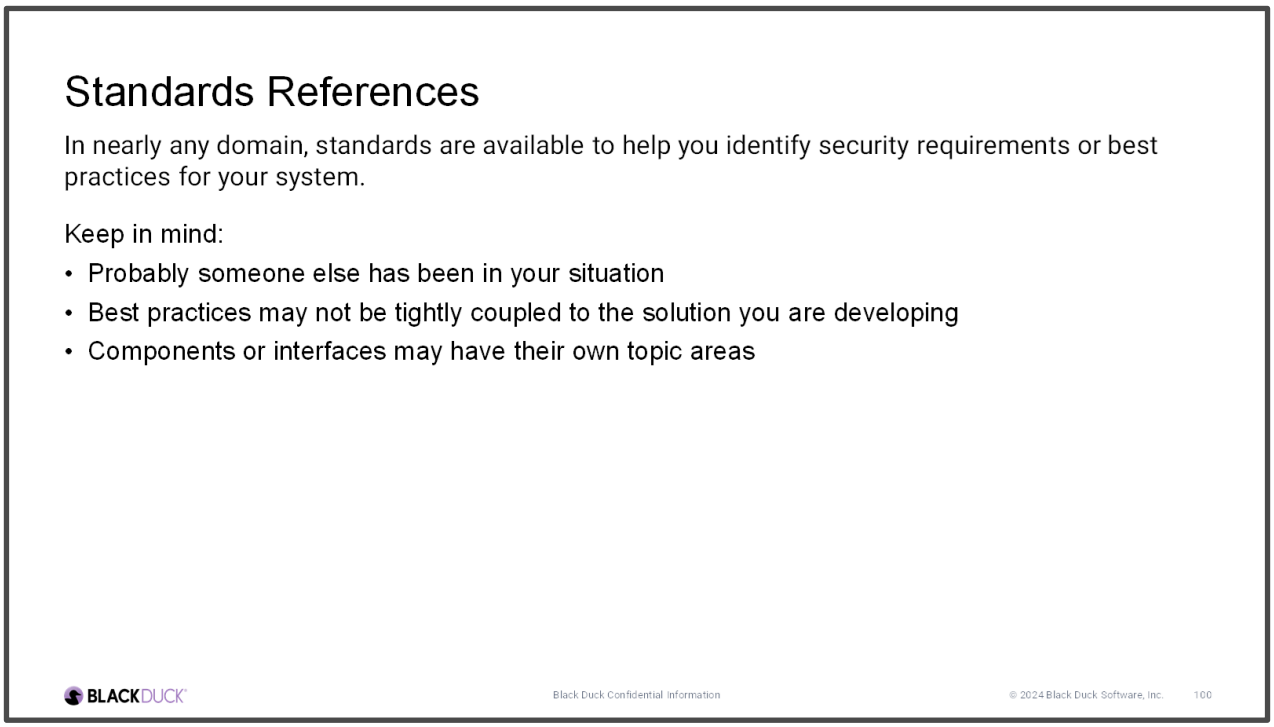
An important part of product development is to note that any off-the-shelf components, middleware, or other functionality introduced into a system must be fully evaluated for security flaws and these must be addressed at deployment. A product is only as secure as its supply chain.

Always consider the ongoing operations and maintenance phases of a product lifecycle when designing your security architecture.



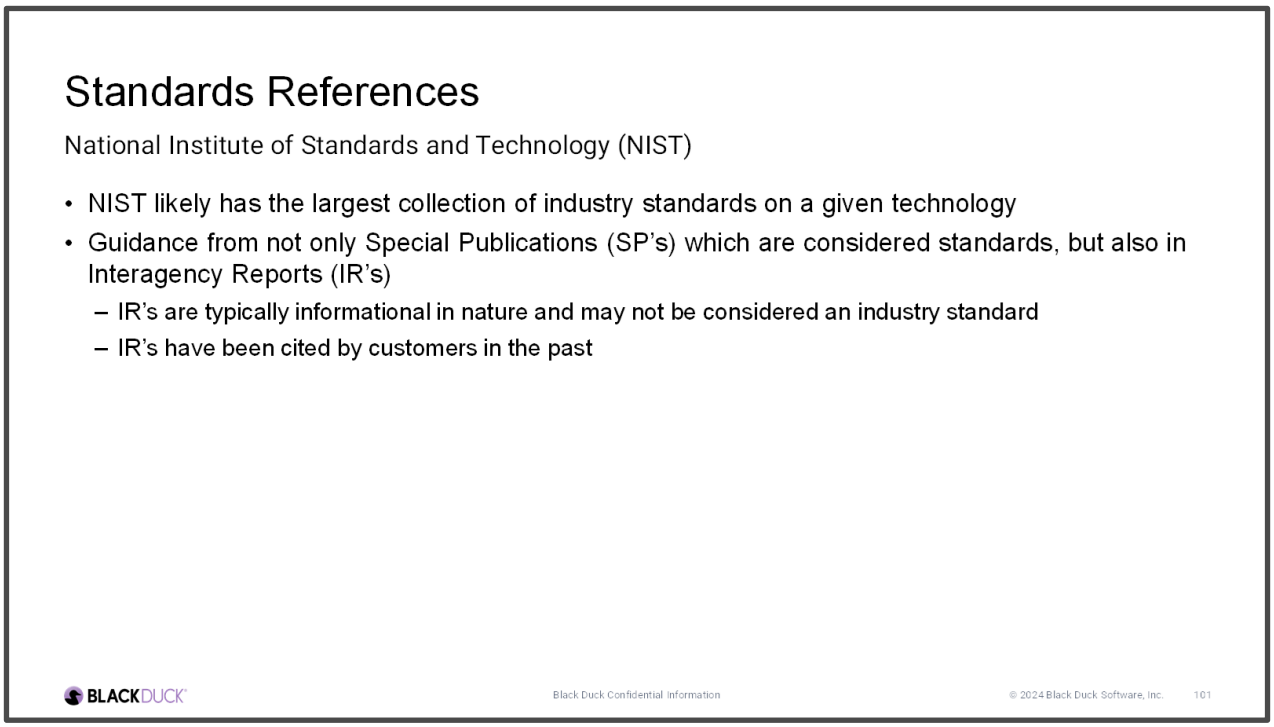
Slide 99

Black Duck (formerly part of Synopsys, and Cigital before that) has a long history producing expert-based threat modeling tools. This slide is about Cortex, the knowledge-management system at the core of the current approach. It collects data from internal sources (e.g. consultant experience) as well as external sources (e.g. MITRE) and provides a flexible data model that can power, among other security activities, threat models. This increases the efficiency of the threat modeling process and ensures that lessons learned in some engagements can be shared company-wide.



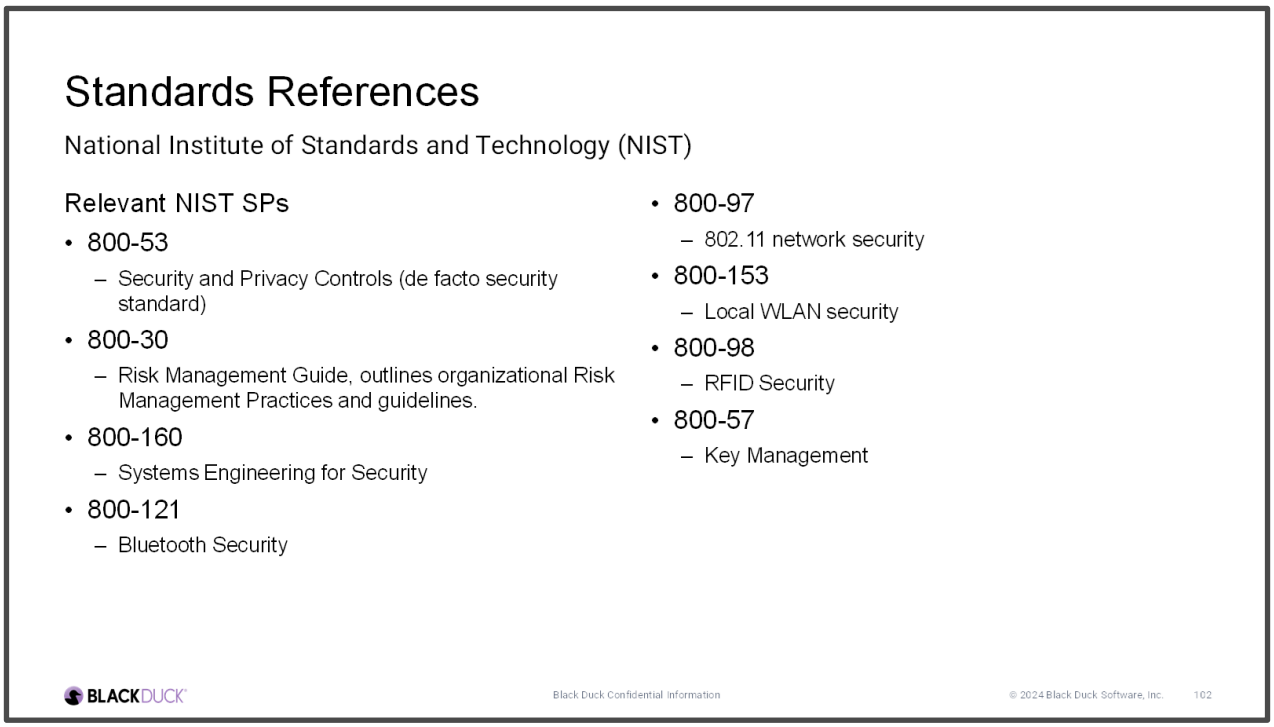
Slide 100

Understand the standards that apply to your product and are “IAW” for your requirements.



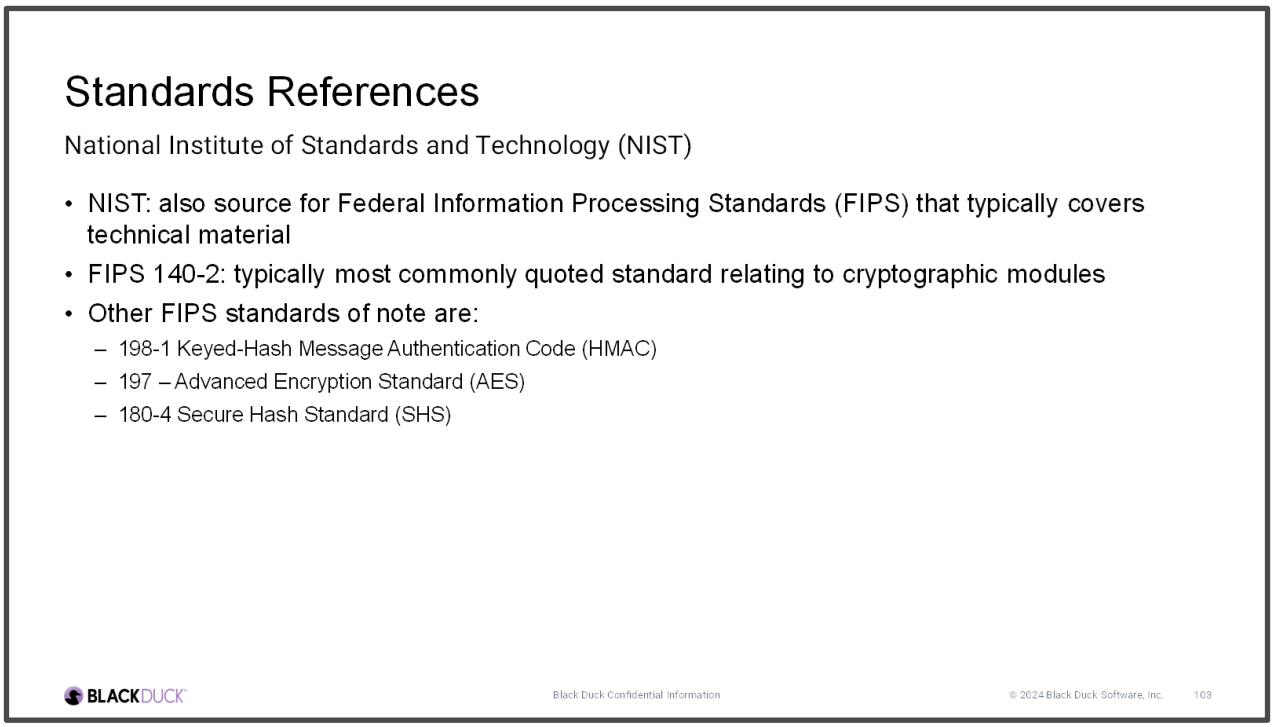
Slide 101

NIST is the gold standard and baseline for all cryptography and security in the US and Canada.



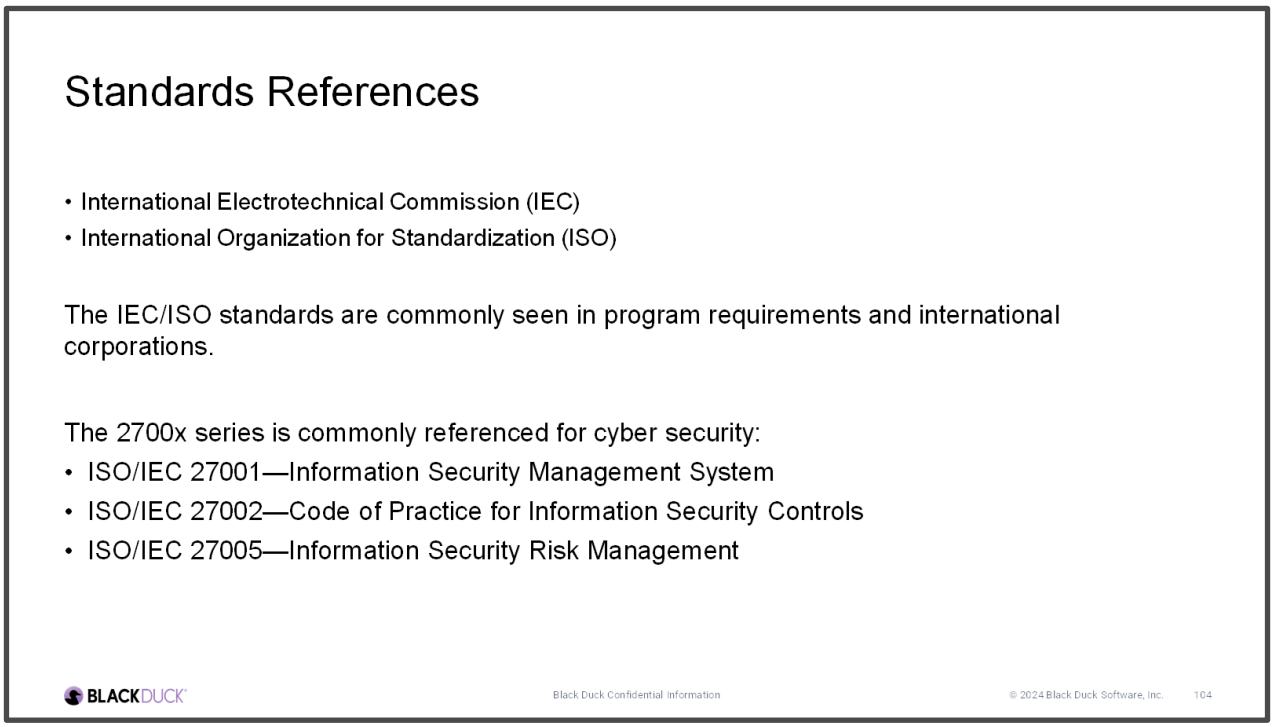
Slide 102

NIST publishes hundreds of documents related to cyber security in the SP 800 series. Which of these apply to your product lines?



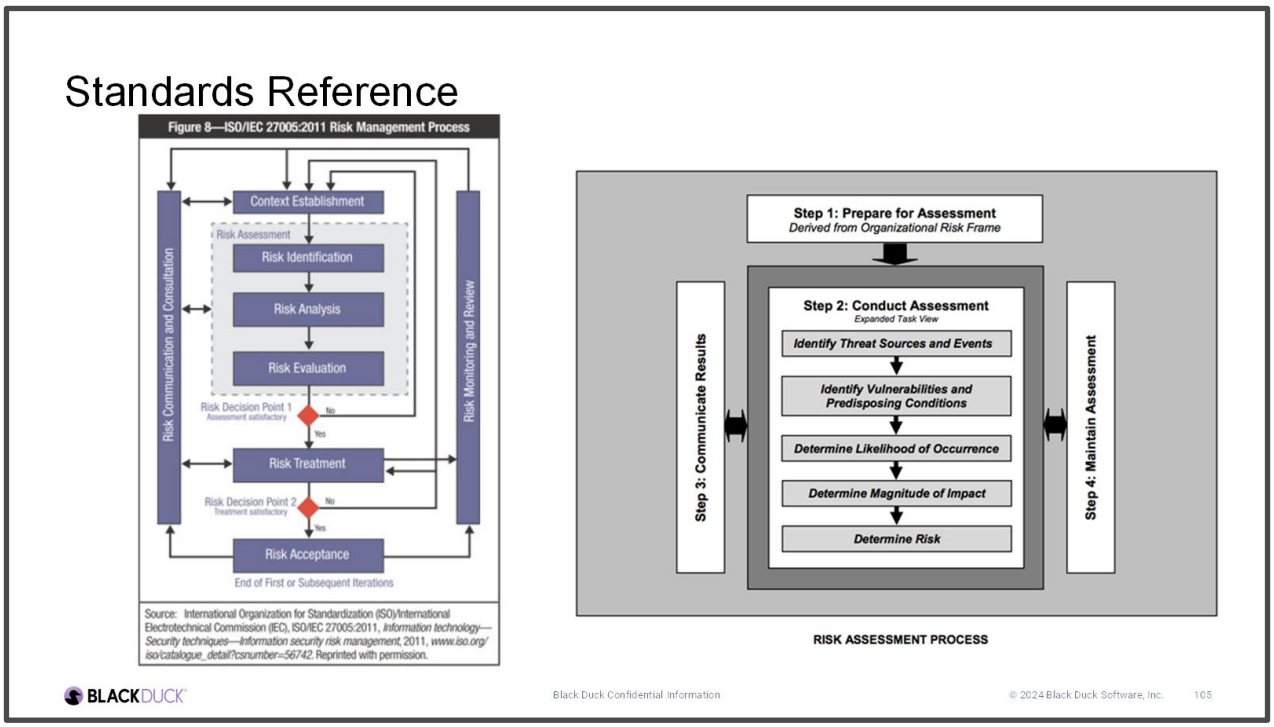
Slide 103

Any designers working crypto implementations, even if they are using third party libraries, should review these standards prior to starting a project.



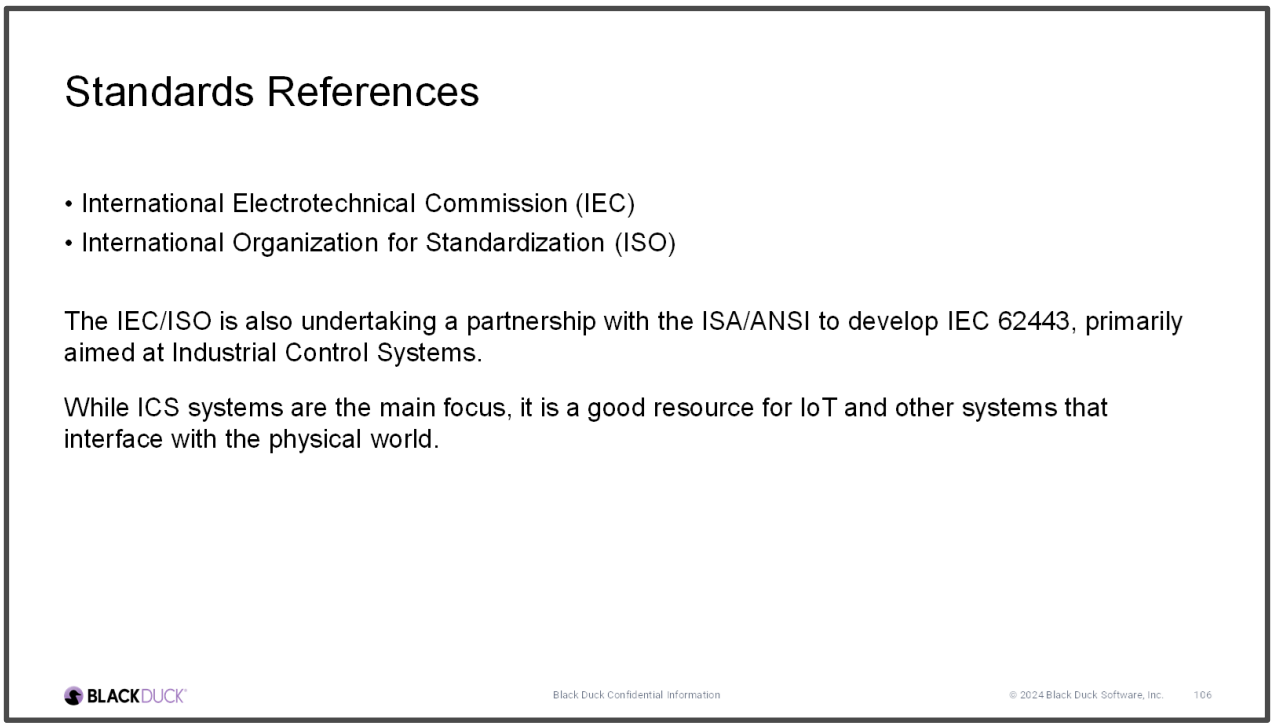
Slide 104

These form a baseline for many other process standards in security.



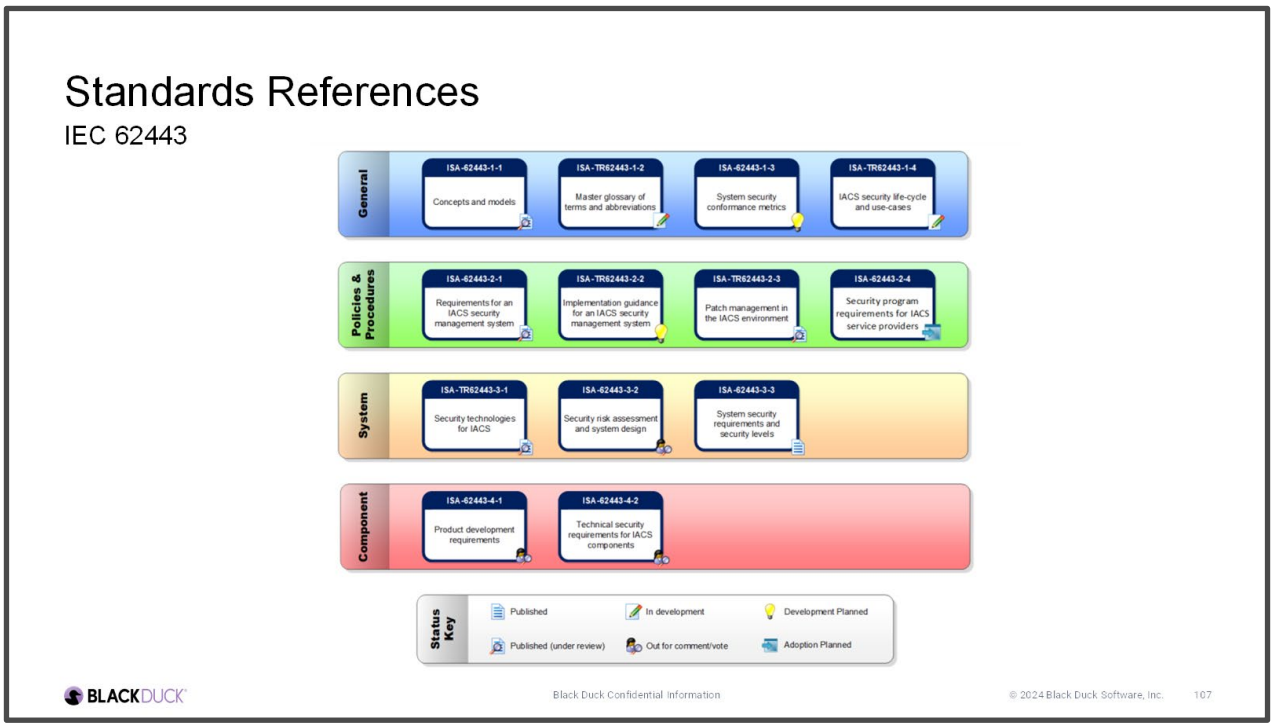
Slide 105

Notice that the ISO and NIST differ in language, but address the same process flow and issues.



Slide 106

The 62443 is another “baseline” document that feeds other industry specific standards.



Slide 107

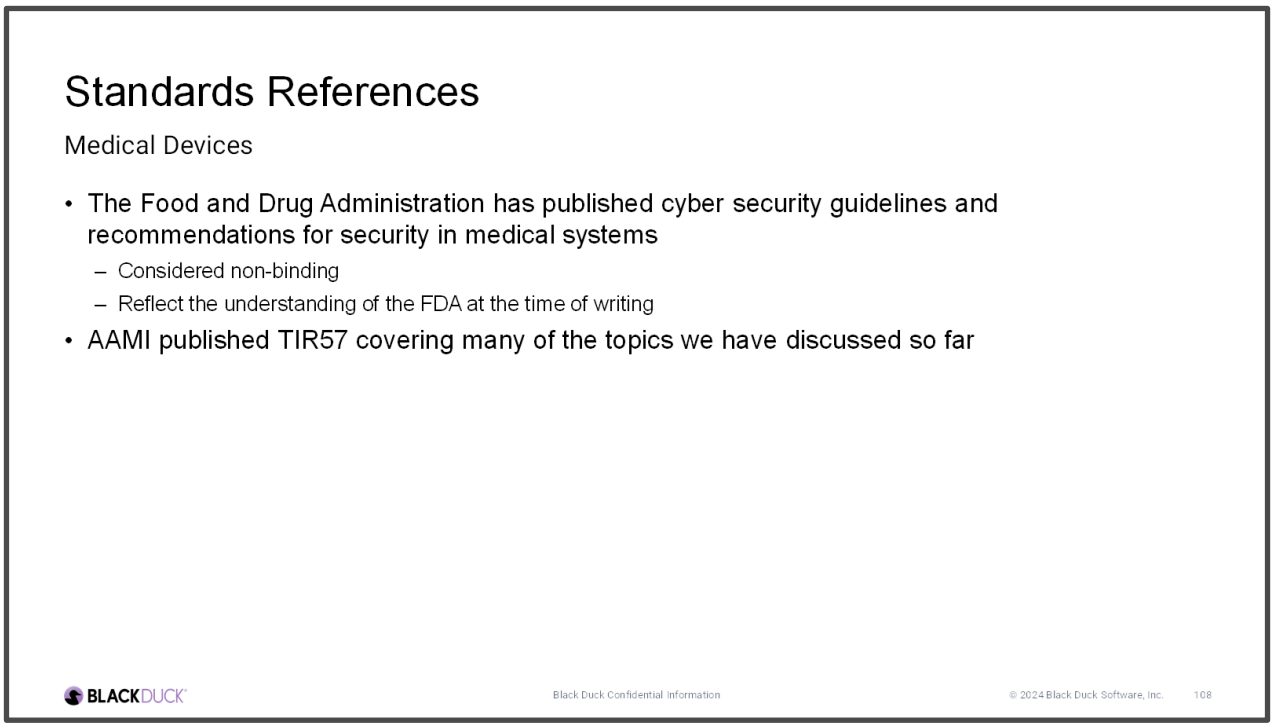
Module content from the slide:

General Information
IEC 62443-1-1 Concepts and Models
IEC 62443-1-2 Master Glossary
IEC 62443-1-3 Security Compliance Metrics
IEC 62443-1-4 Lifecycle and Use Cases

Program Definition
IEC 62443-2-1 Security Management System
IEC 62443-2-2 Implementation Guidance
IEC 62443-2-3 Patch Management
IEC 62443-2-4 Requirements for Solution Suppliers (*)

System Security
IEC 62443-3-1 Security Technologies
IEC 62443-3-2 Risk Assessment and System Design
IEC 62443-3-3 System Requirements and Security Levels

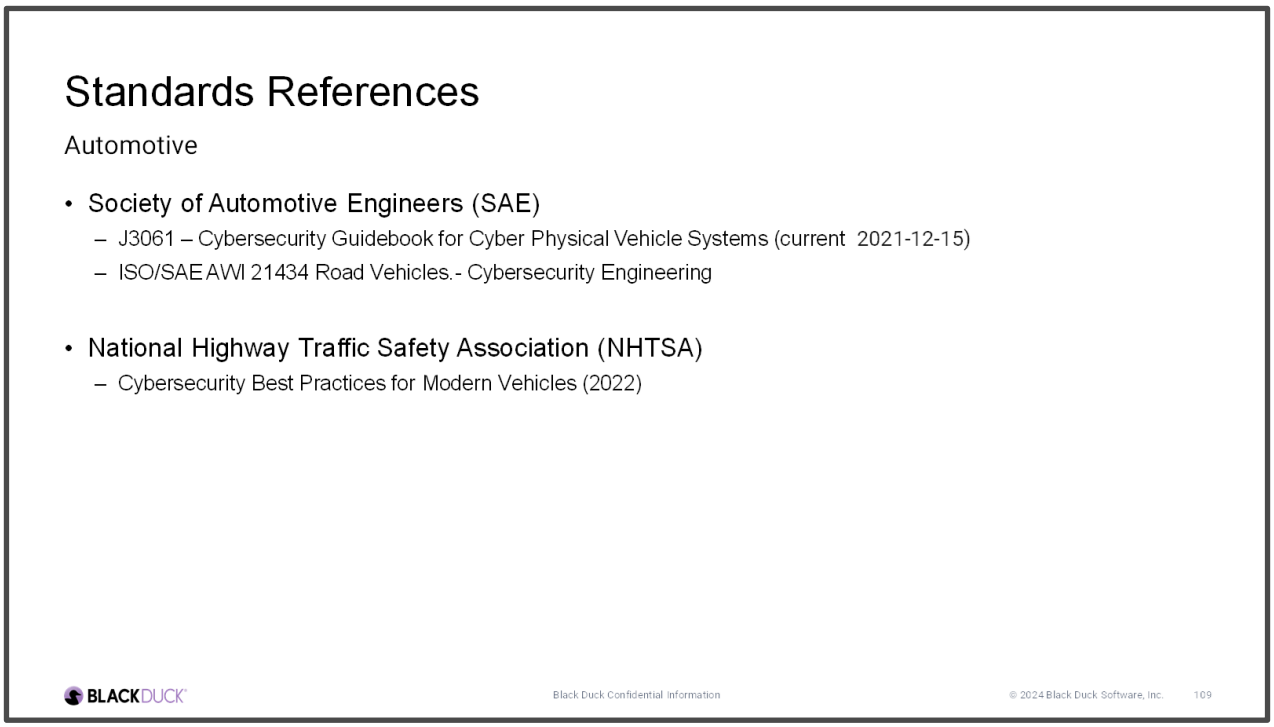
Component Security
IEC 62443-4-1 Product Development Requirements (*)
IEC 62443-4-2 Technical Requirements for Components



Slide 108

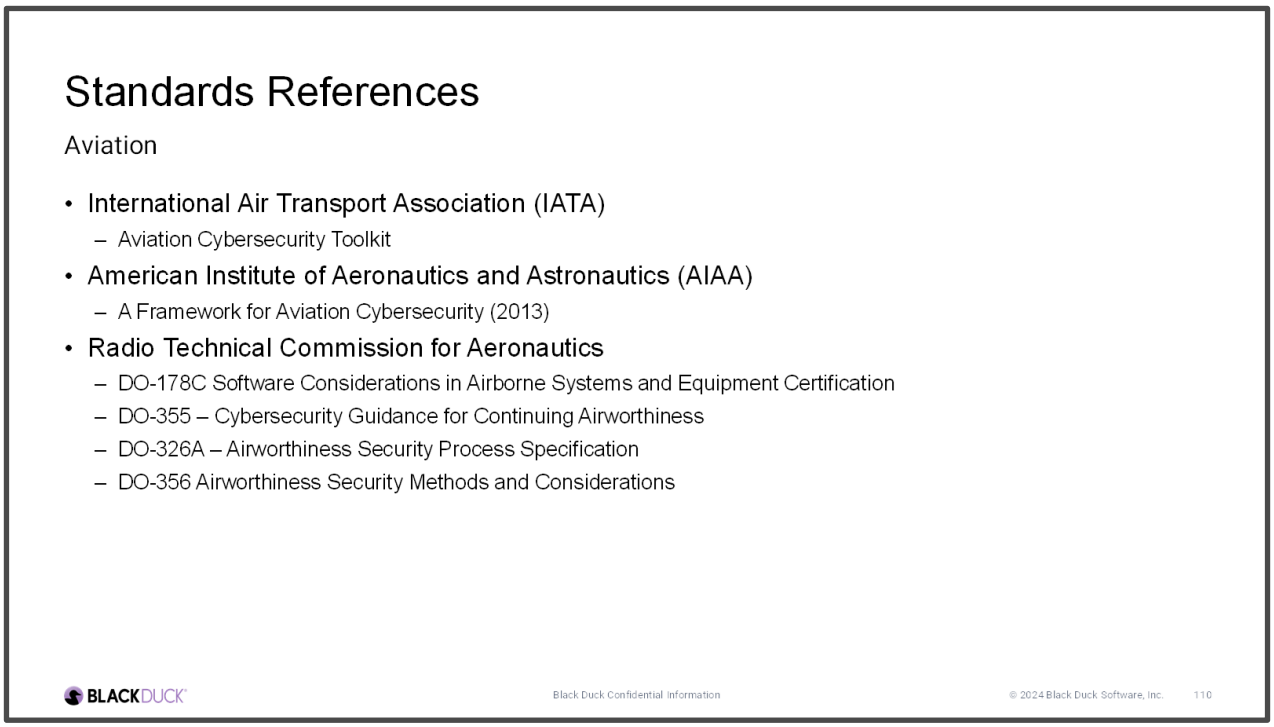
The standard references for medical devices include:

- FDA – Postmarket Management of Cyber Security in Medical Devices, TIR 57
- Medical Device Innovation, Safety and Security Consortium (MDISS)
- Association for the Advancement of Medical Instrumentation (AAMI)



Slide 109

These are in continuous development.



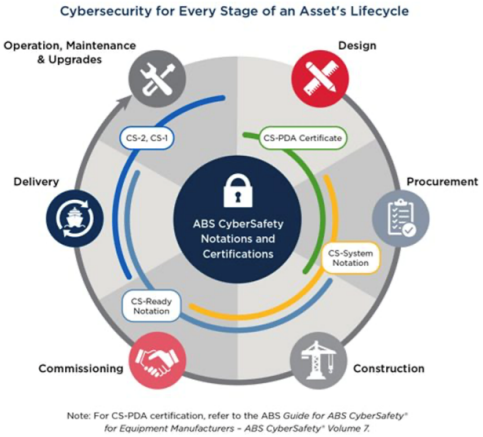
Slide 110

These are in continuous development.

Standards References

Maritime

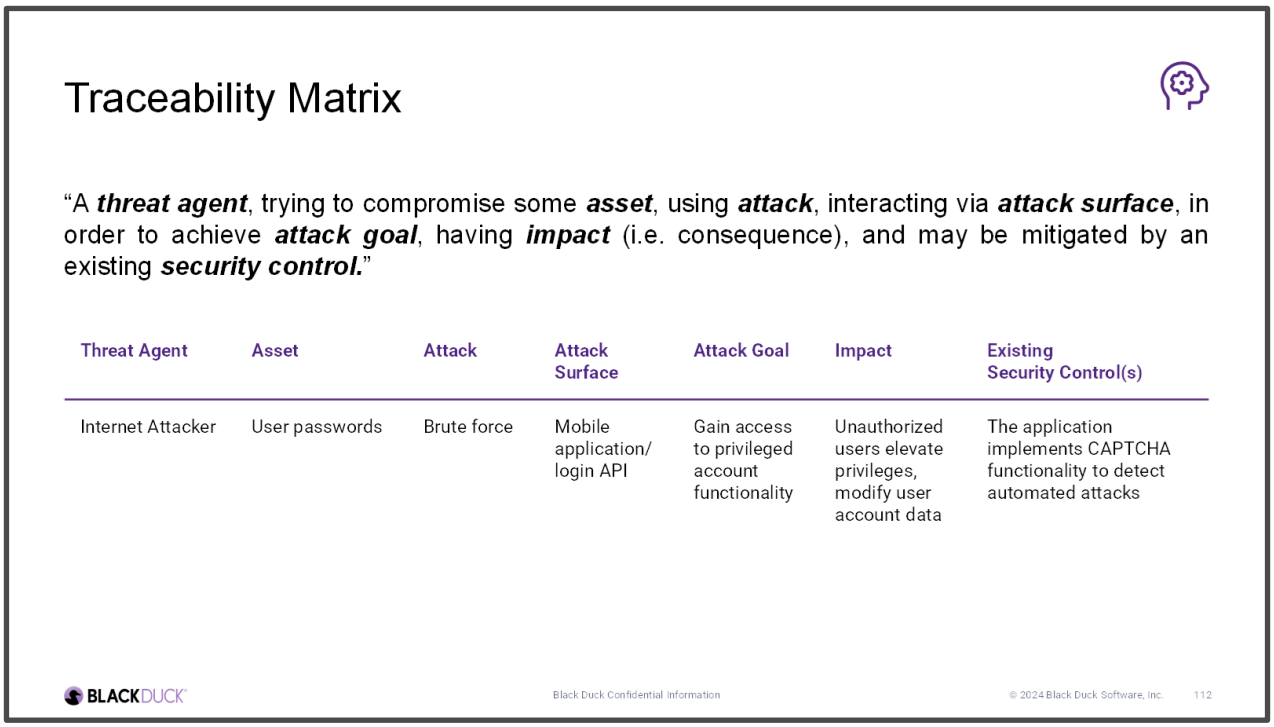
- American Bureau of Shipping (ABS)
  - Released cybersecurity guidance in 2016 with the ABS CyberSafety program
  - Issued two documents as part of the series adapting cybersecurity principles to Marine and Offshore operations
- Offshore providers have also successfully implemented IEC 62443



BLACKDUCK
Black Duck Confidential Information
© 2024 Black Duck Software, Inc.

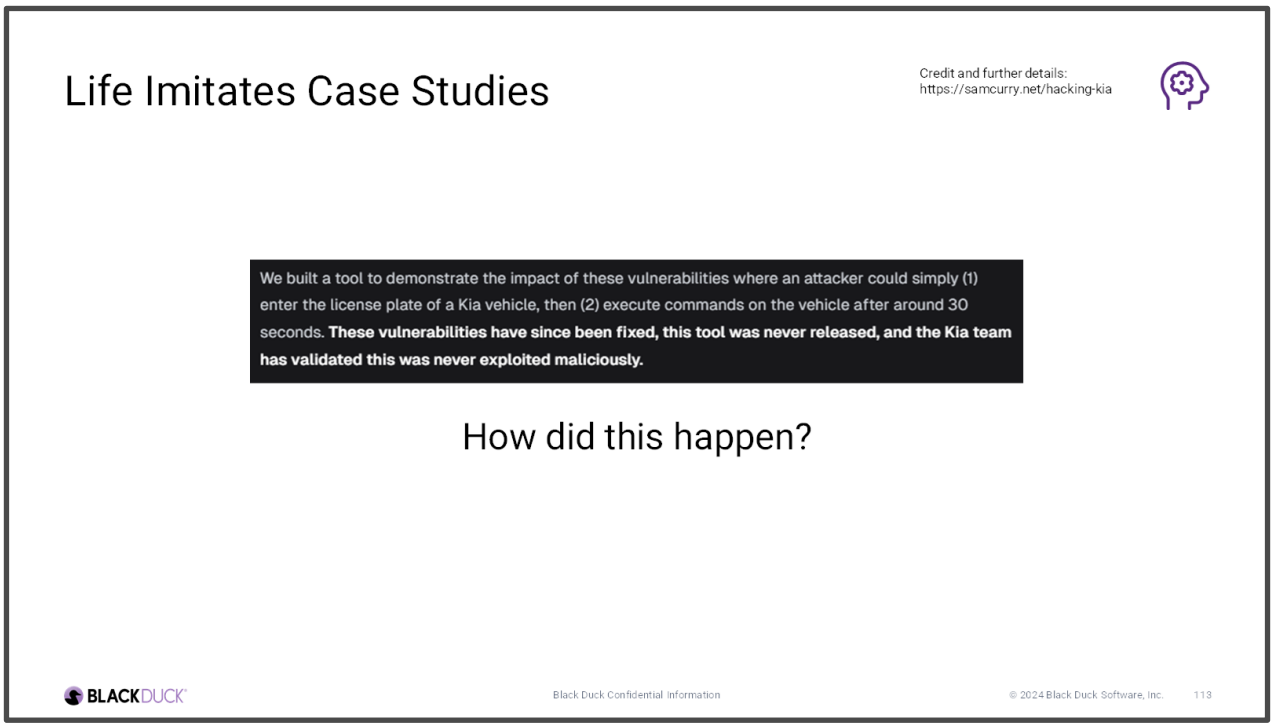
Slide 111

Port security has many layers of concern, and these are just beginning to be addressed.



Slide 112

You can choose to maintain your threat analysis however you like. Tracking it as a table (shown above) is just one example of how it can be done.
Note that the controls listed in the last column are "descriptive" – we list the controls that do exist, not the controls that "should" exist.


Life Imitates Case Studies
Credit and further details:
https://samcurry.net/hacking-kia
We built a tool to demonstrate the impact of these vulnerabilities where an attacker could simply (1) enter the license plate of a Kia vehicle, then (2) execute commands on the vehicle after around 30 seconds. These vulnerabilities have since been fixed, this tool was never released, and the Kia team has validated this was never exploited maliciously.
How did this happen?
BLACKDUCK
Black Duck Confidential Information
© 2024 Black Duck Software, Inc.
113

Slide 113

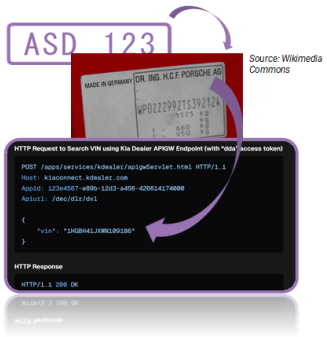
Life Imitates Case Studies

Credit and further details:
https://samcurry.net/hacking-kia

Please note: this is a simplified view. See link above for the official description of the exploit.

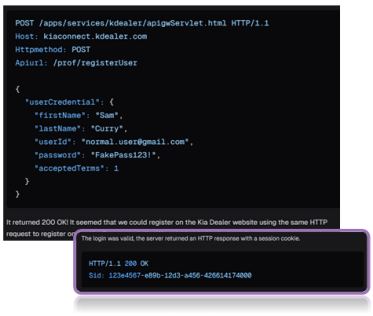
1

Get VIN from license plate, and user email from VIN



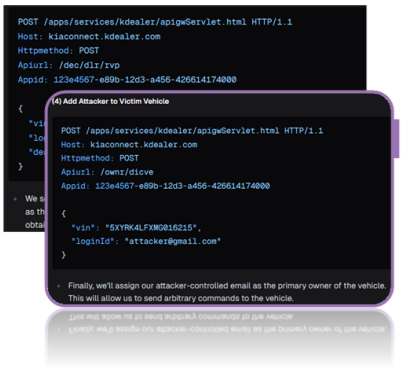
2

Register as dealer



3

Demote owner and take his place!



BLACKDUCK

Black Duck Confidential Information

© 2024 Black Duck Software, Inc. 114

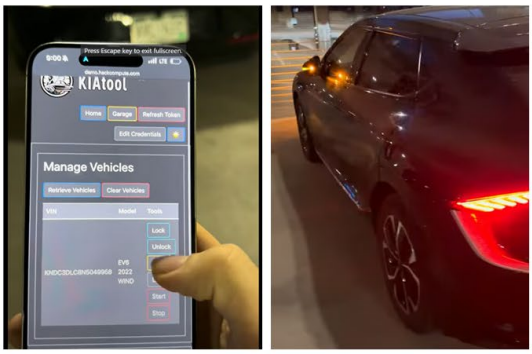
Slide 114

Life Imitates Case Studies

Credit and further details:
https://samcurry.net/hacking-kia

4

Enjoy your "new" car!



BLACKDUCK

Black Duck Confidential Information

© 2024 Black Duck Software, Inc. 115

Slide 115

Life Imitates Case Studies

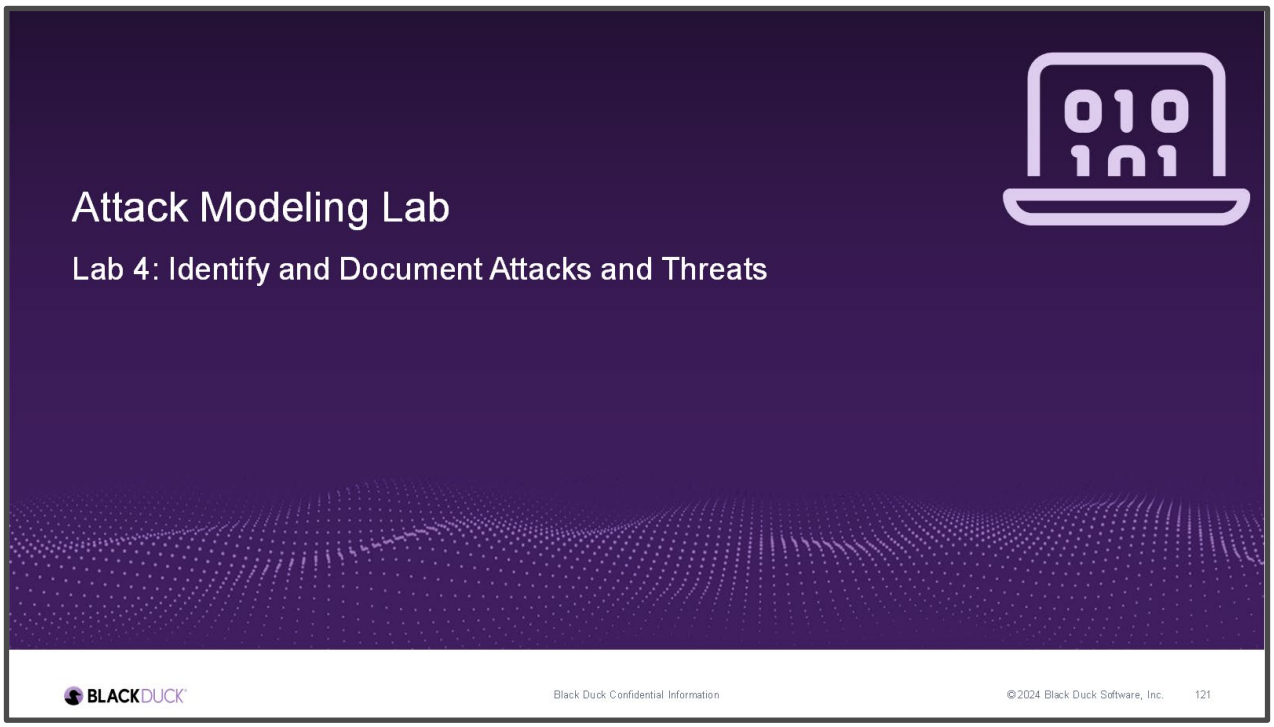
A traceability matrix for this incident

Note: these entries could belong to separate threat models!

Threat Agent	Asset	Attack	Attack Surface	Attack Goal	Impact	Existing Security Control(s)
Internet Attacker	Account creation	Create a dealer's account	Account creation API endpoint	Act as a dealer	Depends on who access given to account	API endpoint is unknown
Dealer	Vehicle ownership, User API key	Replace user assigned to vehicle	Back end API	Disrupt access for vehicle owner	Owner loses access over his vehicle to an attacker	(Perhaps) Logging and monitoring
Dealer	Vehicle command	Trigger vehicle commands	Vehicle API	Trigger actions on vehicle (e.g. open door, start engine)	Facilitate vehicle theft	User API key

BLACKDUCK Black Duck Confidential Information © 2024 Black Duck Software, Inc. 116

Slide 116


Attack Modeling Lab
Lab 4: Identify and Document Attacks and Threats
BLACKDUCK
Black Duck Confidential Information
©2024 Black Duck Software, Inc.
121

Slide 121

Lab Instructions

Identify Threats



Identify potential threats using the provided templates:

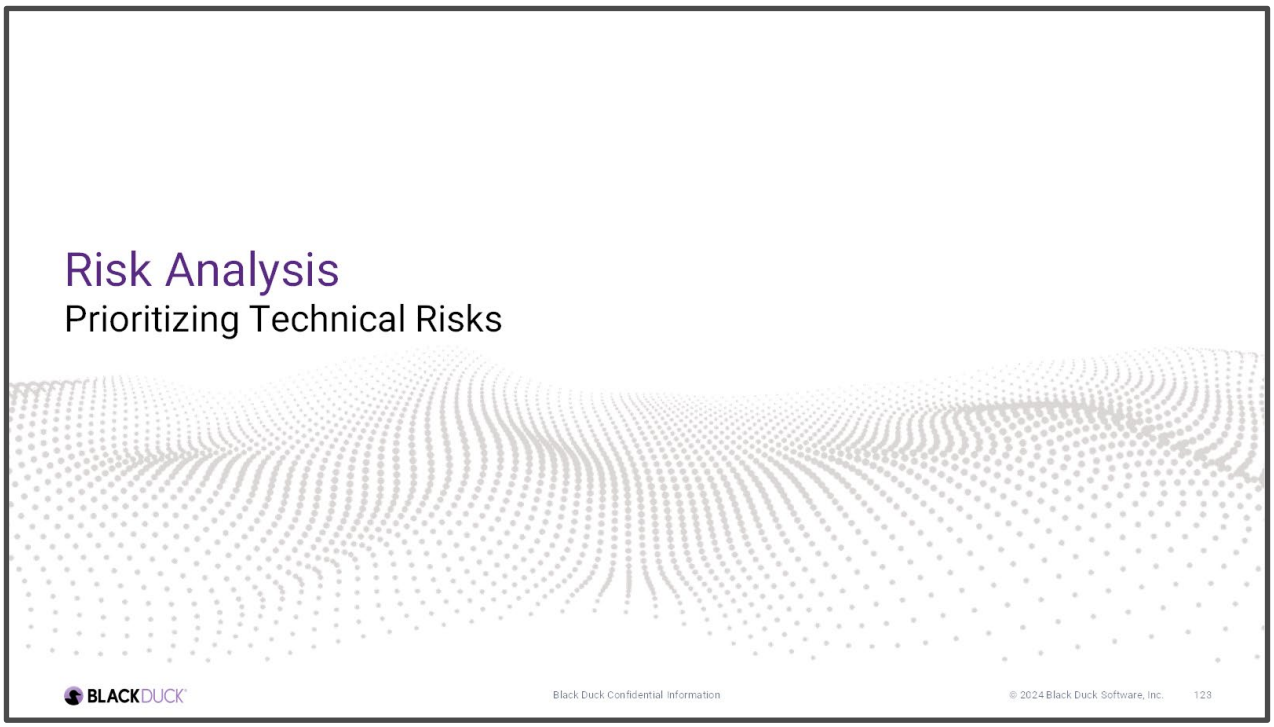
- Identify a potential attack
- Identify consequence if the potential attack were to succeed

Optional:

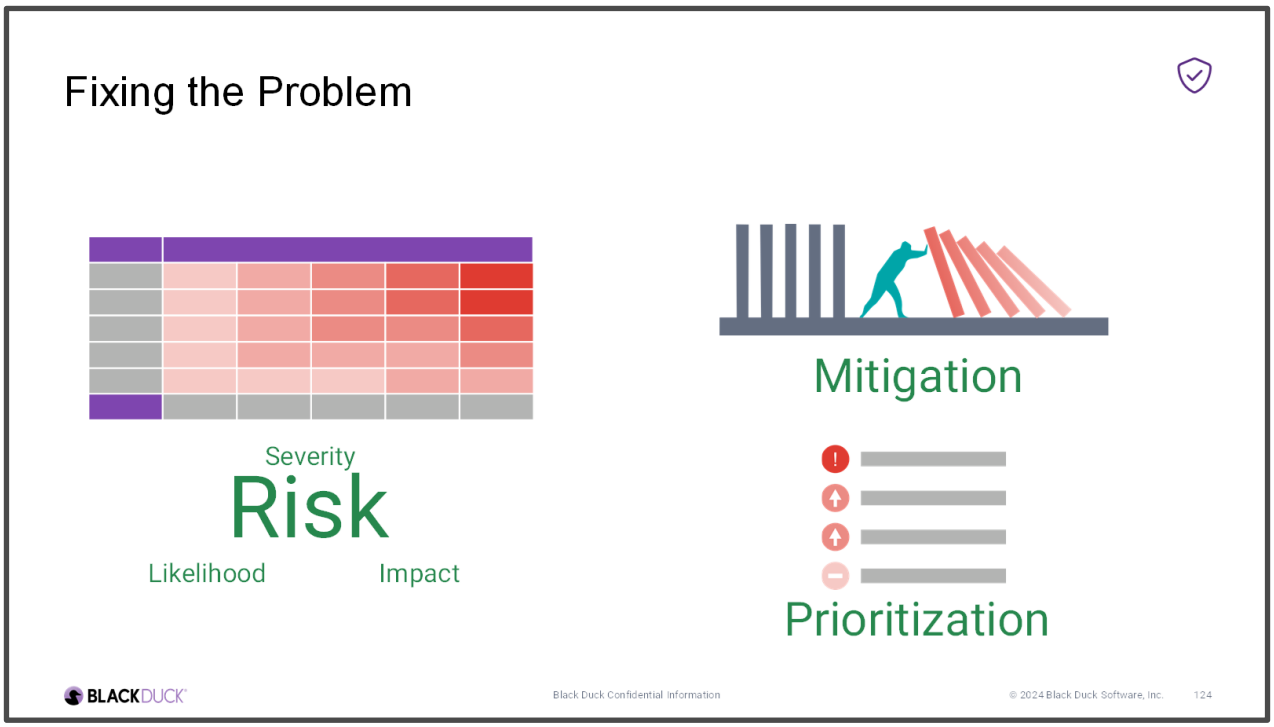
- Identify action items that may help confirm that the vulnerability exists
  - This could be specific instructions to a penetration tester or code reviewer

BLACKDUCK Black Duck Confidential Information ©2024 Black Duck Software, Inc. 122

Slide 122


Risk Analysis
Prioritizing Technical Risks
BLACKDUCK
Black Duck Confidential Information
© 2024 Black Duck Software, Inc.
123

Slide 123


Fixing the Problem
Severity
Risk
Likelihood
Impact
Mitigation
Prioritization
BLACKDUCK
Black Duck Confidential Information
© 2024 Black Duck Software, Inc.
124

Slide 124

Likelihood

Influencing Factors



Access
Can anyone launch the attack?
What are the constraints?



Credit:
trebound.com

Attack sophistication
Complex attacks are less likely to be executed



Attacker motivation
Attractive targets will be more likely to be attacked



Controls
One key goal of controls is to make attacks less likely



Credit:
DLIMedia on DeviantArt

Precedent
Instances of a successful attack may open the door for similar incidents in the future

Slide 125

Likelihood

Example: Determining Likelihood



What is the **likelihood** of a successful brute force attempt if the following exists:

- An online banking application is available over the internet, but you must have valid credentials to access any account or user information.
- Each user requires a unique authentication credential and is restricted to accounts they are owner or co-owner.
- After a series (5) of failed attempts, access to the account is locked and requires some manual re-activation.
- The backend logs, monitors and alerts on all failed authentication attempts.

BLACKDUCK Black Duck Confidential Information © 2024 Black Duck Software, Inc. 126

Slide 126

Let's consider the scenario described on the slide. What influencing factors exist that increase or decrease the likelihood of successful compromise? Does anyone have any thoughts on what the likelihood should be and why?

Likelihood

Example: Determining Likelihood

▲ **Access** to the application is through the internet, where it is exposed to all sorts of malicious actors.

▲ **Sophistication** level of a brute-force attack is very low.

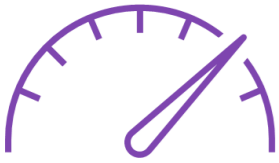
▲ Banking applications present a high-value target and likely increases an attacker's **motivation**, making them a prime target for attack.

▼ An attacker is **limited to 5 failed attempts** and the system actively **monitors suspicious activity**.

▲ There is extensive **precedent** of targeted brute-forcing and dictionary attacks to public-facing applications, which increases the likelihood that it happens again.

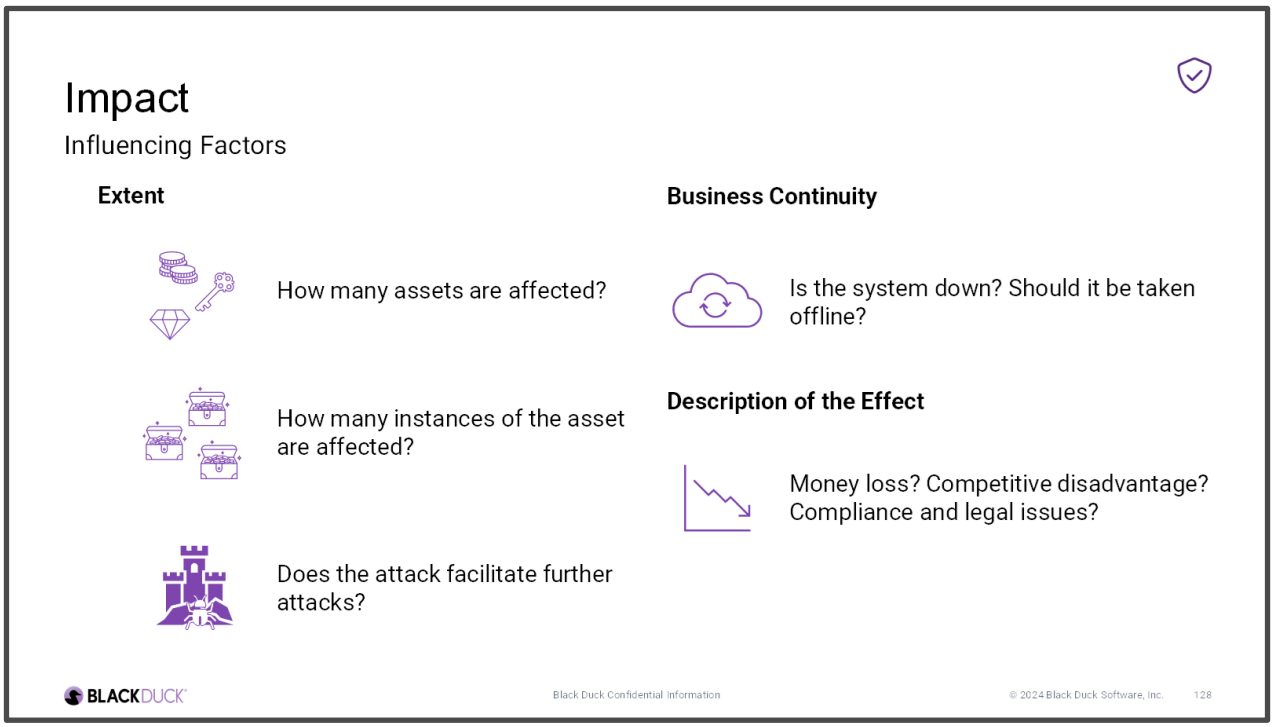
The sheer number of adversaries increases the likelihood.

Access to tools and dictionaries of compromised passwords is relatively easy to obtain.



BLACKDUCK Black Duck Confidential Information © 2024 Black Duck Software, Inc. 127

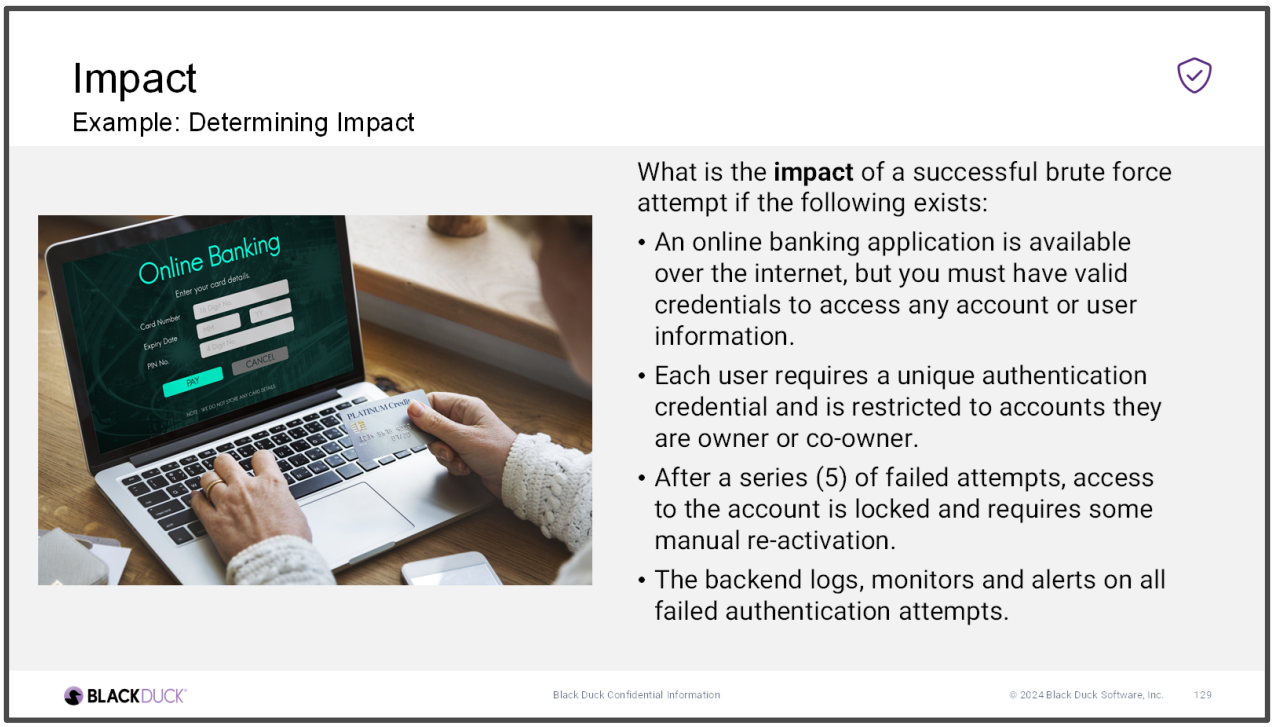
Slide 127



Slide 128

Impact can be determined by evaluating the number of assets compromised, how they were compromised and how many users were affected. Oftentimes, there are specific industry factors that much be considered as well. Any one of the factors on their own could result in a critical impact if the business goals reflect such a scenario. Be consistent.

- **Extent/Number of Assets:** How many assets are compromised? Are any of them particularly sensitive (e.g., private information, safety functionality, intellectual property)? Is a recall in order?
- **Extent/Sort of Compromise:** How are the assets affected? Was secret information leaked? Were important records tampered with?
- **Extent/Scale:** How much data is affected? Is a single person affected, or was it the entire customer base? Does the attack affect the security of many devices on the field?
- **Availability:** Is the system unavailable to users? For how long? Does it affect a large number of users or different geographic areas?
- **Industry-Specific Factors:** Depending on the industry, additional factors may need to be considered. In most systems, safety is not a real concern, but it becomes critical in systems with physical impact, such as medical devices or connected cars.



Slide 129

Let's consider the scenario described on the slide. What influencing factors exist that increase or decrease the impact of successful compromise? Does anyone have any thoughts on what the impact should be and why?

Impact

Example: Determining Impact

▼ A successful attack reveals the credentials of a single user. Consequently, the scale of the attack is limited.

▲ The information stolen from the banking application is likely to contain PII and financial information, valuable information even in small volumes.

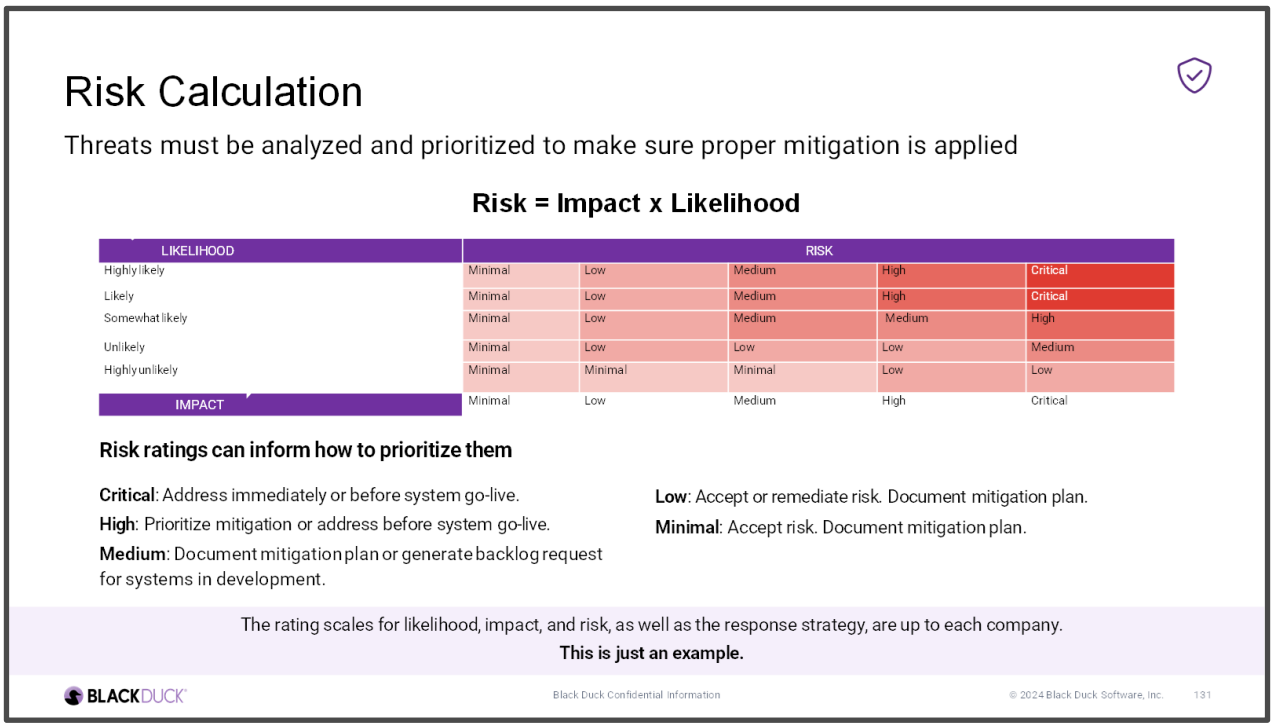
▲ A successful attack would grant the attacker full read and write access to the compromised account.

▲ As a result of the existing account lockout control, the account could become temporarily unavailable to the account holder.



BLACKDUCK Black Duck Confidential Information © 2024 Black Duck Software, Inc. 130

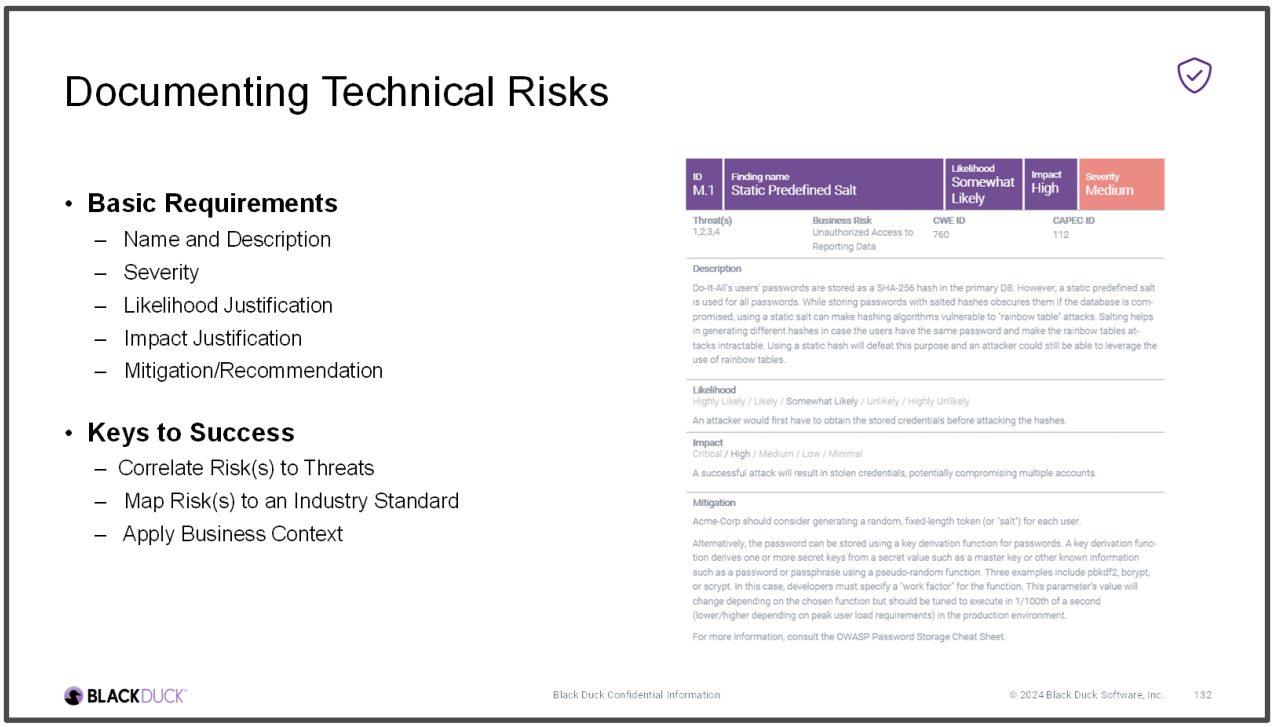
Slide 130



Slide 131

Analyzing and prioritizing risks is crucial for being able to apply appropriate mitigations. Without prioritizing risks, mitigations might be applied arbitrarily, and money might be spent on mitigating low risks while leaving high risks unmitigated.

The level of risk can be calculated using the NIST 800-30 standard. This standard determines the risk based on the likelihood that an attacker would be able to exploit the vulnerability as well as the impact that a successful exploit would have on the business. In general, the lower of the two values is used to determine the severity, but impact weighs a little higher than likelihood. The original standard used only three levels (Low, Medium, and High). The revised version from 2012 uses five levels (Very Low, Low, Moderate, High, and Very High) for impact and likelihood, which can be appropriate if you need a more detailed prioritization.

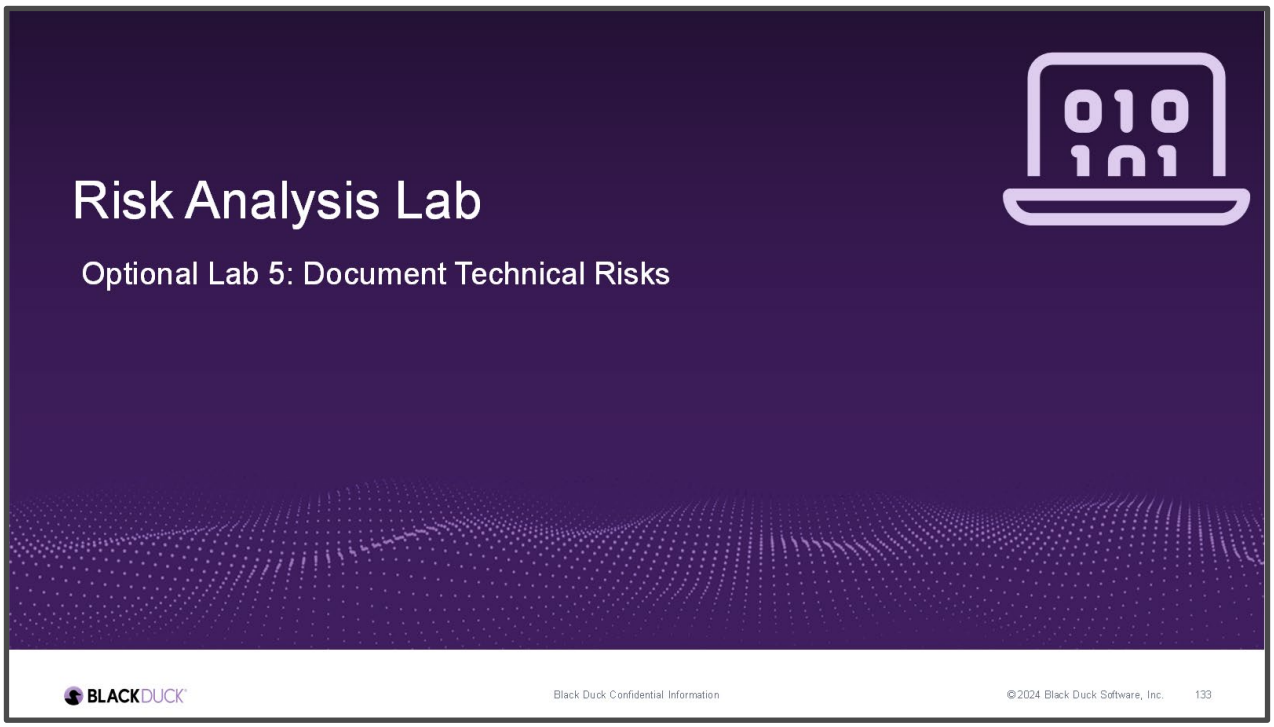


Slide 132

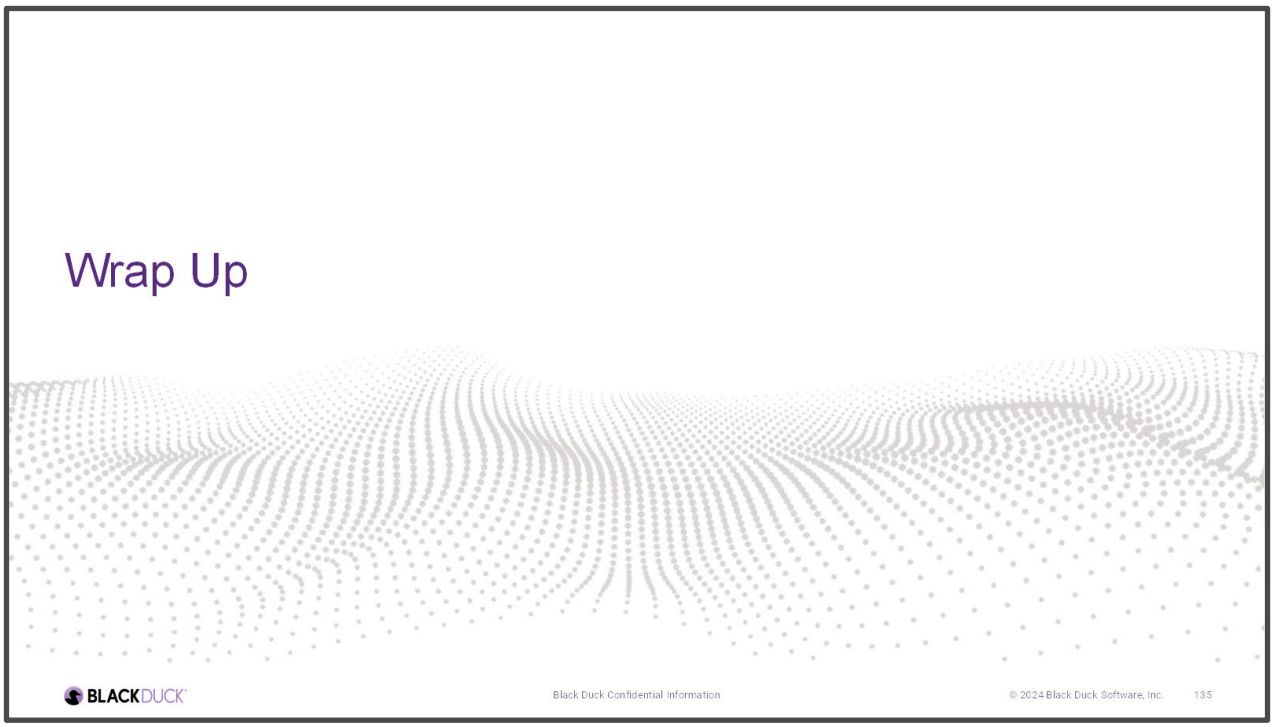
Technical risks can be documented in many ways. Often, the most important deliverable to high-level or nontechnical stakeholders. To properly communicate risk, we must talk through what was identified, why it is a problem and how to fix it. In many cases we need to prioritize our risks and justify our final ratings.

To ensure the data is consumable, show completeness, and showcase an ability to pass an audit, it is also great to map risks (and threats) to associated industry standards and best practices.

Finally, each threat in our traceability matrix does not need to be a technical finding. Oftentimes, some threats are already mitigated with existing controls and need not be reported. Other times, many threats can be fixed with the same solution. Make sure to show the consumer the threats that were evaluated to generate each of the given risks.

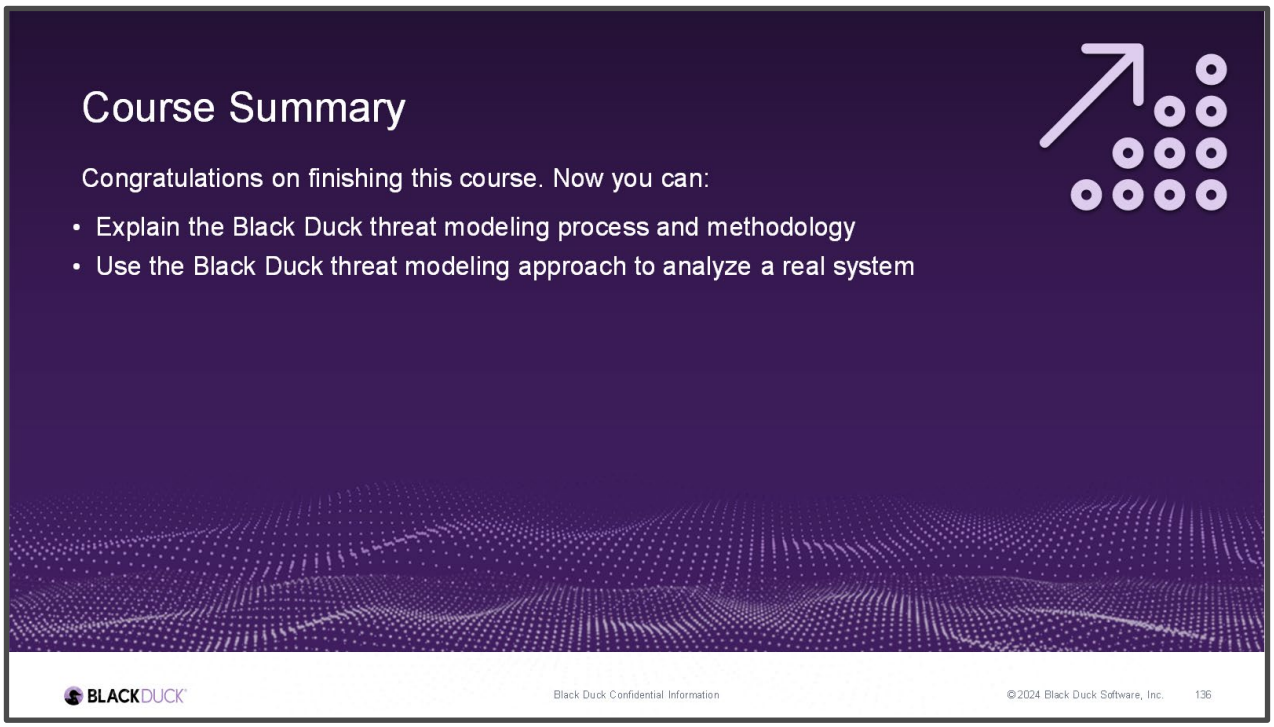

Risk Analysis Lab
Optional Lab 5: Document Technical Risks
BLACKDUCK
Black Duck Confidential Information
©2024 Black Duck Software, Inc.
133

Slide 133

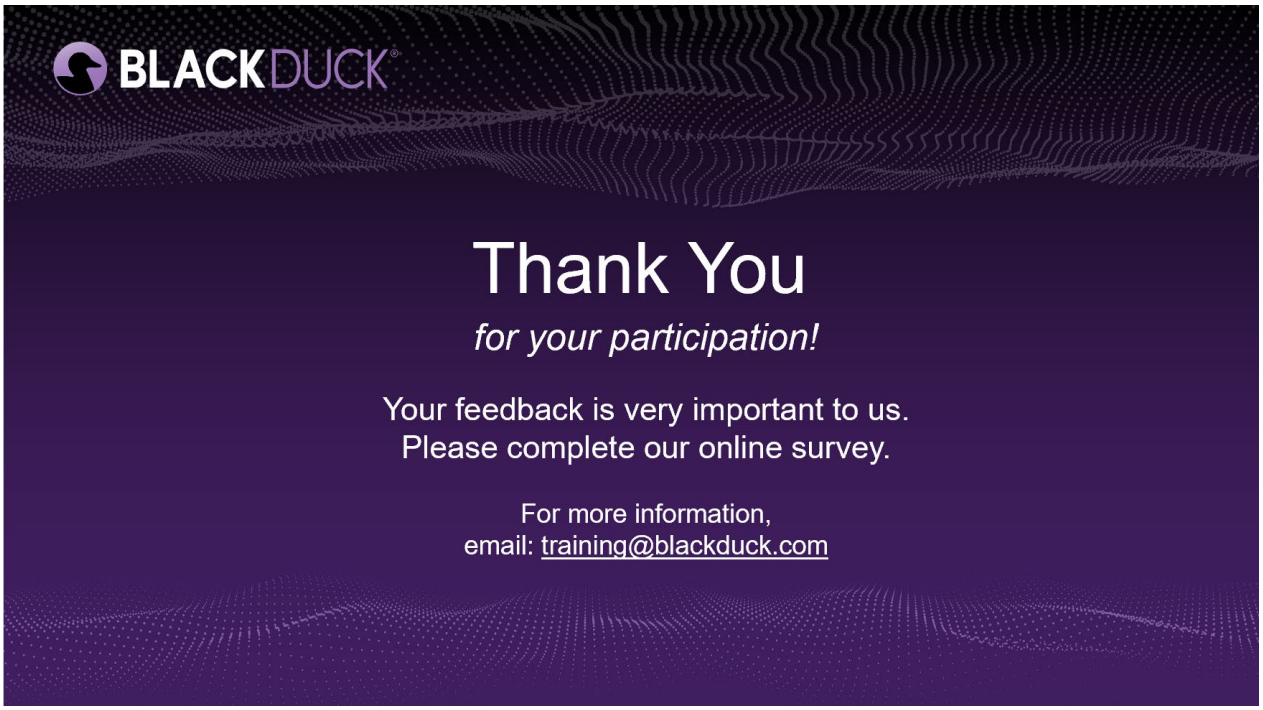


Slide 135

Let’s recap what you learned today…


Course Summary
Congratulations on finishing this course. Now you can:
• Explain the Black Duck threat modeling process and methodology
• Use the Black Duck threat modeling approach to analyze a real system
BLACKDUCK
Black Duck Confidential Information
©2024 Black Duck Software, Inc.
136

Slide 136


BLACKDUCK
Thank You
for your participation!
Your feedback is very important to us.
Please complete our online survey.
For more information,
email: training@blackduck.com

Slide 137

Acronyms

The following table provides a list of acronyms used in the course material:

ACRONYM	DEFINITION	ACRONYM	DEFINITION
3DES	Triple DES	**CORS**	Cross-Origin Resource Sharing
ACK	Acknowledge	**COTS**	Commercial-Off-The-Shelf
ACL	Access Control List	**CPU**	Central processing Unit
ASLR	Address Space Layout Randomization	**CRLF**	Carriage Return and Line Feed
AEH	Automated Ethical Hack	**CRUD**	Create, Read, Update, Delete
AES	Advanced Encryption Standard	**CSPRNG**	Cryptographically Secure Pseudo-random Number Generator
AFL	American Fuzzy Loop	**CSRF**	Cross-Site Request Forgery
AJAX	Asynchronous JavaScript and XML	**CSS**	Cascading Style Sheet
AMF	Action Message Format	**CSV**	Comma-Separated Values
ANSI	American National Standards Institute	**CVE**	Common Vulnerabilities and Exposures
AOP	Aspect-Oriented Programming	**CWE**	Common Weakness Enumeration
API	Application Programming Interface	**DAST**	Dynamic Analysis Security Testing
APNS	Apple Push Notification Services	**DB**	Database
AppSec	Application Security	**DBMS**	Database Management system
ARA	Architecture Risk Analysis	**DDoS**	Distributed Denial of Service
ARP	Address Resolution Protocol	**DEP**	Data Execution Prevention
ARC	Automatic Reference Counting	**DES**	Data Encryption Standard
ASCII	American Standard Code for Information Interchange	**DevOps**	Development and Operations
ASLR	Address Space Layout Randomization	**DevSecOps**	Development, Security, and Operations
ASM	Assembly Language	**DHCP**	Dynamic Host Configuration Protocol
AuthN	Authentication	**DLL**	Dynamic-Link Library
AuthZ	Authorization	**DMZ**	Demilitarized Zone
A/V	Audio-Visual	**DNS**	Domain Name System
CA	Certificate Authority	**DOM**	Document Object Model
CAC	Cigital Assessment Center	**DoS**	Denial of Service
CALMS	Culture, Automation, Learning, Measurement, and Sharing	**DPAPI**	Data Protection API
CAPEC	Common Attack Pattern Enumeration and Classification	**DRM**	Digital Rights Management
CAT5	Category 5	**DSA**	Digital Signature Algorithm
CBC	Cipher-Block Chaining	**DSO**	Dynamic Shared Object
CDMA	Code Division Multiple Access	**DTD**	Document Type Definition
CDN	Content Delivery Network	**ECU**	Engine Control Unit
CGI	Common Gateway Interface	**EE**	(Java) Enterprise Edition
CI/CD	Continuous Integration/Continuous Delivery/Deployment	**ELF**	Executable and Linking Format
CIL	Common Intermediate Language	**EMET**	Enhanced Mitigation Experience Toolkit
CIO	Chief Information Officer	**EMI**	Electromagnetic Interference
CISO	Chief Information Security Officer	**FIPS**	Federal Information Processing Standard
CLDC	Connected Limited Device Configuration	**FOSS**	Free Open-Source Software
CLI	Common Language Infrastructure	**FTP**	File Transfer Protocol
CLR	Common Language Runtime	**GCC**	GNU Compiler Collection
COAX	Coaxial	**GOT**	Global Offset Table

© 2024 Black Duck

ACRONYM	DEFINITION	ACRONYM	DEFINITION
GPS	Global Positioning System	**JSF**	Java Server Faces
GRC	Governance, Risk Management, and Compliance	**JSON**	JavaScript Object Notation
GSM	Global System for Mobile Communications	**JSONP**	JSON with Padding
HCI	Human-Computer Interaction	**JSP**	Java Server Pages
HIPAA	Health Insurance Portability and Accountability Act of 1996	**JSR**	Java Specific Requests
HMAC	Hash-based Message Authentication Code	**JVM**	Java Virtual Machine
HR	Human Resources	**JWT**	JSON Web Token
HSM	Hardware Security Module	**KB**	Knowledge Base
HTML	Hypertext Markup Language	**KDF**	Key Derivation Function
HTTP	Hypertext Transfer Protocol	**KPI**	Key Performance Indicator
HTTPS	Hypertext Transport Protocol Secure	**L2TP**	Layer 2 Tunneling Protocol
IA-32	Intel Architecture 32-bit	**LAN**	Local Area Network
IaaS	Infrastructure as a Service	**LDAP**	Lightweight Directory Access Protocol
IaC	Infrastructure-as-Code	**LFH**	Low Fragmentation Heap
IAST	Interactive Application Security Testing	**LIFO**	Last-In-First-Out
ICMP	Internet Control Message Protocol	**LLC**	Logical Link Control
IDE	Integrated Development Environment	**LLVM**	Low-Level Virtual Machine
IdP	Identity Provider	**MAC**	Medium Access Control
IDS	Intrusion Detection System	**MAST**	Mobile Application Security Testing
IEEE	Institute of Electrical and Electronics Engineers	**MD5**	Message Digest Algorithm v5
IGRP	Interior Gateway Routing Protocol	**MDM**	Mobile Device Management
IMEI	International Mobile Station Equipment Identity	**MEH**	Manual Ethical Hack
I/O	Input/Output	**MFA**	Multi-Factor Authentication
IoC	Inversion of Control	**MIDP**	Mobile Information Device Profile
IoT	Internet of Things	**MitM**	Man-in-the-Middle Attack
IP	Internet Protocol	**MSF**	Metasploit Framework
IPC	Inter-Process Communication	**MVC**	Model-View-Controller
IPsec	Internet Protocol Security	**NATO**	North Atlantic Treaty Organization
IPv4	Internet Protocol version 4	**NetBIOS**	Network Basic Input/Output System
IPv6	Internet Protocol version 6	**NIST**	National Institute of Standards and Technology
ISO	International Organization for Standardization	**Nmap**	Network Mapper
IT	Information Technology	**NOP**	No-Operation
IV	Initialization Vector	**NoSQL**	Not Only SQL
JAR	Java ARchive	**NTLM**	NT Lan Manager
Java EE	Java Enterprise Edition	**NX**	No Execute
Java ME	Java Platform, Micro Edition	**ODBC**	Open Database Connectivity
Java SE	Java Standard Edition	**OLE DB**	Object Linking and Embedded Database
JCE	Java Cryptography Extension	**ORM**	Object Relational Mapping
JCL	Job Control Language	**OS**	Operating System
JMX	Java Management Extensions	**OSI**	Open System Interconnection
JNI	Java Native Interface	**OSPF**	Open Shortest Path First
JRE	Java Runtime Environment	**OSS**	Open Source Software

ACRONYM	DEFINITION	ACRONYM	DEFINITION
OTP	One-time Password	**SCA**	Software Composition Analysis
OWASP	Open Web Application Security Project	**SCADA**	Supervisory Control and Data Acquisition
P2P	Peer-to-Peer	**SCE**	Strict Contextual Escaping
PA	Privileged Access	**SCM**	Source Code Management
PaaS	Platform as a Service	**SCR**	Source Code Review
PASTA	Process for Attack Simulation Threat Analysis	**SD**	Secure Digital
PBE	Password-Based Encryption	**SDK**	Software Development Kit
PBKDF2	Password-Based Key Derivation Function 2	**SDL**	Security Development Lifecycle
PCI	Payment Card Industry	**SDLC**	Software Development Life Cycle
PCI DSS	Payment Card Industry Data Security Standard	**SDP**	Service Delivery Platform
PGP	Pretty Good Privacy	**SEH**	Structured Exception Handling
PIE	Position Independent Executable	**SHA-1**	Secure Hash Algorithm 1
PII	Personally Identifiable Information	**SHA-2**	Secure Hashing Algorithm 2
PIN	Personal Identification Number	**SLA**	Service-Level Agreement
PKCE	Proof Key for Code Exchange	**SMB**	Server Message Block
PKI	Public-Key Infrastructure	**SO**	Shared Object
PLT	Procedure Linkage Table	**SOA**	Service-Oriented Architecture
PnG	Persona non Grata	**SOP**	Same Origin Policy
POSIX	Portable Operating System Interface	**SpEL**	Spring Expression Language
PPTP	Point-to-Point Tunneling Protocol	**SQL**	Structured Query Language
PRNG	Pseudo-random Number Generator	**SSG**	Software Security Group
QoS	Quality-of-Service	**SSH**	Secure Shell
RAII	Resource Acquisition Is Initialization	**SSI**	Software Security Initiative
RASP	Runtime Application Self-Protection	**SSL**	Secure Sockets Layer
RCE	Remote Code Execution	**SSP**	Stack Smashing Protector
RDBMS	Relational Database Management System	**STRIDE**	Spoofing identity, tampering with data, repudiation, information disclosure, denial of service, elevation of privilege
RDP	Remote Desk Protocol	**SYN**	Synchronize
RELRO	Relocation Read-Only	**TCP**	Transmission Control Protocol
RIA	Routing Information Protocol	**Telnet**	Teletype Network
RIP	Rich Internet Applications	**TLS**	Transport Layer Security
RMI	Remote Method Invocation	**TOCTOU**	Time of Check/Time of Use
ROP	Return-Oriented Programming	**TVRA**	Threat Vulnerability and Risk Assessment
RPC	Remote Procedure Call	**UDID**	Unique Device Identifier
RWX	Read, Write, Execute	**UDP**	User Datagram Protocol
SaaS	Software as a Service	**UPnP**	Universal Plug and Play
SafeSEH	Safe Structured Exception Handling	**URI**	Universal Resource Identifier
SAML	Security Assertion Markup Language	**VM**	Virtual Machine
SAST	Static Analysis Security Testing	**VPN**	Virtual Private Network
SAX	Simple API for XML	**WAN**	Wide Area Network

© 2024 Black Duck

ACRONYM	DEFINITION	ACRONYM	DEFINITION
Web App	Web Application	**XHTML**	Extensible Hypertext Markup Language
WHATWG	Web Hypertext Application Technology Working Group	**XML**	Extensible Markup Language
Wifi	Wireless Fidelity	**XOR**	Exclusive OR
WLAN	Wireless Local Area Network	**XPath**	XML Path Language
WPA	Wi-Fi Protected Access	**XSD**	XML Schema Definition
WPA2	Wi-Fi Protected Access 2	**XSS**	Cross-Site Scripting
WPAD	Web Proxy Auto-discovery	**XXE**	XML External Entities
XEE	XML Entity Expansion	**ZAP**	Zed Attack Proxy
XHR	XML HTTP Request	**ZIP**	Zone Improvement Plan Code
XHR2	XML HTTP Request 2		

Notes